

Python Fundamentals: A Functional Approach

Vol - I

Dr. Goutam Majumder

Dr. Ganesh Khekare

Copyright © 2012 Author Name

All rights reserved.

ISBN:

DEDICATION

This book is dedicated to all the curious minds who find joy in the pursuit of knowledge and to the educators who light the path. May this text serve as a beacon for those embarking on their journey in computer science.

CONTENTS

	Acknowledgments	I
1	Introduction	1
2	Setting up and Getting Started with Python	13
3	Getting familiar with python	20
4	Operators	51
5	Strings	78
6	Branching and Looping	109
7	Exploring Python Collections	141
8	Functions	184

ACKNOWLEDGMENTS

The First author extends his heartfelt gratitude to their family and friends for their unwavering support and encouragement throughout the writing of this book. Special thanks to my friend Mr. Salil Batra, Assistant Prof., at the Department of Computer Science of Lovely Professional University for his valuable insights and feedback.

I am also immensely grateful to the students and educators in the field of computer science whose questions and experiences have greatly influenced this work. Their enthusiasm and dedication to learning have been a constant source of inspiration.

Lastly, I would like to acknowledge the contributions of the technical reviewers whose meticulous critiques have helped enhance the quality and accuracy of this text.

1 INTRODUCTION

Before we deep dive into the Pythonic content of this chapter, we must go back to the history of computer programming language or computer language. First, we need to know why a computer programming language is required. Although some of us already know the answer is very simple “**To automate a task**” or “**To reduce the workload of a human**”. For this, the first foundation of computer languages was started in the early 1940s and since then many programming languages have been designed with some specific purpose. The details of all computer languages other than Python are discussed in the first section of this chapter. The remaining part of the chapter belongs to Python and is sub-categorized as follows:

- The Section **Why Python?** highlights the intention of the Python developer along with language features.
- Next Section **Downside of Python** provides the real drawback of this language, as compared with other languages.
- Third Section **What Python can do?** highlights some of the important tasks, where Python is mostly used.
- The complete evaluation of Python language is highlighted in the next Section namely **Evaluation of Python**
- The most used Python libraries are listed in Section **Popular Python Libraries** so that any reader can target any specific library for his purpose of learning this language.
- Finally, the last Section **Python Implementations** includes the details of all the implementations of Python till today.

1.1 History of Computer Languages

Programming languages differ from natural languages in that natural languages are used to exchange information between humans, while programming languages are used by humans to interact with machines. Programming languages generally contain abstractions for defining and manipulating the data structures or controlling the flow of execution.

Most of the programming languages are text-based and mainly composed of two components syntax and semantics. Some of them are specified in a document called ISO and some are referenced such as Perl. The third category has a mix of two, with the basic language defined by a standard and extensions taken from the dominant implementation.

Early computers such as Colossus¹ were programmed without stored programs, simply by modifying

¹ https://en.wikipedia.org/wiki/Colossus_computer

their circuit or by setting banks of physical controls. After a few years, the first-generation (1GL) computer language was designed in the form of machine language, where each instruction is written in numeric form and executed the instructions in the hardware directly.

The second-generation languages (2GL) are also very close to machine architecture known as assembly language. These assembly languages are more human-readable form than 1GL and relieve the programmer of tedious and error-prone address calculations. In the 1950s between 1943 and 1945 the first high-level language Plankalkül was written by Konrad Zuse for the German Z3 computer.

In 1949 Khon Mauchly's Short Code² was the first high-level language developed for computers. Short Code statements are close to machine code represented in mathematical expression and much more in understandable form.

The first high-level compiled language Autocode was founded by Alick Glennie at the University of Manchester in 1952. A compiler is a system program used to translate a high-level instruction to machine language. The second auto code was designed by R. A. Brooker in 1954 known as Mark 1 Autocode. Both of these Autocodes were designed to work with Mark 1 computers only. The required auto code for the EDSAC³ 2 computer was founded by D. F. Hartley at the Mathematical Laboratory of the University of Cambridge. Similar to this a separate auto code for the Atlas 1 machine was founded at the University of Manchester.

The most popular general-purpose high-level programming language called FORTRAN⁴ was founded in 1954 at IBM by Jhon Backus. FORTRAN is still popular for high-performance computing and used for programs that benchmark and rank the world's fastest supercomputers.

Another early general purpose language FLOW-MATIC⁵ was founded by Grace Hopper and specifically designed to program in the first electronic computer UNIVAC I. Grace and her team members found it difficult to process business data related to customers using mathematical notations and in early 1955 they came up with specification for an English programming language and FLOW-MATIC become publicly available in 1959.

The increased use of high-level languages introduced another requirement in the programming paradigm. At this point in the early 1960s, there was a requirement for low-level system programming languages, which provided facilities to perform tasks that required direct access to hardware facilities with high-level control structures with error checking. Between the periods of 1960s to 1970s, most of the languages are now in use. Some of the present-era languages are inspired by them and some are limited in use. Table 1.1 provides a list of all such computer languages that exist at present along with their original purpose at the time of development and the programming paradigm they support.

Table 1.1 List of most popular programming languages with their purpose and programming paradigm

Language	Purpose	Paradigm	Year
Fortran	especially suited to numeric computation and scientific computing	Multi-paradigm: structured, imperative (procedural, object-oriented), generic, Array	1957
Lisp	Popular for artificial intelligence research	functional, procedural, reflective, meta	1960
APL	describing procedures in the processing		

² [https://en.wikipedia.org/wiki/Short_Code_\(computer_language\)](https://en.wikipedia.org/wiki/Short_Code_(computer_language))

³ <https://en.wikipedia.org/wiki/EDSAC>

⁴ <https://en.wikipedia.org/wiki/FORTRAN>

⁵ <https://en.wikipedia.org/wiki/FLOW-MATIC>

	of information		
PASCAL	Used to develop dynamic and recursive data structures	Imperative and structured	1970
Shell script	designed to be run by a Unix shell, a command-line interpreter.	Scripted event	1971
Prolog	Used for logic building in artificial intelligence along with theorem proving, expert systems, term rewriting, type systems, and automated planning.	Supports only logic building	1972
C	use in operating systems, device drivers, and protocol stacks, though decreasingly for application software.	Multi-paradigm as procedural and structured	1972
SQL	The domain-specific language used in RDBMS for data management and manipulating	Supports declarative paradigm	1974
C++	Mostly used for system programming and embedded systems, resource-constrained software, and large systems in mind, with performance, efficiency, and flexibility.	Supports procedural, imperative, functional, object-oriented, generic, modular	1979
Ada	Originally designed for embedded and real-time systems.	Supports structured, imperative, object-oriented, aspect-oriented, concurrent, array, distributed, generic, procedural, meta	1980
MATLAB	Mostly used for numeric computing. It allows the design of user interface and plotting of data as well.	functional, imperative, procedural, object-oriented, Array	1984
J	Is an array programming language mostly used for mathematical and statistical programming suitable for matrices operations.	Supports functional programming via its tactic programming feature. Also be used for both object and prototype-based programming.	1990
Python	Discussed later in this Chapter	object-oriented, procedural, functional, structured, reflective	1991
R	Mostly used for data mining, bioinformatics, statistics for data analysis	procedural, object-oriented, functional, reflective, imperative, Array	1993
HTML	For web development for webpage design	Markup language	1993
Java	It is a compiled language designed to write once and run anywhere. Mainly used for client-server web applications.	structured, imperative, object-oriented, aspect-oriented, concurrent, array, distributed, generic, procedural, meta	1995

PHP	Used for Web development	Supports imperative, object-oriented, procedural, reflective	Multi-paradigm: functional, object-oriented	1995
JavaScript	Mostly used for web development along with HTML and CSS	Supports event-driven, procedural, programming	functional, object-oriented	1995
Ruby	It was designed with an emphasis on programming productivity and simplicity.	functional, imperative, object-oriented, reflective		1995
C#	It was designed for developing software components suitable for deployment in distributed environments.	structured, imperative, object-oriented, event-driven, task-driven, functional, generic, reflective, concurrent		2000
Scala	It was designed as scalable and designed to grow with the demand of users.	concurrent, functional, imperative, object-oriented		2004
Go or Golang	Similar to C but with memory safety, garbage collection, structural typing, and CSP-style concurrency.	concurrent imperative, object-oriented		2009
Rust	Popular for system programming also offers features for functional programming	concurrent, functional, imperative, structured	generic, generic	2010
TypeScript	It is a superset of JavaScript and intended to be designed for the development of large applications and transpires into JavaScript	functional, generic, imperative, object-oriented		2012
Julia	Well-suited for numeric analysis and computational science	multiple dispatch (primary paradigm), procedural, functional, meta, multistage		2012

This list is taken from IEEE Spectrums⁶, which ranked top performing programming languages. At the time of writing this book some new languages are added or removed from this list. The key point that we need to note down from the evaluation of programming languages is, that every language needs to evolve with time and must add some unique programming features as compared to existing languages with rich library support.

Most important we need to note that at the time of writing this chapter Python is the most trading language followed by Java, C, JavaScript, and C++ as the top five. As of now, SQL is leading the Job market followed by Java, Python, JavaScript, and C#. I hope that as a writer of this book, I can answer this question why Python? Explained in the next Section.

Being an instructor for programming languages over the past 10 years my suggestion for all the readers is to go for a Python computer language with a goal and if you find the answer to why then it's time to start learning that language by writing small programs. Remember any computer language can't be fitted in just one book or syllabus, you have to go beyond that.

⁶ <https://spectrum.ieee.org/>

1.2 Why Python?

The developer intended to develop a Computer Programming Language with a focus on clean syntax. Towards this goal, Van Rossum, the founder of Python launched Computer Programming for Everybody (CP4E), at the time of stay at CNRI. His main intention was to make programming more accessible the people with the knowledge of literacy just like English literacy and Mathematics skills required.

Over the years language Python language has evolved and covers all multi-paradigm programming languages. As of now Python fully supports Object Oriented Programming (OOP) and Structured Programming and through extension, it supports design by contract and logic programming also.

The developer was influenced by Zen of Python⁷ which is a collection of 19 programming guidelines written by the software developer Tim Peters. He first wrote these 19 rules and mailed them to the Python mailing list in 1999. Since then, the original developer Guido van Rossum fulfilled the following principles, to make this language easy to understand. Such as: (a) Beautiful is better than ugly; (b) Explicit is better than implicit; (c) Simple is better than complex; (d) Complex is better than complicated; (e) Readability counts.

Python's versatility is difficult to match, and it's so flexible that it encourages experimentation. This keeps programmers from being pigeonholed into only building one type of application. You can learn just one language and use it to make new and different things.

Computer Weekly calls Python the most versatile programming language, noting that “Although there might be a better solution for any given problem, Python will always get the job done well” [1]. Other advantages of Python include:

- **Platform independent:** Like Java, you can use Python on various platforms, including macOS, Windows, and Linux. You'll just need an interpreter designed for that platform.
- **Fast development:** Because Python is dynamically typed, it's fast and friendly for development. Additionally, it uses asynchronous code to tackle situations and challenges faster because each unit of code runs separately.
- **Offers extensive libraries:** Its large library supports common tasks and commands. It also contains code that can be used for many different purposes, ranging from generating documentation to unit testing to CGI.
- **More flexible:** Python supports a variety of programming styles and has multiple paradigms. Not only is this optimal for programmers who enjoy flexibility, but it also makes it ideal for start-ups that might need to shift approaches abruptly.
- **Free and open-source:** You can download Python without any cost, and because it's so easy to learn and boasts one of the largest and most active communities—you should be able to start writing code in mere minutes.
- **Boost productivity:** NetGuru says that Python is more productive than Java because of how concise it is and because it's dynamically typed. Additionally, it has control capabilities and integration features that can make applications more productive [2].

1.3 Downside of Python

While Python is arguably one of the easiest and fastest languages to learn, it's also decidedly slower to execute because it's a dynamically typed, interpreted language, executed line-by-line. Python does extra work while executing the code, making it less suitable for use in projects that depend on speed. However, if speed isn't a sensitive issue, Python's slower nature won't likely be a problem. Other potential disadvantages include:

⁷ https://en.wikipedia.org/wiki/Zen_of_Python

- **It only executes one thread at a time:** Python has a Global Interpreter Lock that only lets one thread execute at a time, so if you're working on a multi-threaded CPU-bound program, it'll likely be even slower. Using multiprocessing programs instead of multithreaded programs can be an effective workaround.
- **It isn't mobile native:** Python can be effectively and easily used for mobile purposes, but you'll need to put a bit more effort into finding libraries that give you the necessary framework. Some examples include Kivy, which lets you use the same API to create mobile apps and software that you can run on Raspberry PI, Linux, and Windows.
- **It uses a large amount of memory:** If you're working on a project where many objects are active in RAM, this could present an issue for you. Switching to NumPy could be an effective workaround to reduce the amount of memory Python uses for each object.

1.4 What Python can do?

- **Automation:** it describes a wide range of technologies that reduce human effort by predetermining decision criteria, sub-process relationships, and related actions. In automation, Python is used for email automation such as a system engineer who wants to inform all the project members about the system maintenance. There are many popular libraries written in Python for these purposes namely PyAutoGUI, Pywinauto.
- **Data Analytics:** is a process of understanding and getting information from data of previous sales or advertisements. It is used to find meaningful patterns in data used for interpretation and communication purposes. Understanding user experience and sales are two key factors in areas rich with recorded information. Many packages such as PandasGUI, D-Tale, and Mito are the most popular packages used by data analytics engineers.
- **Databases:** relational databases like MySQL, SQLite, and Oracle are used for efficient data storage and management for any organizational growth. Each database requires a specific connector library to be installed in your Python environment to interact with it. Popular examples include sqlite3 for SQLite, mysql-connector-python for MySQL, and psycopg2 for PostgreSQL.
- **GUI:** also known as Graphical User Interface programming, involves creating applications with a visual interface that users can interact with through elements like buttons, text boxes, menus, and windows. This makes the software more user-friendly and accessible compared to command-line interfaces. Most popular Python libraries that offer cross platform developments are Tkinter, wxPython, PyQt.
- **Image Processing:** it refers to the process of enhancing the quality of an image or extracting information from an image. Any image processing tool offers a collection of computer algorithms to work with digital images. It has a wide range of applications such as Medical imaging, remote sensing, computer vision, security and surveillance. Python packages like scikit-image, OpenCV, and Pillow offer a great bunch of image processing algorithms to work with digital images.
- **Machine Learning and Text Processing:** there are two sub-domains of Artificial Intelligence (AI) and nowadays any application with AI requires machine learning algorithms to make any decision. Text processing refers to the automated manipulation and analysis of electronic text. It involves transforming raw text data into a more usable format for various purposes, like: extracting information, cleaning and preparing data, summarizing text, classifying text and many more. Packages like NLTK, spaCy, Gensim, offer many text processing algorithms and scikit-learn, PyTorch, TensorFlow are preferable for machine learning and deep learning algorithms.
- **Web Scrapping:** also known as web harvesting or extracting data from web. Its application includes price comparison, market research, data analysis and content aggregation. The most popular Python libraries that offer web scrapping tools are Requests, BeautifulSoup, Selenium and Scrapy.

1.5 Evaluation of Python

The language name Python was named after the BBC TV show Monty Python's Flying Circus. In the late 1980s, the idea of Python Development was conceived by Guido van Rossum at Centrum Wiskunde & Informatica (CWI) in the Netherlands as a successor to the ABC programming language. Its implementation began in December 1989. Since then, Guido van Rossum carried the sole responsibility as Python dictator as a lead developer until 12 July 2018. In 2019 Python core developers elected a five-member group to lead future Python development.

The first version of Python was released in 1991 and codes were published at alt. sources. At this stage of development language already includes many important features such as classes with inheritance, exception handling, function, and the core datatypes list, dict, str, and so on. Initially, all the codes released in the module (about the module we have written a chapter) were inspired by Modula-3 language. Finally, in 1994 the primary discussion forum comp.lang.python was formed marking a milestone in the growth of Python's userbase.

To form the BeOpen Python Labs team all the core development team members moved to BeOpen.com in 2000. The initial development is carried out in CNRI and requested to release version 1.6 which has all the Python development at that point. Consequently, the release schedules for 1.6 and 2.0 had a significant amount of overlap. After the release of Python 2.0, all the Python Labs developers joined Digital Creations. Over the years Python goes through various releases and Table 1.2 highlights the details of all the previous and recent releases.

Table 1.2 Details of Python Releases with various versions and dates

Version	Date of Release	End of full support	End of security fixes
0.9	20-02-1991		29-07-1993
1.0	26-01-1994		15-02-1994
1.1	11-10-1994		10-11-1994
1.2	13-04-1995		Unsupported
1.3	13-10-1995		Unsupported
1.4	25-10-1996		Unsupported
1.5	03-01-1998		13-04-1999
1.6	05-09-2000		September 2000
2.0	16-10-2000		22-06-2001
2.1	15-04-2001		09-04-2002
2.2	21-12-2001		30-05-2003
2.3	29-06-2003		11-03-2008
2.4	30-11-2004		19-12-2008
2.5	19-09-2006		26-05-2011
2.6	01-10-2008	24-08-2010	29-10-2013
2.7	03-07-2010		01-01-2020
3.0	03-12-2008		27-06-2009
3.1	27-06-2009	12-06-2011	06-04-2012
3.2	20-02-2011	13-05-2013	20-02-2016

3.3	29-09-2012	08-03-2014	29-09-2017
3.4	16-03-2014	09-08-2017	18-03-2019
3.5	13-09-2015	08-08-2017	30-09-2020
3.6	23-12-2016	24-12-2018	23-12-2021
3.7	27-06-2018	27-06-2020	27-06-2023
3.8	14-10-2019	03-05-2021	Oct-2024
3.9	05-10-2020	17-05-2022	Oct-2025
3.10	04-10-2021	June-2024	Oct-2027
3.11	24-10-2022	June-2025	Oct-2028

Python 2.7 was the last release in 2.x series. In November 2014, it was announced that till 2020 the supports were provided and the developers are encouraged to move 3.0 as early as possible. Python 2.7 support ended on January 1, 2020, along with code freeze of 2.7 development branch. A final release, 2.7.18, occurred on April 20, 2020, and included fixes for critical bugs and release blockers. This marked the end of the life of Python 2.

Python 3.0 also known as Python 3000 or Py3K comes with the most necessary changes required in the language. At the time of release, it was not easy to retain the full backward compatibility with 2.x series. So, it broke the compatibility with Python 2 and much Python 2 code does not run unmodified on Python 3. Python supports dynamic typing and combines with the plan to change the semantics of certain methods, making it difficult for mechanical translation from 2.x to 3.0. Some of the translations are done automatically using a tool called 2to3.

Before the roll-out of Python 3.0, projects running in 2.x need to provide two different releases of all such projects. One version is completely written in 2.x for those systems running in Python 2.x and for the systems running in 3.x need to produce a release using 2to3.

1.6 Popular Python Libraries

Python's large standard library provides tools suited for many tasks and is commonly cited as one of its greatest strengths. It includes modules for creating graphical user interfaces, connecting to relational databases, generating pseudorandom numbers, arithmetic with arbitrary-precision decimals, manipulating regular expressions, and unit testing.

As of 14 November 2022, the Python Package Index (PyPI), the official repository for third-party Python software, contains over 415,000 packages with a wide range of functionality. Out of these here we have just listed top Python modules in Table 1.3 used for desktop applications, database operations, game development, web scraping, image processing, data science, machine learning, and natural language processing.

Table 1.3 List of Top ranked Python modules used for various purposes

Module	Used for	Web Address
PyQt5	Most popular Python framework for GUI.	https://riverbankcomputing.com/software/pyqt/intro
Tkinter	Another GUI toolkit comes as a pre-installed	https://wiki.python.org/moin/TkInter
wxPython	Cross-platform GUI Toolkit	https://wxpython.org/
PyGObject	Provides binding for GObject-based libraries such as GTK, GStreamer,	https://pygobject.readthedocs.io/en/latest/

	WebKitGTK, GLib, GIO, and many more	
Pmw	Used for building high-level compound widgets in Python using the Tkinter module.	https://pmw.sourceforge.net/doc/index.html
Tix	Tk Interface eXtension is a powerful set of user interface components that expands the capabilities of your Tcl/Tk and Python applications.	https://tix.sourceforge.net/
Libavg	It is widely considered one of the best libraries for developing user interfaces for modern touch-based devices, and its hardware acceleration is achieved through OpenGL and GPU shaders.	https://www.libavg.de/site/
PyForms	Python implementation of Windows Forms, which enables developers to create highly interactive interfaces for Windows GUI mode, Web mode, and Terminal mode.	https://pyforms.readthedocs.io/en/v4/
Pygame	set of Python modules designed for writing video games.	https://www.pygame.org/wiki/about
PYGLET	cross-platform windowing and multimedia library for Python	https://pyglet.org/
PyOpenGL	is the most common cross-platform Python binding to OpenGL and related APIs.	https://pyopengl.sourceforge.net/
MySQLdb	Used to working with MySQL database	https://sourceforge.net/projects/mysql-python/
PyGreSQL	Interface to connect with PostgreSQL database.	http://www.pygresql.org/about.html
Gadfly	A relational database implemented in Python	https://gadfly.sourceforge.net/
SQLAlchemy	SQL toolkit and Object Relational Mapper that gives application developers full power.	https://www.sqlalchemy.org/
KInterbasDB	Supports open-source relational database Firebird	https://kinterbasdb.sourceforge.net/
BeautifulSoup	Used for web scraping and converts anything to Unicode	https://www.crummy.com/software/BeautifulSoup/
Scrape	Used for web scraping	http://zesty.ca/python/scrape.html
mechanize	Stateful programmatic web browsing in	https://wwwsearch.sourceforge.net

	Python	/mechanize/
Scrapy	Used to extracting the data that you need from websites	https://scrapy.org/
Pyquery	allows you to make jquery queries on xml documents	https://pythonhosted.org/pyquery/
Selenium	is a suite of tools used to automate a browser	https://www.selenium.dev/
Requests	Requests allows you to send HTTP/1.1 requests extremely easily.	https://pypi.org/project/requests/
Googlemaps	Provides google map API services	https://pypi.org/project/googlemaps/
OpenCV	Used for computer vision and machine learning with artificial intelligence	https://opencv.org/
Scikit-Image	Image processing library	https://scikit-image.org/
Scipy	Used for mathematical and scientific computation	https://scipy.org/
NumPy	Library for numerical analysis	https://numpy.org/
matplotlib	Specialized for 2-d plots as a multi-platform data visualization on NumPy arrays	https://matplotlib.org/
TensorFlow	Used for deep learning and data science projects, developed by the Google Brain team	https://www.tensorflow.org/
Pandas	Widely used Python libraries for Data Science	https://pandas.pydata.org/
Scikit-learn	The machine-learning library provides a variety of useful machine-learning algorithms, and it is designed to be interpolated into SciPy and NumPy.	https://scikit-learn.org/stable/
Keras	Used for neural network modules, similar to TensorFlow	https://keras.io/
PyTorch	Created by Facebook AI for scientific computing package relies on the power of GPU.	https://pytorch.org/
Theano	A numerical computation Python library specifically developed for machine learning and deep learning.	https://github.com/Theano/Theano no
Microsoft CNTK	The open-source deep-learning library is used to implement distributed deep learning and machine learning tasks.	https://learn.microsoft.com/en-us/cognitive-toolkit/
NLTK	Best python library for Natural Language Processing	https://www.nltk.org/

SpaCy	SpaCy enables developers to create applications that can process and understand huge volumes of text	https://spacy.io/
Genism	The fastest library for training of vector embeddings.	https://radimrehurek.com/gensim/
Bertnlp	performs various NLP tasks using Bidirectional Encoder Representations from Transformers (BERT) related models.	https://pypi.org/project/bertnlp/
TextBlob	Used for processing textual data. It provides a simple API for diving into common NLP tasks such as part-of-speech tagging, noun phrase extraction, sentiment analysis, classification, translation, and more.	https://textblob.readthedocs.io/en/dev/
PyNLPI	PyNLPI can be used for basic tasks such as the extraction of n-grams and frequency lists, and to build a simple language model.	https://pynlpl.readthedocs.io/en/latest/
Pattern	Web mining module for python	https://github.com/clips/pattern
Polyglot	natural language pipeline that supports massive multilingual applications	https://polyglot.readthedocs.io/en/latest/
vaderSentiment	is a lexicon and rule-based sentiment analysis tool that is specifically attuned to sentiments expressed in social media, and works well on texts from other domains.	https://pypi.org/project/vaderSentiment/
pySonic	is a Python wrapper around the high performance, cross platform FMOD sound library.	https://pysonic.sourceforge.net/
pyMedia	is a Python module for wav, mp3, ogg, avi, divx, dvd, cdda etc files manipulations.	http://pymedia.org/

1.7 Python Implementations

CPython was the first and referenced implementation written using C89 standards. Later Python 3.11 uses C11 with several C99 features. CPython compiles every Python program into intermediate bytecode and executed by its virtual machine. Bytecode also called portable code is a form of instruction set used for efficient execution by interpreter. CPython is distributed with large set of libraries mostly written in C and native Python and publicly available for many platforms such as Windows and most modern UNIX like systems. Python have many other implementations, includes:

- PyPy is a fast, compliant interpreter of Python 2.7 and 3.8 comes with just-in-time compiler for speed improvement over CPython [3].

- Stackless Python is a significant fork of CPython that implements microthreads; it does not use the call stack in the same way, thus allowing massively concurrent programs [4].
- For microcontroller including Lego Mindstorms EV3, Micro Python and Circuit Python are the two Python 3 variants [5].
- Pyston is a variant of the Python runtime that uses just-in-time compilation to speed up the execution of Python programs [6].
- Cinder is a performance-oriented fork of CPython 3.8 that contains a number of optimizations including bytecode inline caching, eager evaluation of coroutines, a method-at-a-time JIT, and an experimental bytecode compiler.

In addition to the above list, it also has some other implementations designed as cross compiler to other languages. These includes:

- Brython⁸, Transcrypt⁹ and Pyjs compile Python to JavaScript.
- Cython compiles (a superset of) Python 2.7 to C and the resulting code is also usable with Python 3.
- Nuitka¹⁰ compiles Python into C.
- Numba¹¹ uses LLVM to compile a subset of Python to machine code.
- Pythran¹² compiles a subset of Python 3 to C++11 only.
- RPython¹³ can be compiled to C and is used to build the PyPy interpreter of Python.
- MyHDL¹⁴ is a Python-based hardware description language (HDL), that converts MyHDL code to Verilog or VHDL code.

These are some older implementations that didn't support any of the Python 3.x syntax. Such as:

- Google's Grumpy¹⁵ with latest release in 2017, translates Python 2 to Go.
- IronPython¹⁶ allows running Python 2.7 and 3.4 programs on the .NET Common Language Runtime.
- Jython¹⁷ compiles Python 2.7 to Java bytecode, allowing the use of the Java libraries from a Python program.
- Pyrex¹⁸ and Shed Skin compile to C and C++ respectively.

*****END OF CHAPTER*****

⁸ <https://brython.info/>

⁹ <https://www.transcrypt.org/>

¹⁰ <https://nuitka.net/>

¹¹ <https://numba.pydata.org/>

¹² <https://pythran.readthedocs.io/en/latest/>

¹³ <https://pypi.org/project/rpython/>

¹⁴ <https://myhdl.org/>

¹⁵ <https://github.com/google/grumpy>

¹⁶ <https://ironpython.net/>

¹⁷ <https://www.jython.org/>

¹⁸ <https://www.csse.canterbury.ac.nz/greg.ewing/python/Pyrex/>

2 SETTING UP AND GETTING STARTED WITH PYTHON

This chapter shows the required steps to make your system ready and to write Python scripts. Sections of this chapter are segregated as follows:

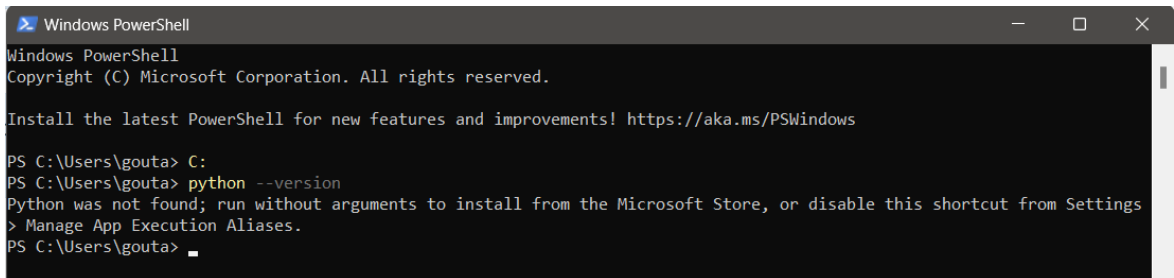
- First check your system comes with any version of Python
- Install or update Python on Windows, macOS, and Linux
- Use Python on mobile devices like phones or tablets
- Use Python on the Web with online interpreters

2.1 Installing Python on Windows

Before start to install, it is better to check if you already have Python on your Windows machine, first open a command-line application, such as PowerShell.

- Press the Windows  key
- Type PowerShell
- Press Enter Key 

Instead of PowerShell alternately you can also use cmd.exe or windows Terminal. With the command line open, type in the following command and press Enter :



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\gouta> C:
PS C:\Users\gouta> python --version
Python was not found; run without arguments to install from the Microsoft Store, or disable this shortcut from Settings
> Manage App Execution Aliases.
PS C:\Users\gouta> _
```

It seems Python is not available in your system, and you can find three ways to install the official Python distribution on Windows:

- Microsoft Store package: The most straightforward installation method on Windows involves

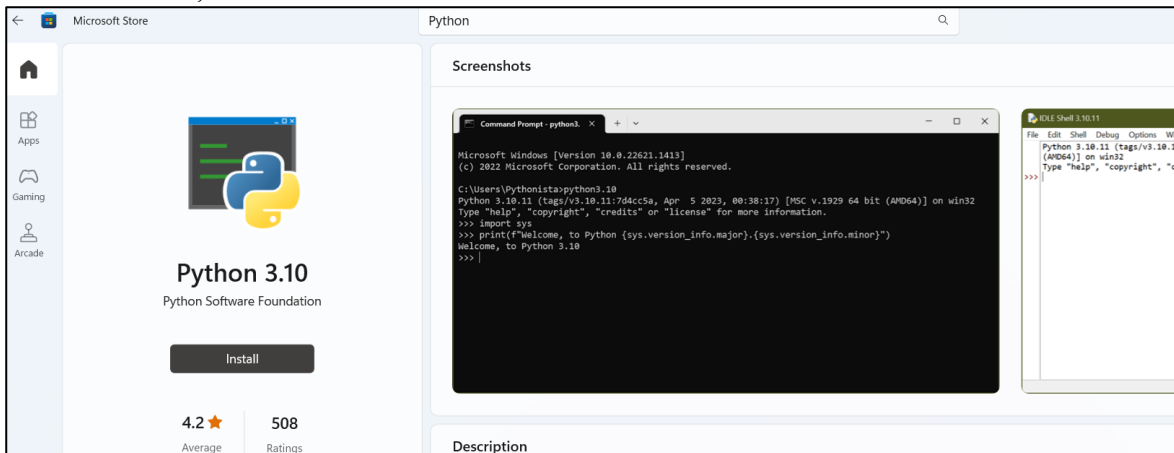
installing from the Microsoft Store app. This is recommended for beginner Python users looking for an easy-to-set-up interactive experience.

- Full Installer: This approach involves downloading Python directly from the Python.org website. This is recommended for intermediate and advanced developers who need more control during the setup process.

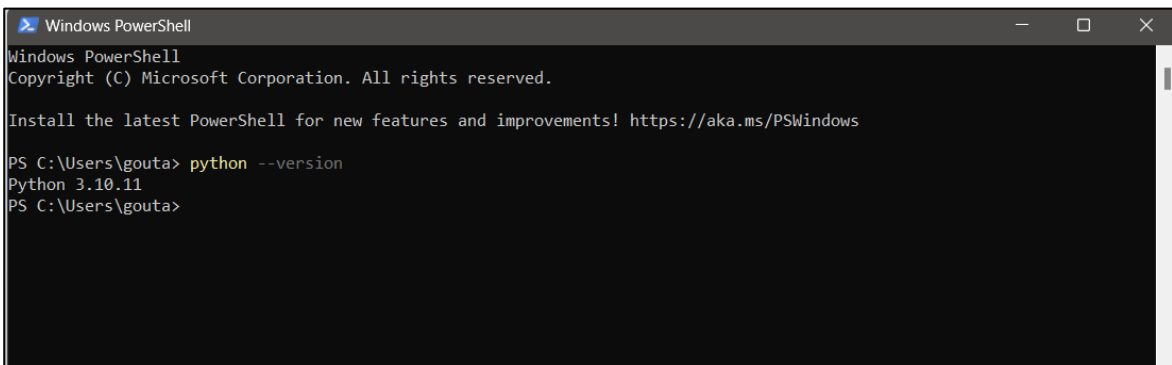
2.1.1 Installing from Microsoft App Store

If you're new to Python and looking to get started quickly, then the Microsoft Store package is the best way to get up and running without any fuss. You can install from the Microsoft Store in two steps.

- Open the Microsoft App Store, click  from Taskbar or alternately type python in PowerShell and press Enter ↵. The Microsoft Store will automatically launch and take you to the latest version of Python in the store. Select the most latest version that you can see or at least Python ver. 3.10, as all illustrations are tested on this version.



- Install the Python App and to do that click over **Install**. You can see downloading is start and wait for it. Make sure that the Python application you've selected is created by the **Python Software Foundation**.
- To check Python is installed or not type the given command **python --version** in the PowerShell. You can see the installed version, if installation is completed.



2.1.2 Installing from Full Installer

Follow the following steps:

- Open a browser window and navigate to the Python.org Downloads page for Windows.
- Under the **Download Tab** select **Windows** and till the writing of this chapter you can see latest stable version is 3.12.2. If you want any specific version, check the right side of the page as **Pre-releases heading**.
- Now select either Windows Installer 64-bit executable installer for 64-bit or Windows Installer 32-bit for 32-bit processor architecture.

You can also complete the installation on Windows using alternative distributions, such as Anaconda, but this chapter covers only official distributions. Anaconda¹⁹ is a popular platform for doing scientific computing and data science with Python. To download you can visit anaconda download page <https://www.anaconda.com/download>.

2.2 Installing Python in Ubuntu

In this section, you'll learn how to install Python using Ubuntu's apt package manager. Depending on the version of the Ubuntu distribution you run, the process for setting up Python on your system will vary. You can determine your local Ubuntu version by running the following command:

```
$ lsb_release -a
```

No LSB modules are available.

Distributor ID: Ubuntu

Description: Ubuntu 16.04.4 LTS

Release: 16.04

Codename: xenial

Follow the instructions below that match the version number you see under Release in the console output. Ubuntu 18.04, Ubuntu 20.04 and above, doesn't come by default Python 3.8, but it is available in the Universe repository. To install version 3.8, open a terminal application and type the following commands:

```
$ sudo apt-get update
```

```
$ sudo apt-get install python3.8 python3-pip
```

Once the installation is complete, you can run Python 3.8 with the python3.8 command and pip with the pip3 command.

2.3 Installing Python on macOS

Like Windows, in macOS also you can install it from direct installer by navigate to the Python.org Downloads page for macOS. Select any specific version under Pre-releases heading and download and install it. Before that, it is better to check any existing installer in available or not. To check which Python version you have on your Mac, first open a command-line application, such as Terminal. To open the

¹⁹ <https://www.anaconda.com/>

Terminal, follow the steps given below:

- Press the  **Cmd** + **space** keys.
- Type Terminal
- Press Enter  key.

Alternatively, you can open Finder and navigate to Applications → Utilities → Terminal. With the command line open, type in the following commands:

```
# Check the system Python version
$ python --version
# Check the Python 2 version
$ python2 --version
# Check the Python 3 version
$ python3 --version
```

If you have Python on your system, then one or more of these commands should respond with a version number.

2.4 Online Python Interpreters

If you want to try out the examples in this tutorial without setting up Python on your machine, then there are several websites that offer an online Python interpreter:

- Python.org Online Console
- Repl.it
- Python Fiddle
- Trinket
- Python Anywhere

These cloud-based Python interpreters may not be able to execute some of the more complex examples in this tutorial, but they're adequate for running most of the code and may be a nice way to get started. More information on using these sites is presented in the next tutorial in this series.

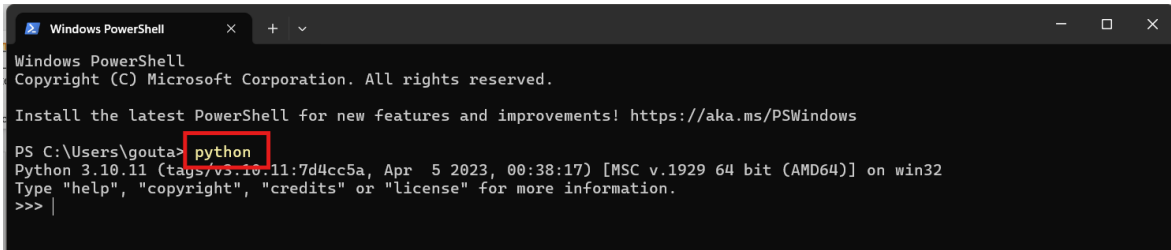
2.5 Executing Python Scripts

There are many ways you can execute a python script; these are as follows:

2.5.1 Executing Python Command using command line

Type PowerShell in windows search bar and open it as Administrator Mode

Then type python

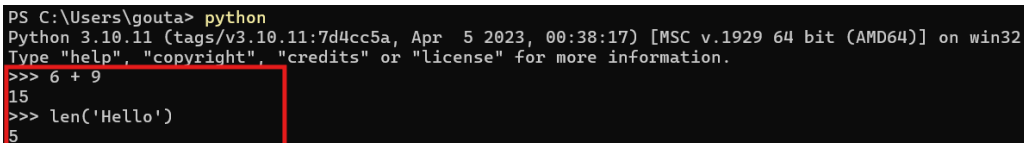


```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\gouta> python
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Now you can type any python command in the command prompt (>>>) and check the correct outcome, as shown below:



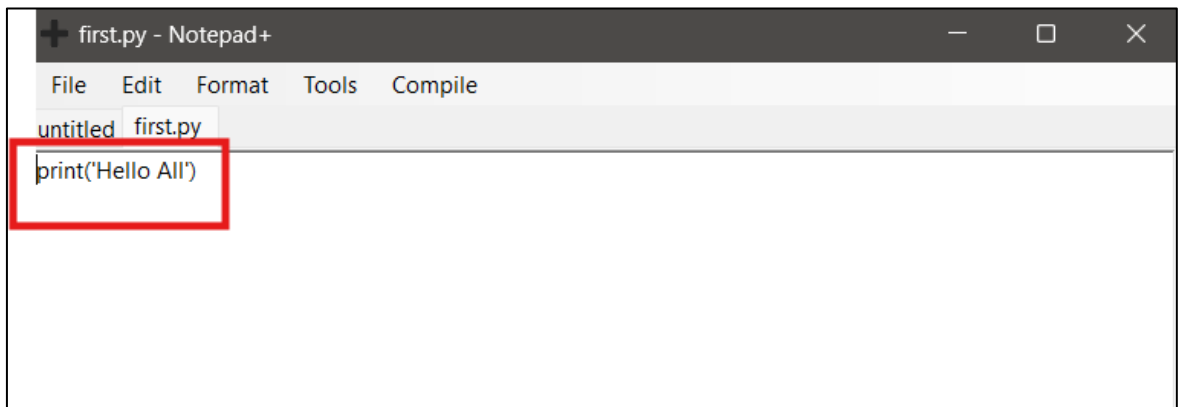
```
PS C:\Users\gouta> python
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> 6 + 9
15
>>> len('Hello')
5
```

2.5.2 Executing Python Script file using Command Line

You can use any text editor for writing your script and save the file with **.py** extension.

Let's assume that we have save the file a **first.py** in inside a folder **Python code** of **D:** drive.

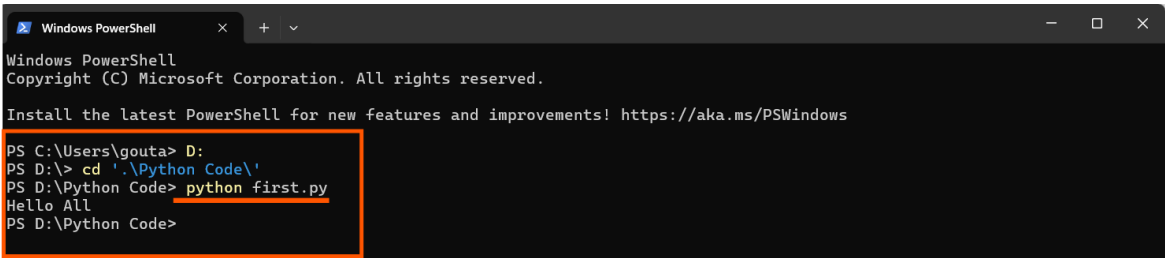
Now open any code editor such as Notepad++ and type python script and save it in location as **D:\Python code**



```
first.py - Notepad+
File Edit Format Tools Compile
untitled first.py
print('Hello All')
```

Now open PowerShell and set the path of the PowerShell to the desired location.

Now type the command as **python filename.py** in our case **python first.py**



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

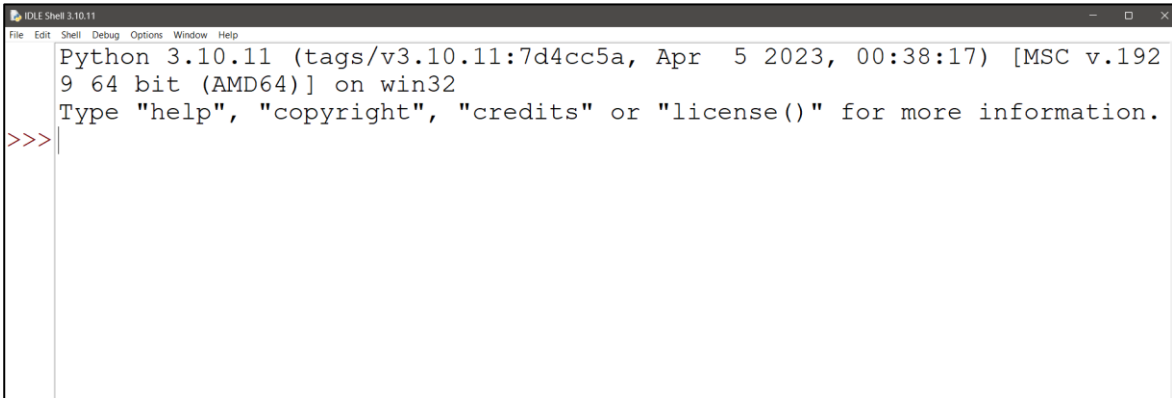
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\gouta> D:
PS D:\> cd '.\Python Code\'
PS D:\Python Code> python first.py
Hello All
PS D:\Python Code>
```

2.5.3 Executing Python Scripts using IDLE

If you follow the guide of python installation as discussed in Section 2.1 you can use Python Integrated Development and Learning Environment (IDLE) to write and execute any python script. Follow the steps given below:

Open the IDLE by clicking over the IDLE icon or search using windows search bar. Once it is open it will be look like this:



```
IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.192
9 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
```

You can directly type any python command after the Python command line (>>>), as shown below:



```
>>> a = 8
>>> b = 9
>>> a+b
17
>>>
```

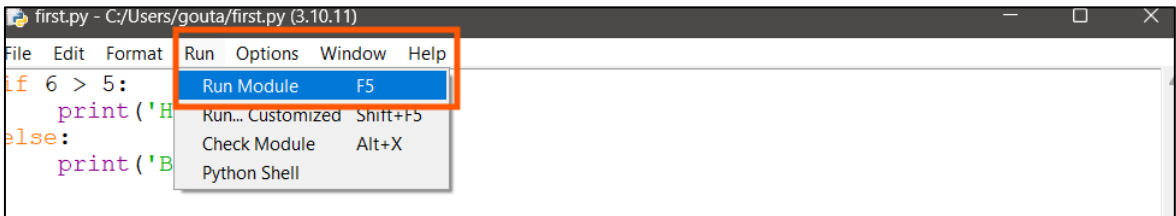
If you are learning any programming language for the first time, it is better to write python command through script file. To open a new script file, you can open it by clicking over **File** menu -> **New File**.



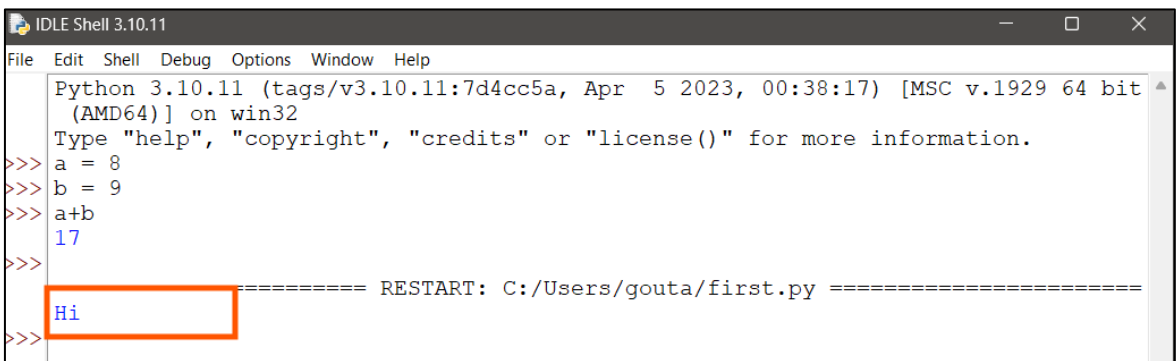
```
first.py - C:/Users/gouta/first.py (3.10.11)
File Edit Format Run Options Window Help
if 6 > 5:
    print('Hi')
else:
    print('Bye Bye')
```

Save the file and save it in your preferred location.

To execute the script Click **Run**→ **Run Module** or **Press F5** and output will display in the IDLE window.



```
first.py - C:/Users/gouta/first.py (3.10.11)
File Edit Format Run Options Window Help
if 6 > 5:
    print('Hi')
else:
    print('Bye Bye')
```



```
IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> a = 8
>>> b = 9
>>> a+b
17
>>> Hi
>>>
```

*****END OF CHAPTER*****

3 GETTING FAMILIAR WITH PYTHON

Before start writing extensive Python scripts, this chapter gently helps you to understand the importance of basic Python programming components. It starts with the explanation and limitations of a pre-defined set of words called **keywords**. Although we have made this discussion very limited about the keywords you will get to know more about them according to their need in later chapters.

After that, we have explained two other programming tools that are important for documentation purposes such as **Naming Conventions** for identifiers and **Commenting** a Python script. Mostly these two tools are required by the Python developer which improves code readability and acceptance among other developers.

The next section describes **Python Types** deeply discussed various data types also known as Objects either as pre-defined or created with their uses. Later in the Sections **Object Types, Input and Output** and **Variable Assignments** discussed how to use these types either with directly assigned values or by getting values from the user as input and producing output by manipulating them.

Finally, the last Section highlights **Built-in Functions** comes with python interpreter. In this chapter based on the necessity, we have also imported some of the modules in code examples using import statement. But we have not discussed much about importing a module in this Chapter, rather we keep it separately in Chapter 8.

3.1 Keywords

Every programming language has special reserved words, or keywords, that have specific meanings and restrictions around how they should be used. Python is no different. Python keywords are the fundamental building blocks of any Python program.

Python keywords are special reserved words that have specific meanings and purposes and can't be used for anything but those specific purposes. These keywords are always available—you'll never have to import them into your code.

Python keywords are different from Python's built-in functions and types. The built-in functions and types are also always available, but they aren't as restrictive as the keywords in their usage.

An example of something you can't do with Python keywords is assign something to them. If you try, then you'll get a `SyntaxError`. You won't get `SyntaxError`, if you try to assign something to a built-in function or type, but it still isn't a good idea. As of Python 3.10, there are thirty-five keywords in Python. A list of keywords is given in Table 3.1.

Table 3.1 List of Keywords in Python 3.10

False	Class	from	or
None	Continue	global	pass
True	Def	If	raise
And	Del	import	return
As	Elif	In	try
assert	else	Is	while
async	except	lambda	with
await	finally	nonlocal	yield
break	For	not	

The list of keywords is also retrieved using help in-built functions by passing 'keywords' as a parameter to it. Consider the code snippet given below:

```
>>>help('keywords')
```

`help()` function can be used to obtain more information about any keyword, by passing that keyword as argument to it, as follows:

```
>>> help('pass')
```

```
The "pass" statement
*****

    pass_stmt ::= "pass"

"pass" is a null operation — when it is executed, nothing happens. It
is useful as a placeholder when a statement is required syntactically,
but no code needs to be executed, for example:

    def f(arg): pass      # a function that does nothing (yet)

    class C: pass        # a class with no methods (yet)
```

Python offers a module namely keyword to check a given string is a keyword or soft keyword (added in Python 3.10). Soft keywords are only reserved for specific contexts and identifiers **match**, **case** and **_** (underscored sign) can syntactically act as a keyword's context related to pattern matching. Table 3.2 highlights the details of keyword module.

Table 3.2 Methods and Constants defines in the keyword module

Methods	and	Description
Constants		
iskeyword(s)		Return True if <i>s</i> is a Python keyword.
kwlist		Return a sequence containing all the keywords defined for the interpreter.
issoftkeyword(s)		Return True if <i>s</i> is a Python soft keyword.
softkwlist		Return a sequence containing all the soft keywords defined for the interpreter.

The import statement is necessary to use any of these methods and constants. See the code sample

given below:

```
>>>import keyword
>>>keyword.softkwlist
['_', 'case', 'match']
>>>keyword.iskeyword('is')
True
>>>keyword.kwlist
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break',
'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from',
'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass',
'raise', 'return', 'try', 'while', 'with', 'yield']
>>>keyword.issoftkeyword('match')
True
```

3.2 Naming Convention

These are set of rules for choosing the character sequence to be used for the naming of identifiers such as variables, constants, types, functions, and other entities in source code and documentation. For the following reasons naming conventions are followed by the programmers:

- to reduce the effort to understand the source code and to avoid naming collisions of the identifiers.
- to enable code reviews to focus on issues rather than focusing on the syntax and naming standards.
- to provide additional information (i.e., metadata) about the use to which an identifier is put.
- to promote consistency within a development team, so that each one can understand other's code.
- to enable the use of automated refactoring or search and replace tools with minimal potential for error handling.
- to provide a better understanding in case of code reuse after a long interval of time.
- In Python, a variable can have a short name (like *x* and *y*) or a more descriptive name such as **age**, **carname**, **total_volume**. A Python developer needs to follow the following rules as a set of naming conventions according to the coding standards:
 - A name must be start with an alphabet (A-Z, a-z) or the underscore (_);
 - name can't start with a number (0-9);
 - names can be alpha-numeric but strictly start with an alphabet (A-Z, a-z) or the underscore (_) only.
 - names are case-sensitive in Python, which means age, Age, and AGE represent three different identifiers.

Listing 3.1 highlights a couple of examples of valid identifier names and some of the invalid names are listed in Listing 3.2 and using any of these will raise `SyntaxError` by the interpreter.

Listing 3.1 Examples of valid identifier names

```
>>>lan = 'Python'
>>>ver_1 = 3.7
>>>present_ver = 3.11
>>>_ = 3
>>>_IDE = 'idle'
>>>Vers = [2.7,3.7,3.11]
```

Listing 3.2 Examples of invalid identifiers names

```
>>>my name = 'Goutam'
>>>my-name = 'Goutam'
>>>#marks = 3
>>>marks# = 3
>>>1_address = 'VIT'
>>>1 address = 'VIT'
```

Sometimes it is necessary that an identifier needs a descriptive name having multiple words. There are several techniques you can use to make them more readable:

- **Camel Case:** each word, except the first, must be starts with an uppercase letter (A-Z) only, such as myFirstName, presentAddress;
- **Pascal Case:** each word must be starts with an uppercase letter such as MyFirstName, PresentAddress;
- **Snake Case:** each word is separated by an underscore (_) character such as first_name, present_address;

3.3 Comments

When writing code, it's important to make sure that your code can be easily understood by others. Giving variables obvious names, defining explicit functions, and organizing your code are all great ways to do this. Another awesome and easy way to increase the readability of your code is by using comments!

Comments are an integral part of any program. In Python, comments come in the form of module-level docstrings, or even inline explanations that help shed light on a complex function. Before diving into the different types of comments, let's take a closer look at why commenting on your code is so important.

First, this section highlights two possible scenarios in which a programmer decided not to comment on their code. So that you can understand the importance of proper documentation of Python code and later you are going to know how to comment on Python scripts.

▪ **Reading your own code** - Client A wants a last-minute deployment for their web service. You're already on a tight deadline, so you decide to just make it work. All that “extra” stuff—documentation, proper commenting, and so forth—you'll add that later.

The deadline comes, and you deploy the service right on time and you have noted and go back to update the comments, but before you can put it on your to-do list, your boss comes over with a new project that you need to get started on immediately. Within a few days, you'd completely forgotten that you were supposed to go back and properly comment on the code you wrote for Client A.

Suppose after six months Client A needs a patch built for that same service to comply with some new requirements. It's your job to maintain it since you were the one who built it in the first place. Now it will be difficult to recall all the developments that you have made and after spending hours parsing through your old code, you're completely lost.

You were in such a rush at the time that you didn't name your variables properly or even set your functions up in the proper control flow. Worst of all, you don't have any comments in the script to tell you what's what! Developers forget what their code does all the time, especially if it was written a long time ago or under a lot of pressure. When a deadline is fast approaching, and hours in front of the computer have led to bloodshot eyes and cramped hands, that pressure can be reflected in the form of code that is messier than usual. Writing comments as you go is a great way to prevent the above scenario from happening. Be nice to Future You!

▪ **Your code is being read by others** - Imagine you're the only developer working on a small Django project. You understand your own code well, so you don't tend to use comments or any other sort of documentation, and you like it that way. Comments take time to write and maintain, and you just don't see the point.

The only problem is, by the end of the year your “small Django project” has turned into a “20,000 lines of code” project, and your supervisor is bringing on additional developers to help maintain it. The new developers work hard to quickly get up to speed, but within the first few days of working together, you've realized that they're having some trouble. You used some unorthodox variable names and wrote with super short syntax. The new developer spends a lot of time stepping through your code line by line, trying to figure out how it all works. It takes a few days before they can even help you maintain it!

Using comments throughout your code can help other developers in situations like this one. Comments help other developers skim through your code and gain an understanding of how it all works very quickly.

Now that you understand why it's so important to comment on your code, let's go over some basics so you know how to do it properly. Comments are for developers. They describe parts of the code where necessary to facilitate the understanding of programmers, including yourself. A comment can be any of these two types:

3.3.1 Single line commenting

We can do this by putting a hash mark (#) before your desired comment. Consider the example given below:

```
>>>#This is a comment
>>>print('Commenting in python')
```

Commenting in python

Python interpreter doesn't interpret a line of code if the line is preceded with a # till the end of the line. That's why we didn't see anything in the output from the first line of the code.

Note: Comments should be short, sweet, and to the point. While PEP 8 advises keeping code at 79 characters or fewer per line, it suggests a max of 72 characters for inline comments and docstrings. If your comment is approaching or exceeding that length, then you'll want to spread it out over multiple lines.

3.3.2 Multiline Commenting

Python doesn't have a way to write multiline comments as you can in other languages such as C, Java, and Go. Don't try this with your code, or else you will end with a `SyntaxError`.

```
>>># So you can't
just do this
in python
```

`SyntaxError: invalid syntax`

For the above code interpreter only ignores the first line as single line comment and it interprets as wrong syntax for the others and raised a `SyntaxError`. While Python doesn't have native multiline commenting functionality, you can create multiline comments in Python in two ways. The first way is simply by pressing the return key after each line, adding a # symbol, and continuing your comment from there. See the example given below:

```
>>># This is a pretty good example
>>># of how you can spread comments
>>># over multiple lines in Python
```

The second approach is embedding comments in your code as multiline strings by wrapping your comment inside a set of triple quotes, as follows:

```
>>>"""
If I really hate pressing enter and
typing all those hash marks, I could
just do this instead
"""
```

While this gives you multiline functionality, this isn't technically a comment. It's a string that's not assigned to any variable, so it's not called or referenced by your program. Still, since it'll be ignored at runtime and won't appear in the bytecode, it can effectively act as a comment.

Note: Sometimes it also depending on where we are putting these three quotes ("""---""") in the code. If we put it inside the function definition it will be the part of Python byte code. This one is known as docstring.

3.4 Python Types

In Python data types take in the form of objects – where some of them comes as in-built with Python or some of them we can create using classes. In this Section, the discussion mainly focuses on built-in types that come with Python.

Before we start with the discussion let's see where data types fall in Python. From a real-world point of view, every information that can be processed and stored is known as data. Such as your name, address, contact details, age, etc. and Python provides certain types of built-in objects used to hold the meaningful information as identifier.

If you are coming from other programming domain such as C, C++ or Java, you will be responsible to take care the types of every data, which means you must decide it in advance, what will be the type of data an identifier can hold throughout its lifetime. But due to the dynamic nature of Python most of this work goes away, which means an identifier can hold a value of any type at any point of execution. Table 3.3 highlights Python built-in types with their class name.

Table 3.3 Python built-in Object types

Object Type	Data Type
Numeric	int, float, complex
Numeric with user-defined precision	Fraction for rational and Decimal for floating point numbers
Boolean	bool for truth values
None	NoneType holding no values
Iterator	Can be used with any of the sequence types
Sequence types	list, tuple, range
Set	set, frozenset
Mapping type	dict called dictionary as pair of key and value
Text Sequence type	str known as string
Binary Sequence types	bytes, bytearray, memoryview

In Table 3.3 three classes are categorized as sequence type, but sets, dictionary, and string can also be considered as sequence type objects in python. It means values holding by each sequence types can be accessed sequentially.

The class names or data types are highlighted with bold text in Table 3.3 and in next three sub-sections numeric types are discussed broadly in Section Numeric Type and all sequence types are formally introduced in Section Sequence Type. This discussion on strings is also limited to the introductory part only, which is broadly discussed separately along with its various operations.

3.4.1 Numeric Type

Python has three numeric data types such as **int** as integer, **float** as floating-point, and **complex** for complex numbers. In addition, Booleans are also considered as subtypes of integers, supporting only two values True and False. Integers don't have any fractional part; floating-point numbers are designed to work with real numbers and complex numbers have two parts namely **real** and imaginary (as **img**) and individually both are floating-point types. Later in Python 2.4 and 2.6 introduces two other numeric types as Decimal and Fraction classes. Decimal type is introduced with fixed precision and Fractions are implemented on rational objects.

Integers

In Python integers are whole numbers i.e., no fractions, no decimal points, nothing fancy. Well, aside from a possible initial sign and bases, if you want to express numbers in other ways than the usual decimal (base 10). The `int()` constructor method can be used to create an integer object or directly a number with possible sign can be assigned. Any sequence of digits in Python represents an integer literal and examples of all valid integer literal forms are given in code Listing 3.3:

Listing 3.3 Example of integers of decimal base

```
>>>a = +5
>>>b = 5
>>>c = -5
>>>d = 1_00_000
>>>e = int(4)
```

A negative integer number should be preceded with a negative (-) sign, and numbers with optional plus (+) sign are considered positive integers as assigned to the variable **a** (with sign) and **b** (without sign). Integers are assumed to be decimal (base 10) unless prefix characters are used to specify another base. You might never need to use these bases, but you'll probably see them in Python code somewhere, sometime. In Python, you can express literal integers in three bases besides decimal with these integer prefixes as: (a) 0b or 0B for binary (base 2); (b) 0o or 0O for octal (base 8); and (c) 0x or 0X for hexadecimal (base 16).

These bases are all powers of two (2), and are handy in some cases, although you may never need to use anything other than good old decimal integers as shown in code Listing 3.3. Check the code Listing 3.4 to know how to assign values in other integer form using prefix characters.

Listing 3.4 Examples of other Integer types

```
>>>a = 0b11
>>>b = 0B110
>>>c = 0o245
>>>d = 00345
>>>e = 0x2345a
>>>f = 0X34af
>>>print(a, b, c, d, e, f)
3 6 165 229 144474 13487
```

The prefix characters are supported in upper (0b, 0o, or 0x) as well as lower (0B, 0O, or 0X) case. We must note that while using any integer to an expression for example in `print()` statement at line 7 values will be always processed in decimal form, as values are printed in the output of code Listing 3.4. Pythons have three in-built function to do that. `bin()`, `oct()`, and `hex()` are used to convert the decimal form of integer to binary, octal and hexa-decimal respectively.

```
>>> print(bin(a), oct(c), hex(e))
0b11 0o245 0x2345a
```

Python 3.x supports unlimited precessions means any size of integer:

[illegible]

Unlimited precision integers are built-in tools in Python and support up to memory of your device. They are also why we can raise large exponent numbers:

```
>>>45 ** 40
1344313472276061687044450174711376326749814325012266635894775390625
```

Floating-Point

In Python real numbers are represented by the class float with fixed 8 bytes (or 64 bits) of memory. These 64-bits are aligned into three different parts, as (a) a single bit is reserved as sign (positive or negative) bit; (b) 11 bits are reserved for exponent $1.5e^{-5}$ 1.5×10^{-5} (exponent is -5) within a range of $[-1022, 1023]$; and (c) 52 bits are significant digits.

In computers floating-point numbers are represented in base 2 (binary) fractions. For example, the decimal fraction 0.125 has value $1/10 + 2/100 + 5/1000$, and in the same way the binary fraction 0.001 has value $0/2 + 0/4 + 1/8$. These two fractions have identical values, the only real difference being that the first is written in base 10 fractional notation, and the second in base 2. For example, 12.345 can be represented as a base-ten floating-point number:

$$12.45 = \underbrace{12345}_{\text{significand}} \times \underbrace{10}_{\text{base}} \overset{\text{exponent}}{\underbrace{-3}}$$

The constructor method `float()` of the `float` class returns a floating-point number based on the number or a string:

```
>>>float(3.45)
3.45
>>>float('3.45')
3.45
```

In computers there are very few fractional numbers represented as binary fractions, instead an

approximated binary floating-point number is stored into memory. Consider the fraction $1/3$, in decimal base 10 it can be 0.3 or 0.33 or 0.333 and so on. No matter how many digits you're willing to consider, the result will never be same. Let's do a comparison between two float numbers:

```
>>>x = 0.1 + 0.1 + 0.1
>>>y = 0.3
>>>x==y
False
```

It looks very strange, variables x and y having same value but comparison states different story. This is because Python cannot use a finite number of digits to represent the numbers. Let's check this using *format()* function:

```
>>>format(x, '.20f')
'0.30000000000000004441'
>>>format(y, '.20f')
'0.29999999999999998890'
```

Here we have taken upto 20 decimal place and exactly the numbers are not stored into computer memory while using binary decimal. Just remember, even though the printed result looks like the exact value of 0.3, the actual stored value is the nearest representable binary fraction.

Note: This is in the very nature of binary floating-point: this is not a bug in Python, and it is not a bug in your code either. You'll see the same kind of thing in all languages that support your hardware's floating-point arithmetic (although some languages may not display the difference by default, or in all output modes).

PEP485²⁰ provides a solution that fixes this problem by using relative and absolute tolerances. It provides *isclose()* function of math module and returns True if two numbers are relatively close to each other:

```
>>>x = 0.1 + 0.1 + 0.1
>>>y = 0.3
>>>from math import isclose
>>>isclose(x,y)
True
```

Some of these drawbacks are handled using Decimal and Fraction data types are discussed in the next two sections.

Decimal

Decimals are like floating-point numbers with fixed precisions. It was introduced in Python 2.4 to core numeric type as decimal object, formally known as Decimal. Like floating-point type we can't just assign any literals to a name, for that we need to import the decimal module. After all we need to answer why another numeric type like floating-point. Let's first list all the advantages of decimal type over floating-point type:

- Decimal numbers can be represented exactly. In contrast, numbers like 1.1 and 2.2 do not have exact representations in binary floating point. End users typically would not expect $1.1 + 2.2$ to display as 3.3000000000000003 as it does with binary floating point.

²⁰ <https://peps.python.org/pep-0485/>

- The exactness carries over into arithmetic. In decimal floating point, $0.1 + 0.1 + 0.1 - 0.3$ is exactly equal to zero. In binary floating point, the result is $5.5511151231257827\text{e-}017$, near to zero. For this reason, decimal is preferred in accounting applications that have strict equality invariants.
- The decimal module incorporates a notion of significant places so that $1.30 + 1.20$ is 2.50 . The trailing zero is kept indicating significance. This is the customary presentation for monetary applications. For multiplication, the “schoolbook” approach uses all the figures in the multiplicands. For instance, $1.3 * 1.2$ gives 1.56 while $1.30 * 1.20$ gives 1.5600 .

To create a decimal type need to import the decimal module first and need to go through the current context with `getcontext()` and if necessary we can set new values to its parameters:

```
>>>from decimal import *
>>>getcontext()
```

```
Context(prec=28, rounding=ROUND_HALF_EVEN, Emin=-999999, Emax=999999, capitals=1,
clamp=0, flags=[], traps=[InvalidOperation, DivisionByZero, Overflow])
```

The decimal number supports up to 28 decimal places and Python interpreter raises exception on getting signals of `InvalidOperation`, `DivisionByZero` and `Overflow`. The context for arithmetic is an environment specifying precision, rounding rules, limits on exponents, flags indicating the results of operations, and trap enablers which determine whether signals are treated as exceptions. Table 3.4 lists all the rounding options of decimal module:

Table 3.4 List of all rounding options of decimal type

Rounding options	Description
<code>decimal.ROUND_CEILING</code>	Round towards infinity
<code>decimal.Round_DOWN</code>	Round towards zero
<code>decimal.ROUND_FLOOR</code>	Round towards -infinity
<code>decimal.ROUND_HALF_DOWN</code>	Round to nearest with ties going towards zero.
<code>decimal.ROUND_HALF_EVEN</code>	Round to nearest with ties going to nearest even integer.
<code>decimal.ROUND_HALF_UP</code>	Round to nearest with ties going away from zero.
<code>decimal.ROUND_UP</code>	Round away from zero
<code>decimal.ROUND_05UP</code>	Round away from zero if the last digit after rounding towards zero would have been 0 or 5; otherwise round towards zero.

Signals are groups of exceptional conditions arising during computation. Depending on the needs of the application, signals may be ignored, considered as informational, or treated as exceptions. The signals in the decimal module are: `Clamped`, `InvalidOperation`, `DivisionByZero`, `Inexact`, `Rounded`, `Subnormal`, `Overflow`, `Underflow`, and `FloatOperation`.

A decimal number is immutable. It has a sign, coefficient digits, and an exponent. To preserve significance, the coefficient digits do not truncate trailing zeros. Decimals also include special values such as `Infinity`, `-Infinity`, and `NaN`. The standard also differentiates `-0` from `+0`. By calling `Decimal` constructor we can make a decimal object and passing the value as string with the desired number of decimal digits:

```
>>>import decimal
>>>decimal.Decimal('3.68')
Decimal('3.68')
```

On passing a floating-point value, it will take it up to the precision value of the current context:

```
>>>decimal.Decimal(3.68)
Decimal('3.6800000000000000159872115546022541821002960205078125')
```

Unlike hardware based binary floating point, the decimal module has a user alterable precision (defaulting to 28 places) and can be altered as follows:

```
>>>getcontext().prec=3
>>>Decimal('1')/Decimal('7')
Decimal('0.143')
```

If the FloatOperation signal is trapped, accidental mixing of decimals and floats in constructors or ordering comparisons raises an exception:

```
>>>getcontext().traps[FloatOperation] = True
>>>Decimal(5.7)
Traceback (most recent call last):
  File "<pyshell#68>", line 1, in <module>
    Decimal(5.7)
decimal.FloatOperation: [<class 'decimal.FloatOperation'>]
```

The significance of a new Decimal is determined solely by the number of digits input. Context precision and rounding only come into play during arithmetic operations only, not at the time of creation.

Listing 3.5 Use of context parameters in Decimal

```
>>>getcontext().prec = 6
>>>Decimal('3.0')
Decimal('3.0')
>>>Decimal('3.1415926535')
Decimal('3.1415926535')
>>>Decimal('3.1415926535') + Decimal('2.7182818285')
Decimal('5.85987')
>>>getcontext().rounding = ROUND_UP
>>>Decimal('3.1415926535') + Decimal('2.7182818285')
Decimal('5.85988')
```

In Listing 3.5, precision is set to 6, it doesn't make any impact on the next two Decimal creations. It considers all the digits those are passed to *Decimal()* constructor built-in function. But the addition operations of line 6 and 9 produces the result within the length of precision and the second expression rounding up the result up to 6th decimal place.

Fractional

The shortcoming of floating-point arithmetic in binary is address by Fraction data type. It's not coming as another built-in types like int or float rather we need import it from fractions module. If we need to work with financial data and require infinite precision for some calculation it will be preferred over floating-point type. Using Fraction class we can create many ways a fraction object depending on number of values that we are passing to **Fraction()** constructor method of fractions class. The class constructor accepts zero, one, or two arguments of various types:

Listing 3.6 Creation of Fraction Objects

```
>>>import fractions
>>>a = fractions.Fraction()
>>>b = fractions.Fraction(3)
>>>c = fractions.Fraction(3,4)
>>>print(a, b, c)
0 3 3/4
```

The first version in Listing 3.6 simply returns 0 as fractional number. The second variant of constructor takes 1 argument and returns 3 as Fractional number. The last variant takes two numbers as numerator and denominator. The third variant only accepts instances of rational number.

Note: To get the fractional value in human-friendly texture (with slash / character) it is necessary to `print()` the Fraction object.

For the third variant of `Decimal()` constructor, when we pass a valid numerator and denominator, they will be normalized if they have a common divisor:

```
>>>fractions.Fraction(3,6) #gcd(3,6) = 3
Fraction(1, 2)
>>>fractions.Fraction(0,5) #gcd(0,5) = 5
Fraction(0, 1)
```

The normalization also takes the minus sign into account when you define negative fractions:

```
>>>-fractions.Fraction(3,12)
Fraction(-1, 4)
>>>fractions.Fraction(-3,12)
Fraction(-1, 4)
>>>fractions.Fraction(3,-12)
Fraction(-1, 4)
```

The second variant constructor of Listing 3.6 also takes arguments of real and string types. From real types it accepts both floating-point and decimal number and both considered irrational number.

```
>>>print(fractions.Fraction(0.5))
1/2
>>>print(fractions.Fraction('3'))
3
```

For the floating-point type of result is the same in fractional notation. However, this code works as expected only by coincidence. In most cases, you won't get the intended value due to the representation error that affects float numbers, whether they're rational or not. For example, a fraction 1/10 can be accommodated as 0.1 in floating-point number. But in python story is different, because of binary representation of 0.1 stores an approximate value in memory.

```
>>>print(fractions.Fraction(0.1))
3602879701896397/36028797018963968
```

The previous number 0.5 can be accommodated in memory, so it gets the exact representation of fraction 1/2. But 1/10 can only be approximated with a non-terminating repeating expansion of binary digits. The numerator and denominator of a Fraction object can be access separately using numerator and denominator member of Fraction object:

```
>>>a = fractions.Fraction(3,4)
>>>print(a.numerator, a.denominator)
3 4
```

We have seen how to convert other data types to Fraction type. Converting between built-in data types in Python can be done by calling constructor methods `int()`, `float()`, `str()` and `complex()`. A fractional type can be directly converted to float type and internally it's done by dividing the numerator by denominator of the fraction.

```
>>>a = fractions.Fraction(3,4)
>>>float(a)
0.75
>>>str(a)
'3/4'
>>>complex(a)
(0.75+0j)
```

Fraction didn't provide any implementation directly to convert to integer. It simply truncates the fraction to smallest integer and like converting a float to an integer:

```
>>>a = fractions.Fraction(22,3)
>>>print(a)
>>>22/3
>>>int(a)    #truncates the integer part
7
```

Converting to Decimal type will raise a `TypeError`, since Fractions represents a division and to bypass this, we can convert any part as Decimal and dividing them manually:

```
>>>a = fractions.Fraction(12,3)
>>>d = a.numerator / Decimal(a.denominator)
>>>d
Decimal('4')
```

Choosing between Float, Decimal and Fraction

The float data type should be your default choice for representing real numbers in most situations. Floating-point numbers are mostly suitable for science, engineering, and computer graphics, where execution speed is more important than precision. Hardly any program requires higher precision than you can get with a floating-point anyway.

Floating-point stores values in binary float point and sometimes using the binary system doesn't provide enough precision for real numbers. One notable example is financial calculations, which involve dealing with very large and very small numbers at the same time. They also tend to repeat the same arithmetic operation repeatedly, which could accumulate a significant rounding error.

Real numbers can be stored using decimal floating-point arithmetic to mitigate these problems and eliminate the binary representation error. It's like a float as it moves the decimal point around to accommodate larger or smaller magnitudes. However, it operates in the decimal system instead of in the binary.

Both the Fraction and Decimal types share a few similarities. They address the binary representation error, they're implemented in software, and you can use them for monetary applications. Nevertheless,

Listing 3.8 Formation of complex number

```

>>>a = 4j
>>>b = 3j
>>>c = .2j
#Adding complex number with integer
>>>d = 3 + a
#Adding complex number with float
>>>e = 3.4 + b
>>>a, b, c, d, e
(4j, 3j, 0.2j, 3+4j, (3.4+3j))

```

It is also possible to access the real and imag part of a complex number separately using the real and imag attribute of a complex object:

```

>>>c = 4+6j
>>>c.real, c.imag
(4.0, 6.0)

```

Note: For the type of consistency in Python each part of the complex number is considered as float, because absolute value of a complex number is floating type. So, there is no reason to make a special case and return an **int**, if the output happens to be a round number. Also, note that the same reasoning applies for the components of your imaginary numbers which are always stored as float.

Boolean

Python 3 has another built-in type as Boolean, which has two values as True and False. Using built-in **type()** functions, it is possible to check the type of **True** and **False**:

```

>>>type(True)
<class 'bool'>
>>>type(False)
<class 'bool'>

```

An expression containing conditional, logical, identity and membership operators always evaluates to bool type. Details about working of Boolean type expression is discussed in Chapter 4. In Python built-in names are not keyword and we can assign them any value. But True and False are not built-in types, they are keywords and assigning any value to them will be raised a **SyntaxError**, as follows:

```

>>>bool = 45
>>>type(bool)
<class 'int'>
>>>a = True
>>>type(a)
<class 'bool'>
>>>True = 3

```

SyntaxError: cannot assign to True

Booleans are considered as numeric type in Python and any value other than 0 or **None**, will be treated as True. Because they are considered as numeric types, it is possible to perform arithmetic operations with them. At any point if you want to count the number of items that satisfy a condition, means number of True counts.

```

>>>bool(3)
True
>>>True + True
2
>>>int(True)
1
>>>bool(56.7)
True
>>>bool(0)
False
>>>bool(None)
False

```

Built-in function `len()` returns the length of a sequence type and in Python the length of an object considered as `True`:

```

>>>a = [3,4]
>>>len(a)
2
>>>bool(a)
True
>>>b = []
>>>len(b)
0
>>>bool(a)
False

```

The Python Boolean is a commonly used data type with many useful applications. You can use Booleans with operators like **not**, **and**, **or**, **in**, **is**, **==**, and **!=** to compare the values and check for membership, identity, or equality (discussed in Chapter 4). You can also use Boolean testing with **if** statement (discussed in Chapter 6) to control the flow of your programs based on the truthiness of an expression.

3.4.2 String

Discussion in this section is limited to string declaration only, details about string operations and methods can be found in Chapter 5. In Python strings are known as collection of characters and type of string is known as `str`. Strings literal is delimited using single or double quote and any number of characters between the opening and matching closing delimiter is considered as string:

```

>>>str1 = 'Hello'
>>>str2 = "Python"
>>>type(str1)
<class 'str'>

```

In Python a string can contain any number of characters that supports your system memory and it can be empty also by assigning a pair of single or double quotes or simply by calling `str()` constructor method of string class:

```
>>>str1 = ''
>>>str2 = str()
>>>str1, str2
('', '')
```

Any type can be converted to string by passing a value of type to `str()` constructor method inside quotes:

```
>>>str('1')
'1'
>>>str('4.6')
'4.6'
```

If you wish to include either type of quote character within the string, the simplest way is to delimit the string with the other type. If a string is to contain a single quote, delimit it with double quotes and vice versa:

```
>>>str1 = '"Contains double quotes"'
>>>str2 = "'Contains Single quotes'"
```

In Python a multiline string can be formed using any form of quotes three times. A multiline string can include escape sequences, but single quotes, double quotes, and newlines can be included without escaping them.

```
>>>str1 = '''This is a multiline
string.
Thats good'''
>>>print(str1)
This is a multiline
string.
Thats good
```

Strings are also categorized as a type of sequence means can be iterate each characters using a loop, which is covered separately in Chapter 5. Other built-in sequence types of lists, tuple, dictionary, sets are covered in the next Section.

3.4.3 Sequence Types

In Python sequence types supports a variety of operations as listed in Table 3.5 for both mutable and immutable type. In Table 3.5, s and t are sequences of the same type, n , i , j and k are integers and x is an arbitrary object that meets any type and value restrictions imposed by s . Details about these operations can be found in the respective chapter mentioned in 4th column of the table.

The **in** and **not in** operations have the same priorities as the comparison operations. The **+** (concatenation) and ***** (repetition) operations have the same priority as the corresponding numeric operations.

Table 3.5 List of operations supported by sequence types.

Operation	Operation Type	Result	Discussed in Chapter
$x \text{ in } s$	Membership	Return True if an item of s is x or False	4

<i>x not in s</i>	Membership	Return True if <i>x</i> is not an item of <i>s</i>	4
<i>s + t</i>	Concatenation	Appending <i>t</i> at the end of <i>s</i>	4, 5, 7
<i>s * n</i> or <i>n * s</i>	Addition	Equivalent to adding <i>s</i> to itself <i>n</i> times	4, 5, 7
<i>s[i]</i>	Indexing	<i>i</i> th item of <i>s</i>	5 and 7
<i>s[i:j]</i>	Slicing	Slice of <i>s</i> from <i>i</i> to <i>j</i>	5 and 7
<i>s[i:j:k]</i>	Slicing with Step	Slice of <i>s</i> from <i>i</i> to <i>j</i> with step <i>k</i>	5, 6, and 7
<i>len(s)</i>	size or length	Length of <i>s</i>	5, 6 and 7
<i>min(s)</i>	Minimum	Smallest item of <i>s</i>	7
<i>max(s)</i>	Maximum	Largest item of <i>s</i>	7
<i>s.index(x)</i>	Searching	Index of first occurrence of <i>x</i> in <i>s</i>	5 and 7
<i>s.count(x)</i>	Counting	No. of times <i>x</i> in <i>s</i>	5, 6 and 7

List

List is one of the built-in data structures in Python. It can accommodate multiple values of similar or dissimilar types. If you are coming from other programming domain, Python list is like the Vectors in C++ or ArrayList in JAVA. Lists are dynamic in nature, can grow or shrink through its insertion and deletion operation. List also supports searching for an item using membership operator or sequentially using a loop. Inserting and deleting the element from the beginning of the list is considered as costliest operation as all the elements are needed to be shifted by one position. This Section is all about the introduction to list as a data type and its assignments, detailed explanation can be found in Chapter 7. We will begin with declaration of list and accessing of it:

```
>>>list1 = list()
>>>list2 = []
```

Like other built-in types an empty list can be created using constructor method **list()** or by assigning a pair of empty square brackets (**[]**). List can be created by passing other sequence types (see Listing 3.9) to the constructor including strings.

Listing 3.9 Creation of list using other objects

```
>>>list('abc')
['a', 'b', 'c']
>>>list(range(3))
[0, 1, 2]
>>>d = {2,3,4} #set
>>>list(d)
[2, 3, 4]
>>>s = (1,2,3) #tuple
>>>list(s)
[1, 2, 3]
>>>di = {'a':2, 'b':3} #dictionary
>>>list(di)
['a', 'b']
```

Numeric types aren't iterable, and trying to do so will raised a `TypeError`, as follows:

```
>>>list(11)
Traceback (most recent call last):
  File "<pyshell#2>", line 1, in <module>
    list(11)
TypeError: 'int' object is not iterable
```

A list can be created using by assigned comma separated similar or dissimilar values inside a pair of square brackets:

```
>>>a = [2,3,4] #similar types
>>>b = [2,5.6,7+3j,'Python'] #different types
```

Tuple

Tuples are immutable (means can't alter after creation) sequence, typically used to store collections of dissimilar data. Tuples are also used for cases where an immutable sequence of homogeneous data is needed. Number of ways tuple can be created (see code Listing 3.10):

Listing 3.10 Ways of creating a tuple

```
1 >>>a = () #returns an empty tuple
2 >>>b = tuple() #built-in constructor method
3 >>>c = 4, #using trailing comma
4 >>>d = (4,) #using singleton object
5 >>>e = tuple([1,2]) #passing an iterable object
6 >>>f = 1,2,3 #separating items with comma
7 >>>g = (1,2,3) #separating items with comma
```

Constructor `tuple()` can take maximum one argument and creates a tuple of an iterable object of same items and same order. Without comma at line 3 and 4, Python would consider them as single-expression statement. At line 6 and 7, comma delimit individual elements within the parentheses, forming tuples with multiple elements. Each comma acts as a separator, indicating that the values on either side of it are distinct items in the tuple.

Binary Sequence Types – bytes and bytearray

Both bytes and bytearrays can be used to store binary data such as images, audio files, network packets, or serialized objects. They can also be used in cryptography, network communication, and low-level system programming. Some of the operations that can be performed on bytes and bytearrays include slicing, concatenation, decoding, encoding, hashing, and encryption.

Bytes are represented by the bytes type and can be created using the `'b'` prefix before a string literal or by using the `bytes()` constructor. Once created, a bytes object cannot be modified in place. Some examples of bytes literals given below:

```

1 >>>b'hello'
2 b'hello'
3 >>>my_bytes = bytes([0x48, 0x65, 0x6c, 0x6c, 0x6f])
4 >>>print(my_bytes)
5 b'Hello'
6 >>>print(type(my_bytes))
7 <class 'bytes'>

```

In the above example, a bytes object named `my_bytes` is created using the `bytes()` constructor and passing a list of integers representing the ASCII codes in hexa-decimal form for the characters 'H', 'e', 'l', 'l', and 'o'. The resulting bytes object is equivalent to the bytes literal `b'Hello'`. A bytes object can be created from a Unicode string using UTF-8 encoding as follows:

```

my_str = 'unicode string'
my_bytes = my_str.encode('utf-8')
# print the bytes object and its type
print(my_bytes) # prints b'unicode string'
print(type(my_bytes)) # prints <class 'bytes'>

```

The bytearray provides a way to represent sequences of bytes in a more flexible and efficient way than strings or lists of integers. The working of bytearray is shown in code Listing 3. 11 and converted to a string using the `str()` function. Like other sequence types, bytearray objects also support indexing.

Listing 3. 11 Creating and manipulating a bytearray object

```

# Create a byte array
>>>data = bytearray(b'\x00\x01\x02\x03\x04\x05\x06\x07')
# Modify the data in place
>>>data[2:5] = b'\xff\xff\xff'
# Convert the byte array to a string and print it
>>>print(str(data))
bytearray(b'\x00\x01\xff\xff\xff\x05\x06\x07')

```

Some of its purposes and uses are listed below:

- Efficient storage of binary data: Byte arrays provide a compact and efficient way to store binary data, such as image or audio files, network packets, or serialized objects. Because they are mutable, you can modify the data in place without having to create a new copy of the entire sequence.
- Network communication: Byte arrays can be used to represent messages or packets in network communication protocols, such as TCP, UDP, or HTTP. They can be sent and received over a network connection using socket programming, and they can be parsed and processed on the receiving end.
- Cryptography: Byte arrays can be used to represent cryptographic keys, digests, or ciphertexts. They can be processed using cryptographic algorithms, such as encryption, decryption, or hashing, and they can be compared for equality or used as inputs to other cryptographic operations.
- Low-level system programming: Byte arrays can be used to interact with low-level system APIs that require binary data, such as device drivers, file I/O, or memory-mapped files. They can be

passed as arguments to C functions using the ctypes module or accessed directly using the memory view object.

Dictionary

A dictionary is a mapping object which maps hashable (means an object if it has a hash value which never changes during the lifetime) values. It is an unordered collection of data values, and all items are stored in a *key: value* pair. A single key is associated with a value, and it is a mutable type of object. Two ways an empty dictionary can be formed either using a pair of curly braces ({}), or by calling constructor method *dict()*:

```
>>>d1 = {}
>>>d2 = dict()
>>>type(d1)
<class 'dict'>
```

By passing *key: value* pair inside a pair of curly braces ({}), a dictionary can be formed and keep in mind dictionary keys are always immutable type, means non-modifiable objects are allowed as dictionary key.

```
>>>d = {'lan': 'Python', 'ver': 3.11}
>>>d
{'lan': 'Python', 'ver': 3.11}
```

An identifier of immutable type can be used as key-value pair to form a dictionary. The list [(*'lan'*, *'Python'*), (*'ver'*, 3.11)] contains tuples, where each tuple represents a key-value pair. can also be created by passing a list of tuples.

```
>>> dict([( 'lan', 'Python'), ('ver', 3.11)])
{'lan': 'Python', 'ver': 3.11}
```

Dictionary supports searching an item in arbitrary order and all keys in dictionary should be unique. On adding a new item with existing key can update its value only:

```
>>>d['lan'] = 'c' #updating the value of key lan to c
>>>d['ver'] = 'c11' #updating the value of key ver to c11
>>>d
{'lan': 'c', 'ver': 'c11'}
```

Python dictionaries also support copy from another dictionary, removing a specific item based on key or pop out the last *key: value* pair item. All these operations with creation of dictionary from other data types discussed in Chapter 7.

Sets

A set object is an unordered collection of distinct hashable objects. Common uses include membership testing, removing duplicates from a sequence, and computing mathematical operations such as intersection, union, difference, and symmetric difference.

Python has two built-in set types as **set** and **frozenset**. Out of these set is mutable type means items can be altered later and frozenset is immutable type, can be used as a key of a dictionary. Constructor methods like other built-in types can be used to create any of this type:

```
>>>s1 = set()
>>>s2 = frozenset()
```

Remember using a pair of empty curly braces returns a dictionary object instead of a set. Non-empty sets (not frozensets) can be created by placing a comma-separated list of elements within braces, in addition to the set constructor as shown below:

Listing 3.12 Examples of valid set creation

```
>>>s3 = {1,}
>>>s4 = {1}
>>>s5 = {4,4.3}
>>>s6 = set({1,2,3,6.7,'w',(1,2)})
```

Inside the set and frozenset constructor, items should be of a hashable type. In general, all immutable types listed in Table 3.6 can be used to create a set or frozenset. A dictionary type can also be used to create any of these types, only keys are considered as items of the set.

```
>>>set({3:2})
{3}
>>>frozenset({3:2})
frozenset({3})
```

The discussion of sets concludes here. The remaining concepts and operations will be covered in Chapter 7.

3.4.4 User-defined Type

The purpose of a user-defined type is to create a custom data type that meets the specific needs of a particular application or domain. By defining a new type, you can encapsulate data and its behaviour into a single unit, which makes your code more organized, modular, and reusable. A user defined type has specific need and advantages over other data types discussed in previous sections. In python a user-defined data type can be created using the keyword **class**. Here's an example of a simple class creation without any body:

```
>>> class Sample:
...     pass
```

The indentation is necessary to start with the body of a class and the pass statement in the above class declaration represents an empty class. This class hasn't done anything yet. We can create an object of this class by calling its constructor:

```
>>> obj = Sample()
>>> obj
<__main__.Sample object at 0x000002043F3DF810>
```

In the output, `<__main__.Sample>` indicates that the `Sample` class is defined in the current Python module (`__main__`), signifies that `obj` is an object at memory address `0x000002043F3DF810`. We mostly interact with objects through methods and attributes, which is discussed in the Second part of the book.

3.4.5 None Type

Python doesn't have NULL keyword like other computer languages, but there is a **None** type. It is the

function's return value that “doesn't return anything”. `None` is often used to represent the absence of a value, as default parameters are not passed to the function. You can't assign a `NULL` value to the variable; if you do, it is illegal, and it will raise a **`NameError`**. Some important points related to `None` value are given below:

- Comparing `None` to anything will always return `False` except `None` itself.
- `None` is not a `0`.
- `None` is not an empty string.
- `None` is not the same as `False`.

In Python `None` is a special keyword that signifies the absence of value. It's used to indicate that a variable doesn't hold any data or that a function doesn't return anything explicitly.

```
>>> type(var)
<class 'NoneType'>
>>> if var is None:
...     var = 3
...
>>> var
3
```

3.5 Object Types

Python data types discussed in the previous section can be categorized into two groups based on, “how objects need to be handled during their lifetime”? Such as mutable and immutable and categorization of each object are listed in Table 3.6.

Table 3.6 Categorization of Object Types

Class	Explanation	Type
<code>bool</code>	boolean value	Immutable
<code>int</code>	integer value	Immutable
<code>float</code>	floating point value	Immutable
<code>decimal</code>	<code>decimal.Decimal</code> value	immutable
<code>fraction</code>	<code>fractions.Fraction</code> value	Immutable
<code>complex</code>	Numbers with real and imaginary part	Immutable
<code>list</code>	sequence of objects of mutable nature	Mutable
<code>tuple</code>	sequence of objects of immutable nature	Immutable
<code>range</code>	List of values in a range	Immutable
<code>str</code>	character/ string	Immutable
<code>set</code>	set of distinct objects that are of unordered nature	Mutable
<code>frozenset</code>	set class of immutable nature	Immutable
<code>dict</code>	dictionary or associative mapping	Mutable
<code>bytes</code>	Used to store and manipulate binary data	Immutable
<code>bytearray</code>	Used to store and manipulate binary data	Mutable

3.5.1 Mutability

Mutable in Python can be defined as the object that can change or be regarded as something changeable in nature. In Python mutable objects means the ability to modify the value of any object. In Listing 3.13, memory address of the list `my_list`, is printed using built-in function `id()`, i.e., 3048260948800. Later in the next line, 0th index item of `my_list` is updated to 10 without any error. It means a list object is mutable by nature and its id also remains same.

Listing 3.13 Mutability testing of list type

```
>>> my_list = [1, 2, 3]
>>> id(my_list)
3048260948800
>>> my_list[0] = 10
>>> id(my_list)
3048260948800
```

3.5.2 Immutability

Immutable objects in Python can be defined as objects that do not change their values and attributes over time. These objects become permanent once created and initialized. Python has numbers, tuples, strings, frozen sets, and user-defined classes with some exceptions. Values of these types can't be changed and remains permanent once they are initialized and hence called immutable. Once modifying different memory block is refereed by the name and this can be verified using `id()`. In Listing 3.14, immutability has been tested over string object `str1`, and after setting to new value 'Python', its memory address is changed. Means after modifying `str1` start referring to a new memory block.

Listing 3.14 Immutability testing of string type

```
>>> str1 = 'Hello'
>>> id(str1)
3048259007088
>>> str1 = 'Python'
>>> id(str1)
140715314864976
```

3.6 Variable Assignments

We have already seen couple of examples of variable assignments, a statement having an equal sign (=). Variables are symbolic names in any programming and in this section, we will learn various types of assignments based on data. Like others, in Python, variables need not be declared or defined in advance. To create a variable, you just assign it a value and then start using it.

```
>>>a = 4
```

This statement can be interpreted as “a is assigned the value 4.” Once this is done, `a` can be used in any statement or expression, and its value will be substituted, as shown below:

```
>>>a+4
8
```

In the last expression the value of **a**, is substituted and added with 4 and gets the result as 8. It is also possible to update the value of **a**, or can be used to assign to a new variable **b**, as shown below:

```
>>>a = 10 #updating the value of a
>>>a
10
>>>b = a #assigned to b
>>>b
10
```

Python also allows chained assignment, which makes it possible to assign the same value to several variables simultaneously:

```
>>>a = b = c = 5
>>>a
5
>>>b
5
```

The chained assignment in the above example, assigns 5 to the variables **a**, **b**, and **c** simultaneously. It is possible to assign multiple values to different variables at the same time. Python assigns values from right to left. When assigning multiple variables in a single line, different variable names are provided to the left of the assignment operator separated by a comma. The same goes for their respective values except they should be the right of the assignment operator.

```
>>>a, b = 4, 8
>>>a
4
>>>b
8
```

When using this method to declare variables, it's essential to pay attention to the order of variable names and their associated values. The first variable name on the left side of the assignment operator will receive the first value on the right side, and this pattern continues for subsequent variables. This approach is commonly referred to as multiple assignment. Importantly, the values being assigned do not have to be of the same data type; instead, they can consist of a mixture of valid Python data types.

```
>>>a, b, c, d = 4, "Python", 3.14, True
```

The above statement is an example of multiple variable assignment in a single line. It assigns values to four variables (**a**, **b**, **c**, and **d**) in a single statement. Each variable is assigned a corresponding value from the right-hand side of the statement.

3.7 Input and Output

3.7.1 Taking Input

Programs often need to obtain data from the user, usually by way of input from the keyboard. One way to accomplish this in Python with **input()** function and its general form is given below:

```
variable_name = input([prompt])
```

[prompt] is optional inside the parentheses of the **input()** function, you can include an optional

prompt message, which is a string enclosed in a single or double quotes. This prompt provides guidance or instructions to the user about what kind of input is expected. If you don't specify any prompt, the **input()** function will simply wait for the user to type something. Here's an example of how you might use this syntax:

```
>>>name = input("Please enter your name: ")
Please enter your name:
```

It is important to note that, **input()** function treats all user input as strings. If you need to work with numerical values, you'll need to convert the input to the appropriate data type using respective conversion functions such as **int()**, **float()**, after capturing it with **input()**.

```
>>>user_input = input('Enter a number.....')
Enter a number.....34
>>>print(type(user_input))
<class 'str'>
```

Next example demonstrates the conversion of a user input to integer. In this example, the user is prompted to enter their age. The input is captured as a string (**age_str**) and then converted to an integer using the **int()** function.

```
>>> age_str = input("Please enter your age: ")
Please enter your age: 34
>>> age = int(age_str)
>>> type(age)
<class 'int'>
```

In the next example, user is prompted to enter their weight in kilograms. The input is captured as a string (**weight_str**) and then converted to a floating-point number using the **float()** function.

```
>>> weight_str = input("Please enter your weight in kilograms: ")
Please enter your weight in kilograms: 74.5
>>> weight = float(weight_str)
>>> type(weight)
<class 'float'>
```

These examples demonstrate how to use the **input()** function to capture user input as strings and then convert them to different data types as needed for your program. Remember to handle input validation and errors to ensure that the user provides input in the expected format.

3.7.2. Printing Output

In Python, the **print()** function is used to display output to the standard output, typically the console or terminal. It allows you to output text, numbers, variables, and other data to the screen. The simplest form of **print()** function is to display a single string or variable's value:

```
>>>print("Hello, World!")
Hello, World!
```

The **print()** function can take multiple arguments, separated by commas. It will print each argument, separated by a space, and automatically add a newline character (**\n**) at the end by default:

```
>>>name = "Alice"
>>>age = 30
>>>print("Name:", name, "Age:", age)
Name: Alice Age: 30
```

Calling `print()` without arguments results in a blank line, which is a line comprised solely of the newline character, e.g. `print('\n')`. Don't confuse this, with an empty line (means zero characters), not even the newline e.g. `print('')`. Before passing a string object for printing, it can be formatted in many ways and all such methods are discussed in Section 5.9 of Chapter 5.

In Python 3, `print()` function uses two keyword arguments `'sep'` and `'end'` to modify the output string. While printing, `print()` function inserts a space between the values and that can be modified by using the keyword argument `'sep'`. See the listed example and in the output of second `print()`, comma(,) has been placed after every string literals.

```
>>> print('I am',37, 'Years old')
I am 37 Years old
>>> print('I am',37, 'Years old',sep=',')
I am,37,Years old
```

Separator keyword is very useful while formatting a date and based on our requirement we can pass as slash '/' or a hyphen '-' to separate day and month.

```
>>> print(12,11,2022,sep='/')
12/11/2022
>>> print(12,11,2022,sep='-')
12-11-2022
```

Unless the `end` keyword is used within `print`, it will always append a newline character (`'\n'`), at the end of what is being printed.

```
>>> print('A',end='');print('B');print('C')
AB
C
```

Due to use of `end` keyword within first print statement, output of the second print, placed in the same line as `'AB'`. But the third print statement prints `C` in a newline, because second print statement doesn't come with `end` keyword. Both keywords can be used together, that can help us to print data in a readable fashion.

```
>>> print('This','book','is','prepared','for', sep = '_', end='_Students')
This_book_is_prepared_for_Students
```

3.8 Built-in Functions

The Python interpreter has several functions and types built into it that are always available. These are listed in with a short description about their work. We would recommend go to the

link as given in the footnote of this page, to know more about this built-in²¹ functions.

Table 3.7 List of built-in functions

Function	Description
<code>abs(x)</code>	Return the absolute value of a number x. The argument may be an integer, a floating-point number. For complex number, functions return its magnitude.
<code>aiter(async_iterable)</code>	Return an asynchronous iterator for an asynchronous iterable.
<code>all(iterable)</code>	Returns True if all the elements of an iterable object are True
<code>any(iterable)</code>	Returns True if atleast one of the elements of an interable is True
<code>ascii(object)</code>	Return a string in printable from, but escaping the non-ASCII characters
<code>bin(n)</code>	Converts an integer number to its binary from with prefixed 0b.
<code>bool(x=False)</code>	Return a Boolean value, i.e. one of True or False.
<code>breakpoint()</code>	A built-in tool introduced in Python 3.7 that simplifies debugging by pausing your code's execution at a specific line.
<code>bytearray()</code>	Return a new array of byte.
<code>bytes()</code>	Return a new “bytes” object which is an immutable sequence of integers in the range $0 \leq x < 256$.
<code>callable(object)</code>	Returns True, if the object argument appears callable, False if not.
<code>chr(i)</code>	Return the string representing a character whose Unicode code point is the integer i. For example, <code>chr(97)</code> returns the string 'a'
<code>classmethod(anymethod)</code>	In Python 3, it is a decorator (<code>@classmethod</code>) instead of function and used to define a class method.
<code>compile()</code>	Compile the source into a code or AST object.
<code>complex()</code>	Returns a complex number
<code>delattr(object, name)</code>	Is used to dynamically delete an attribute from an object.
<code>dict()</code>	Creates a new dictionary
<code>dir()</code>	Without arguments, return the list of names in the current local scope.
<code>dir(object)</code>	With an argument, attempt to return a list of valid attributes for that object.
<code>divmod(a, b)</code>	Take two (non-complex) numbers as arguments and return a pair of numbers consisting of their quotient and remainder when using integer division.
<code>enumerate(iterable)</code>	Return an enumerate object. iterable must be a sequence
<code>eval(expression)</code>	The expression argument is parsed and evaluated as a Python expression
<code>exec(object)</code>	This function supports dynamic execution of Python code and object must be either a string or a code object.
<code>filter(function, iterable)</code>	Construct an iterator from those elements of iterable for which function is true.

²¹ <https://docs.python.org/3/library/functions.html>

<code>float()</code>	Return a floating-point number constructed from a number or string x.
<code>format(value, format_spec=' ')</code>	Convert a value to a “formatted” representation, as controlled by <code>format_spec</code> .
<code>frozenset()</code>	Returns a new frozenset object.
<code>getattr(object, name)</code>	Return the value of the named attribute of object. name must be a string.
<code>globals()</code>	Return the dictionary implementing the current module namespace.
<code>hasattr(object, name)</code>	The arguments are an object and a string. The result is True if the string is the name of one of the object’s attributes, False if not.
<code>hash(object)</code>	Return the hash value of the object (if it has one).
<code>help()</code>	Invoke the built-in help system.
<code>hex(x)</code>	Convert an integer number to a lowercase hexadecimal string
<code>id(object)</code>	Return the “identity” of an object.
<code>input()</code>	Used to take input from user.
<code>int()</code>	Return an integer object constructed from a number or string x
<code>isinstance(object, classinfo)</code>	Return True if the object argument is an instance of the classinfo argument.
<code>issubclass(class, classinfo)</code>	Return True if class is a subclass of classinfo.
<code>iter(object)</code>	Return an iterator object.
<code>len(s)</code>	Return the length (the number of items) of an object.
<code>list(object)</code>	Returns a list of the object, if it is a sequence type.
<code>locals()</code>	Update and return a dictionary representing the current local symbol table.
<code>map(function, iterable)</code>	Return an iterator that applies function to every item of iterable, yielding the results.
<code>max(iterable)</code>	Return the largest item in an iterable or the largest of two or more arguments.
<code>memoryview(object)</code>	Allows direct read and write access to the object's byte-oriented data, offering performance benefits and avoiding unnecessary copies.
<code>min(iterable)</code>	Return the smallest item in an iterable or the smallest of two or more arguments.
<code>next(iterator)</code>	Retrieve the next item from the iterator by calling its <code>__next__()</code> method.
<code>oct(x)</code>	Convert an integer number to an octal string prefixed with “0o”.
<code>open(file, mode)</code>	Open a file with the permission specified as mode.
<code>ord(c)</code>	Given a string representing one Unicode character, return an integer representing the Unicode code point of that character.
<code>pow(base, exp)</code>	Return base to the power exp.
<code>print(*objects)</code>	Print an object
<code>property()</code>	Return a property attribute.
<code>range(stop)</code>	Rather than being a function, range is actually an immutable sequence

	type.
<code>repr(object)</code>	Return a string containing a printable representation of an object.
<code>reversed(seq)</code>	Return a reverse iterator
<code>round(number, ndigits)</code>	Return number rounded to ndigits precision after the decimal point.
<code>set()</code>	Return a new set object
<code>setattr(object, name, value)</code>	The function assigns the value to the attribute, provided the object allows it.
<code>slice(stop)</code>	Return a slice object representing the set of indices specified by range(start, stop, step).
<code>sorted(iterable)</code>	Return a new sorted list from the items in iterable.
<code>staticmethod()</code>	In Python 3, it is a decorator <code>@staticmethod</code> , transform a method into a static method.
<code>str(object)</code>	Return a string version of an object.
<code>sum(iterable)</code>	Returns the sum of an iterable
<code>super()</code>	Allows you to access and call methods from a parent (super) class within a subclass (child class).
<code>tuple()</code>	Returns a tuple
<code>type(object)</code>	Returns the type of an object
<code>vars(object)</code>	Return the <code>__dict__</code> attribute for a module, class, instance, or any other object with a <code>__dict__</code> attribute.
<code>zip(*iterables)</code>	Iterate over several iterables in parallel, producing tuples with an item from each one.
<code>__import__()</code>	Invoke by import statement.

Key Takeaway's

- Keywords are reserved words in Python, and they are defined for specific purpose.
- It is possible to test a string as keyword or not by importing keyword module.
- Commenting your code is an important part of documentation and python provides commenting as single line or multi-line.
- In python some data types are comes with built-in and some of them we can create using class as user defined.
- `int()`, `float()`, `complex()`, `Decimal()`, `Fraction()` constructor methods are used to from a respective type.
- In Python Integers supports variables length precision, it depends on the available memory.
- Python uses a fixed number of bytes (8 bytes) to represent floats. Therefore, it can represent some numbers in binary approximately.
- It is necessary to import decimal module to create an object of Decimal type and pass the number with required digits as string.
- A factional type can be converted to floating-point, integer, string and complex type using built-in constructor methods.

- Converting a Fractional to Decimal using Decimal() will raise TypeError.
- Python supports mutable and immutable objects based on their memory referencing during the program execution.
- Lists, sets and dictionary are mutable type sequences, can be updated later.
- Frozenset, tuple and string objects can't be altered later and considered as immutable objects.

*****END OF CHAPTER*****

4 OPERATORS

In programming, operators are symbols or keywords that represent certain actions to be taken on data. They are used to perform mathematical, logical, and other operations on values and variables in a program. Python, as a versatile programming language, provides a wide range of operators for different types of data and operations. The use of operators is fundamental to programming, and understanding how to use them correctly is essential for developing efficient and effective code. Python operators come in various types, such as arithmetic, comparison, logical, bitwise, and more. Each type of operator has its own set of rules and syntax that govern its usage in a program.

In this chapter, we will explore the different types of operators available in Python and how to use them. We will also delve into the importance of operators in programming and how they improve the efficiency and readability of code. Additionally, we will cover practical examples of using operators in Python and how they can be used in real-world applications.

Python provides a variety of operators, such as arithmetic, comparison, assignment, logical, bitwise, membership, and identity. Table 4.1 lists the importance of each of them, and later in the subsequent sections, we have covered the detailed working process with coding examples.

Table 4.1 Importance and types of Python Operators

Operator Types	Importance	Example
Arithmetic	Perform basic arithmetic operations on numbers.	$a + b, c * d, e / f, a - b$
Comparison	Compare two values and return a Boolean value based on the comparison.	$a > b, a == b, c! = d, e > f, g <= f$
Logical	Combine multiple conditions and return a Boolean value based on the combined conditions.	$a \text{ and } b, c \text{ or } d, \text{not } e$
Bitwise	Used to manipulate binary data and perform bitwise operations	$a \& b, a b, a \gg 2$
Assignment	Used to assign values to variables and modify their values	$a = b, c += d, e -= f, g *= h, o \&= p$
Membership	Used to determine whether a value is present in a sequence or not.	$a \text{ in } b, c \text{ not in } d$
Identity	Used to determine whether two variables refer to the same object type or not	$a \text{ is } b, c \text{ is not } d$

4.1 Arithmetic Operators

In Python, there are nine arithmetic operators broadly categorised into two groups, such as unary and binary depending upon the number of operands required. Unary operators are a special case of arithmetic operators that operate on a single operand. In Python, there are two unary arithmetic operators:

- **Unary plus (+):** This operator simply returns the value of its operand, which can be a positive or negative number. It is rarely used since it does not change the value of its operand.
- **Unary minus (-):** This operator negates the value of its operand, effectively changing the sign of the number. It can be used to represent negative numbers or to perform operations such as subtraction by adding the negated value.

```
1 >>>a = 10
2 >>>b = +a #unary plus
3 >>>print(b)
4 10
5 >>>c = -a #unary minus
6 >>>print(c)
7 -10
```

In the second line, the unary plus (+) operator is used to assign the value of **a** to **b**. Since + is a unary operator in this case, it simply returns the value of **a** as it is, without changing its sign. Therefore, the value of **b** is also 10. In the fifth line, the unary minus (-) operator is used to assign the negation of variable **a** to **c**. This means that the value of **c** is -10, but the value of **a**, remain unchanged.

The use of unary operators is particularly useful when dealing with arithmetic operations on variables. For example, the unary minus operator can be used to invert the sign of a variable and unary '+' operator is less commonly used but can be used to enforce positive values in certain calculations.

In Python, there are seven binary arithmetic operators that can be used to perform basic arithmetic operations on numeric data types such as integers, floating-point numbers, and complex numbers. The details of each one of them is listed in Table 4.2.

Table 4.2 List of binary arithmetic operators

Operator	Purpose	Description
+	Addition	It is used to add two operands. For example, if $a = 20$ and $b = 10$, then $c = a + b$, which results in 30.
-	Subtraction	It is used to subtract the second operand from the first operand. If the first operand is less than the second operand, the result is negative. For example, if $a = 20$ and $b = 10$, then $c = a - b$, which is 10.
/	Division	It returns the quotient after dividing the first operand by the second operand. For example, if $a = 25$ and $b = 10$, then $c = a / b$ results in 2.50 as a floating-point type.
*	Multiplication	It is used to multiply one operand with the other. For example, if $a = 20$ and $b = 10$, then $c = a * b$ would be 200.
%	Modulo	It returns the remainder after dividing the first operand by the second operand. For example, if $a = 21$ and $b = 10$, then $c = a \% b$ will be 1.

**	Exponent	It is an exponent operator represented as it calculates the first operand raised to the power of the second operand. For example, if $a = 2$ and $b = 3$, then $c = a ** b$ results in 8.
//	floor division also called integer division	It gives the floor value of the quotient produced by dividing the two operands. For example, if $a = 25$ and $b = 10$, then $a // b$ returns 2 as an integer.

See the example listed in code Listing 4. 1 and suggesting you write the complete code in Python interpreter to understand the behavior of arithmetic operators.

Listing 4. 1 Sample program of arithmetic operators

```
>>>a = 10
>>>b = 5
>>>print(a+b) #addition
15
>>>print(a-b) #subtraction
5
>>>print(b-a) #subtraction
-5
>>>print(a*b) #multiplication
50
>>>print(a/b) #division
2.0
>>>print(a//b) #floor division
2
>>>print(a**b) #exponent
100000
>>>print(a%b) #remainder
0
```

The result of standard division (a/b) is always a floating-point type, even if the dividend is evenly divisible by the divisor. See the example given below:

```
>>>a = 12
>>>b = 3
>>>print(a/b, type(a/b))
4.0 <class 'float'>
```

Floor division ($//$) returns the largest integer that is less than or equal to the quotient of the division. When the result of floor division is positive, the result is as though the fractional part of the quotient is truncated, leaving only the integer part. When the result is negative, the result is rounded towards negative infinity, meaning it is rounded down to the next largest (more negative) integer. See the example given below:

```
>>>a = -25
>>>b = 10
>>>print(a//b)
-3
```

In the above example, $a//b$ returns -3, as -2.5 rounds down to -3 as more negative. The programming examples 4.1 to 4.3 will add clearer idea to it.

Program 4.1: Write a program to take two integers as input and using arithmetic operator swap their values without using third variable.

```

1 # Take input from the user
2 a = int(input("Enter the first integer: "))
3 b = int(input("Enter the second integer: "))
4
5 # Print the original values
6 print("Original values: a = ", a, " and b = ", b)
7
8 # Swap the values without using a third variable
9 a = a + b
10 b = a - b
11 a = a - b
12
13 # Print the swapped values
14 print("Swapped values: a = ", a, " and b = ", b)

```

In this program, first two integers are taken as input from the user (line 2 and 3) using `input()` function and store them in variables `a` and `b`. Then original values of `a` and `b` are printed using the `print()` function at line 6. Next, their values are swapped without using any third variable and exchanged through subtraction and addition operations (line 9-10).

First, the values of `a` and `b` are added and saved it in `a`. Then, the value of `b` is subtracted from `a`, and store it in `b`. Finally, we subtract `b` from '`a`' again and store the result in `a`. At last, the swapped values of `a` and `b` is printed using the `print()` function.

Program 4.2: A program to calculate the Semester Grade Point Average in a 10 point scale by consider the following steps:

- The number of subjects is 3 with credits 2, 3, and 3.
- Take input of the grade of three subjects that you have scored out of a 10 point scale.
- Multiply the grade of each subject with its credit and add them.
- Add the credit of each subject.
- Finally, divide the total score by the total credits to get the Semester Grade Point Average.

```

1 # Taking input of the grade of three subjects out of 10 point scale
2 subject1_grade = float(input("Enter grade of subject 1 of 10: "))
3 subject2_grade = float(input("Enter grade of subject 2 of 10: "))
4 subject3_grade = float(input("Enter grade of subject 3 of 10: "))
5
6 # Multiplying each subject grade with its credit
7 subject1_score = subject1_grade * 2
8 subject2_score = subject2_grade * 3
9 subject3_score = subject3_grade * 3
10
11 # Calculating total score and total credit
12 total_score = subject1_score + subject2_score + subject3_score
13 total_credit = 2 + 3 + 3
14
15 # Calculating Semester Grade Point Average
16 sgpa = total_score / total_credit
17
18 # Printing the SGPA
19 print("Your SGPA is:", round(sgpa, 2))

```

Program 4.3: Write a program that takes the principal amount, rate of interest, and tenure of the loan in months as input from the user, and calculates the simple interest and total amount to pay.

```

1 # taking input from user
2 principal = float(input("Enter the principal amount: "))
3 rate = float(input("Enter the rate of interest: "))
4 tenure = int(input("Enter the loan tenure in months: "))
5
6 # calculate simple interest
7 interest = (principal * rate * tenure) / 100
8
9 # calculate total amount to pay
10 total_amount = principal + interest
11
12 # display the results
13 print("Simple interest = ", interest)
14 print("Total amount to pay = ", total_amount)

```

First, input is taken from user for the principal amount, rate of interest, and loan tenure in months using the `input()` function (at line 2-4). Then simple interest is calculated (at Line 7) using the formula $(P * R * T) / 100$, where **P** is the principal amount, **R** is the rate of interest, and **T** is the loan tenure in months and save this value in the variable `interest`. Later at Line 10, `total_amount` is calculated by adding principal and interest and displayed the results to the user at Line 13-14.

Program 4.4: Write a program to calculate the compound interest on given principal amount (P), rate of interest (R) and time (T). The compound interest formula, $C.I = P \left(1 + \frac{r}{100}\right)^n - P$, can be used, where **n** represents number of times interest is compounded in a year. Calculate the amount to be repaid on a loan of Rs. 12000 for one and half years at 10% per annum compounded half yearly?

```

p = int(input('Enter Principla Amount:'))
r = float(input('Enter rate of interest:'))
t = int(input('Enter time in months:'))
#duration of compounding interest
com_in_months =int(input('Enter Compounding period:'))
#re-calculating roi based on compounding time
roi_in_com_in_months = com_in_months/12 * r
n = t/com_in_months

total_amount = round(p * ((1+(roi_in_com_in_months/100))**n),2)
print(f'Total Amount to be paid: {total_amount}')
print(f'Interest on Principal: {total_amount-p}')

```

```

Enter Principla Amount:12000
Enter rate of interest:10
Enter time in months:18
Enter Compounding period:6
Total Amount to be paid: 13891.5
Interest on Principal: 1891.5

```

Program 4.5: Write a program to take input an integer n and calculate the expression $n + n^2 + n^3$.

```
1 n = int(input("Enter a number: "))
2 result = n + n**2 + n**3
3 print("Result: ", result)
```

To calculate the result of the expression $n + n^2 + n^3$ the exponent operator ****** is used to raise n to the power of 2 and 3 respectively at line 2, and add the results together. In Python, besides the numeric data types, arithmetic operators can also be used with certain other object types, listed in Table 4.3:

Table 4.3 Use of arithmetic operators with other object types

Object Type	Arithmetic Operator	Purpose
list, tuple, string	+	Can be used to concatenate two objects of same sequence type. For example, <i>"hello" + "all"</i> , results <i>"helloall"</i> .
list, tuple, string	*	Can be used to repeat a sequence a certain number of times. For example, <i>list1 * 2</i> , where list1 is an object of list type.
Set	-	Can be used to find the difference between two sets.
Boolean	+, -, *, %, /, //, **	All the binary arithmetic operators are applicable and resultant outcome will be of int type.

4.2 Comparison Operator

In programming comparison operators allow us to compare values and determine the relationship between them. This helps in decision-making and control flow in programs. For example, we can use comparison operators to check if a value is equal to, greater than, less than, or not equal to another value, and based on the result, we can take appropriate actions.

Comparison operators are used in conditional statements, loops, and functions to make decisions based on the comparison result. For instance, in an if statement, we can use comparison operators to test whether a condition is true or false and execute certain code based on that result. Similarly, in a loop, we can use comparison operators to check if the loop should continue or terminate based on the value being processed (discussed in Chapter 6).

In summary, comparison operators are essential in programming as they provide the means to compare values, make decisions based on those comparisons, and control the flow of the program. The details of each comparison operators are listed in Table 4.4. Use of each of them is listed in code Listing 4.2.

Table 4.4 Details and working principal of comparison operators

Operators	Description
equal to (=)	Returns True if the two values being compared are equal, and False otherwise.
not equal to (!=)	Returns True if the two values being compared are not equal, and False otherwise.
greater than (>)	Returns True if the first value is greater than the second value, and False otherwise.
less than (<)	Returns True if the first value is less than the second value, and False otherwise.
greater than or equal (>=)	Returns True if the first value is greater than or equal to the second value, and False otherwise.

less than or equal (<=)	Returns True if the first value is less than or equal to the second value, and False otherwise.
-----------------------------------	---

Listing 4.2 Illustration of comparison operators

```
>>>a = 10
>>>b = 5
>>>print(a > b) #True relation
>>>print(a < b) #False relation
>>>print(a >= b) #True relation
>>>print(a <= b) #False relation
>>>print(a == b) #False relation
>>>print(a != b) #True relation
```

In this program, two variables **a**, **b** are initialized with the values 10 and 5, respectively. Each of the comparison operators (**>**, **<**, **>=**, **<=**, **==**, **!=**) are used to compare the values of **a** and **b** and print out the result either as **True** or **False**. The output of the program will be as follows:

```
True
False
True
False
False
True
```

These set of operators always returns the relation between two operands either as **True** or **False** of **bool** type. See the next illustration which has verified this:

```
>>>print(type(10>12))
<class 'bool'>
```

Note: When comparing floating-point values in Python for equality, it's important to keep in mind that due to the way computers represent floating-point numbers, small rounding errors can occur. As a result, two floating-point values that should be equal may not be exactly equal when compared using the **==** operator. See the example given below:

```
>>>x = 1.1
>>>y = 2.2
>>>print((x+y)==3.3)
False
```

Yikes! The internal representations of the constant 3.3 may not exactly equal to result of, **1.1 + 2.2**, so you cannot rely on the value of the expression to compare with exactly to 3.3.

The preferred way to determine whether two floating-point values are “equal”, we need to consider this small rounding errors. One common approach is to use a tolerance or “epsilon” value, which is a small value that represents the maximum allowable difference between the two values for them to still be considered equal. Consider the example in Listing 4.3, demonstrates how to compare two floating-point values for equality in Python, using an epsilon value:

Listing 4.3 Comparing floating-point values

```

1 a = 0.1 + 0.2
2 b = 0.3
3 epsilon = 1e-10 # set a small epsilon value
4
5 # compare a and b for equality using epsilon
6 if abs(a - b) < epsilon:
7     print("a and b are equal")
8 else:
9     print("a and b are not equal")

```

To compare *a* and *b* for equality, we have subtracted *b* from *a* and take the absolute value of the difference using the `abs()` function, and compare it to the `epsilon` value using the `<` operator. If the absolute difference between *a* and *b* is less than the `epsilon` value, we consider them as equal.

Note: The choice of `epsilon` value depends on the specific application and the precision required. A smaller `epsilon` value provides greater precision but may also lead to false negatives, while a larger `epsilon` value provides less precision but may also reduce the likelihood of false negatives.

In Python, comparison operators can also be used with non-numeric object types such as strings, lists, tuples, sets, and dictionaries. The comparison is done based on the lexicographical ordering of the elements in the objects. For example, when comparing strings, Python compares the ASCII values of each character in the strings. When comparing lists, Python compares the elements in the lists from left to right, one by one. Similarly, for tuples, sets, and dictionaries, the comparison is done based on the elements contained in them.

4.3 Logical Operator

The importance of logical operators lies in their ability to simplify complex decision-making processes in programs. By combining multiple conditions, programmers can write more concise and readable code that is easier to understand and maintain. Logical operators also play a crucial role in control flow statements, such as loops and conditional statements (discussed in Chapter 6), by allowing programs to perform different actions based on specific conditions.

Like other programming languages Python also offers three logical operators, such as, **and**, **or** and **not** operators. Out of these logical **and**, **or** operators takes two operands and logical not takes one operand. A simple logical and, or expression can be formed by including any object type, as follows:

operand1 and operand2

operand1 or operand2

In general, expression with comparison operators is considered as operands for logical operators. Like comparison operator's logical operators also evaluate every expression and return the outcome as Boolean True or False. We have summarized the outcome of logical and, or operators in Table 4.5.

Table 4.5 Working of logical and, or operators

Operand1	Operand2	And	or
False	False	False	False
True	False	False	True

False	True	False	True
True	True	True	True

The True and False values of Table 4.5 are the outcomes of any comparison expression or values of any Python object type that can be treated as True and False values. The complete working process of logical and, or operators are explained next with suitable coding examples.

```
1 a, b = 10, 20
2 operand1 = a == b
3 operand2 = a > b
4 print(f'{operand1} and {operand2} => {operand1 and operand2}')
5 print(f'{operand1} or {operand2} => {operand1 or operand2}')
```

False and False => False
False or False => False

In the above code, both the comparison expression at line 2, 3 evaluates to False and according the Table 4.5 the result of logical and, or operation with both the operands as False conditions will also be False, as it is verified from the output as well. Let's see the second case in the next example:

```
1 a, b = 10, 20
2 operand1 = a != b
3 operand2 = a > b
4 print(f'{operand1} and {operand2} => {operand1 and operand2}')
5 print(f'{operand1} or {operand2} => {operand1 or operand2}')
```

True and False => False
True or False => True

For the logical and operator if any of the operand represent a **False** value, the outcome of the logical and operator will be **False**. But for the logical or operator if any of the operand evaluates to **True**, then the outcome of the logical or operator will be **True**.

In the above code, the logical operators 'and' and 'or' are used with two operands 'operand1' and 'operand2'. The value of 'operand1' is assigned as the result of the expression 'a != b', which is True since the value of 'a' is not equal to the value of 'b'. The value of 'operand2' is assigned as the result of the expression 'a > b', which is False since the value of 'a' is not greater than the value of 'b'.

The first print statement (at line 4) uses the logical 'and' operator to evaluate the expression 'operand1 and operand2', which is **False** since 'operand2' is **False**. The second print statement (at line 5) uses the logical 'or' operator to evaluate the expression 'operand1 or operand2', which is **True** since 'operand1' is **True**. Let's test the final condition as both the operand as **True**.

```

1 a, b = 10, 20
2 operand1 = a != b
3 operand2 = a < b
4 print(f'{operand1} and {operand2} => {operand1 and operand2}')
5 print(f'{operand1} or {operand2} => {operand1 or operand2}')
```

```
True and True => True
```

```
True or True => True
```

If both the operands of logical and, or operators are True then, final output of operands are also True, as it can be verified from the output of above code. For a programmer, it is important to understand the evaluation order of the operands used with logical and, or operators. In Python, logical and, or operators are evaluated from left to right, as follows:

- When using the logical and operator, if the first operand evaluates to False, the second operand is not evaluated because the expression is already known to be False.
- When using the logical or operator, if the first operand evaluates to True, the second operand is not evaluated because the expression is already known to be True.

The logical not operator in Python is represented by the "not" keyword. It takes a single operand and returns the opposite Boolean value. It returns the inverted truth value for the operand and its general syntax is as follows:

not operand

Here's an example that uses the logical not operator:

```

1 x, y = 10, 5
2
3 # Evaluates to False, because x is greater than y
4 result1 = x < y
5 # Evaluates to True, because result1 is False
6 result2 = not result1
7 print(result1) # False
8 print(result2) # True
```

The values of **x** and **y** is compared using **<** operator, which returns a Boolean value of False because **x** is greater than **y**. Then apply the logical **not** operator to this result using the "**not**" keyword, which returns the opposite Boolean value of **True**. So, the result of **result2** is **True**, because it is the opposite of the original comparison result.

4.3.1 How does Python interpreter consider any value as truthy or falsy?

In Python, any value can be evaluated as either **True** or **False**, depending on whether it represents a "truthy" or "falsy" value. The truth value of an object in Python is determined by its Boolean context, which means the context in which the object is used in a Boolean expression. In general, the following values are considered "falsy" in Python: All the logical operators work with the truth value of their operands – but what exactly is a truth value?

We know that the bool type True represents True and False represents False. Python considers zero to be False and all other numbers, irrespective if they are positive or negative, are considered to be True.

```
>>> print(bool(8))
True
>>> print(bool(0))
False
>>> print(bool(-8))
True
```

In the given example we have checked what type of values are considered as True and False. We can see the output that 8 and -8 considered as True and zero (0) considered as False. In python most of the in-built types considered as False:

- constants defined to be False: None and False.
- zero of any numeric type: 0, 0.0, 0j, Decimal(0), Fraction(0, 1)
- empty sequences and collections: "", (), [], {}, set(), range(0)

Operations and built-in functions that have a Boolean result always return 0 or False for false and 1 or True for true, unless otherwise stated. (Important exception: the Boolean operations or, and always return one of their operands.)

4.3.2 Short-circuiting of logical operators

Logical **and**, **or** operators in Python are short-circuited which means they evaluate only the bare minimum required to get the correct result. For example:

```
>>> 3 < 2 and 4 != 1 and 3 > 1
False
```

Expression `3 < 2` evaluates to False, we know that the output of **and** operator is also False. Then does it make sense to evaluate the remaining expression `4 != 1 and 3 > 1`? Nope it does not make sense, and Python doesn't do it either. It starts evaluating from left to right, as soon as an expression is False, the "and" evaluates to False, skipping the execution of the remaining operands. The same thing happens for the "or" operator:

```
>>> 3 > 2 or 4 != 1 or 3 > 1
True
```

Expression `3 > 2` evaluates to True, then immediately the output of the "or" expression becomes True, ignoring the remaining part of the expressions. This saves spending unnecessary time on evaluating expressions whose output won't affect the final output of the expression anyway.

4.4 Bitwise Operator

A bitwise operator operates on individual bits of a number, rather than whole number. In computers information is stored in binary as 0s and 1s. These operators perform logical operations on these bits directly. The detailed list of bitwise operates are listed in Table 4.6, out of these bitwise NOT (~) requires one operand.

Table 4.6 Bitwise Operators

Operator	Example	Meaning
----------	---------	---------

&	a & b	bitwise and
 	a b	bitwise or
^	a ^ b	bitwise xor (exclusive or)
~	~a	bitwise not
<<	a << b	bitwise left shift
>>	a >> b	bitwise right shift

If you're coming from another programming language such as Java, then you'll immediately notice that Python is missing the unsigned right shift operator denoted by three greater-than signs ($\ggg$). This has to do with how Python represents integers internally. Since integers in Python can have an infinite number of bits, the sign bit doesn't have a fixed position. In fact, there's no sign bit at all in Python.

All binary bitwise operators have a corresponding compound operator that performs an augmented assignment, as listed in Table 4.7:

Table 4.7 Compound Operations with Bitwise Operators

Operator	Example	Equivalent to
&=	A &= b	A = a & b
 =	A = b	A = a b
^=	A ^= b	A = a ^ b
<<=	A <<= b	A = a << b
>>=	A >>= b	A = a >> b

4.4.1 Why Binary Systems are used?

There are an infinite number of ways to represent numbers. Since ancient times, people have developed different notations, such as Roman numerals and Egyptian hieroglyphs. Most modern civilizations use positional notation, which is efficient, flexible, and well suited for doing arithmetic.

A notable feature of any positional system is its base, which represents the number of digits available. People naturally favor the base-ten numeral system, also known as the decimal system, because it plays nicely with counting on fingers.

Computers, on the other hand, treat data as a bunch of numbers expressed in the base-two numeral system, more commonly known as the binary system. Such numbers are composed of only two digits, zero and one.

For example, the binary number 10011100_2 is equivalent to 156_{10} in the base-ten system. Because there are ten numerals in the decimal system—zero through nine—it usually takes fewer digits to write the same number in base ten than in base two.

The binary system requires more storage space than the decimal system but is much less complicated to implement in hardware. While you need more building blocks, they're easier to make, and there are fewer types of them. That's like breaking down your code into more modular and reusable pieces.

More importantly, however, the binary system is perfect for electronic devices, which translate digits into different voltage levels. Because voltage likes to drift up and down due to various kinds of noise, you want to keep sufficient distance between consecutive voltages. Otherwise, the signal might end up distorted.

By employing only two states, you make the system more reliable and resistant to noise. Alternatively, you could jack up the voltage, but that would also increase the power consumption, which you definitely want to avoid.

4.4.2 How do Computers uses Binary?

Before any piece of information can be reproduced in digital form, you have to break it down into numbers and then convert them to the binary system. For example, plain text can be thought of as a string of characters. You could assign an arbitrary number to each character or pick an existing character encoding such as ASCII, ISO-8859-1, or UTF-8.

In Python, strings are represented as arrays of Unicode code points. To reveal their ordinal values, call **ord()** on each of the characters:

```
>>> msg = 'Hello'
>>> for character in msg:
...     print(f'{character} = {ord(character)}')
...
H = 72
e = 101
l = 108
l = 108
o = 111
```

The resulting numbers uniquely identify the text characters within the Unicode space, but they're shown in decimal form. To get binary version of any number we can use the **bin()**.

```
>>> msg = 'Hello$'
>>> for character in msg:
...     print(f'{character} = {bin(ord(character))}')
...
H = 0b1001000
e = 0b1100101
l = 0b1101100
l = 0b1101100
o = 0b1101111
$ = 0b100100
```

Notice that bit-length, which is the number of binary digits, varies greatly across the characters. The dollar sign (\$) requires six bits, while the rest of the characters can comfortably fit on seven bits. We can also check the number of bits of an integer using **bit_length()**.

```
>>> msg = 'Hello$'
>>> for character in msg:
...     print(f'{character} = {bin(ord(character))}, \
...     bit_length = {ord(character).bit_length()}')
...
H = 0b1001000, bit_length = 7
e = 0b1100101, bit_length = 7
l = 0b1101100, bit_length = 7
l = 0b1101100, bit_length = 7
o = 0b1101111, bit_length = 7
$ = 0b100100, bit_length = 6
```

Variable bit-lengths are problematic. If you were to put those binary numbers next to one another on an optical disc, for example, then you'd end up with a long stream of bits without clear boundaries

between the characters.

One way of knowing how to interpret this information is to designate fixed-length bit patterns for all characters. In modern computing, the smallest unit of information, called an octet or a byte, comprises eight bits that can store 256 distinct values.

4.4.3 Bitwise AND

The bitwise AND operator (&) performs logical conjunction on the corresponding bits of its operands. For each pair of bits occupying the same position of the two numbers, it returns 1 only when both bits are switched on. Check the Table 4.8, for bitwise operations:

Table 4.8 Working process of Bitwise AND Operator

Operand1 (bit)	Operand2 (bit)	Operand1 & Operand2
0	0	0
1	0	0
0	1	0
1	1	1

The resulting bit pattern is an intersection of the operator's arguments. It has two bits turned on in the positions where both operands are ones. In all other places, at least one of the inputs has a zero bit. Arithmetically, this is equivalent to a product of two bit values. You can calculate the bitwise AND of numbers **a**, **b** by multiplying their bits at every index **i**:

$$(a \& b)_i = a_i \times b_i$$

A bit having 1 multiplied by 1 gives 1, but anything multiplied by zero will always result in zero. Alternatively, you can take the minimum of the two bits in each pair. Notice that when operands have unequal bit-lengths, the shorter one is automatically padded with zeros to the left. See the coding example, given next.

```
>>> a, b = 16, 8
>>> c = a & b
>>> print(f'{bin(a)} & {bin(b)} = {bin(c)}')
0b10000 & 0b1000 = 0b0
>>> print(f'{a} & {b} = {c}')
16 & 8 = 0
```

In the above example, we have first printed the output in binary and then in decimal form. At first both numbers will be represented to same bit length as 8 bits. Because decimal 16 and 8 requires 5 and 4 bits respectively. During processing CPU will read byte by bytes, both the numbers need to be represented in 8 bits form and then bitwise AND will be applied over each pair of bits, as shown below:

```

  0  0  0  1  0  0  0  0
&  0  0  0  0  1  0  0  0
  -----
  0  0  0  0  0  0  0  0
```

4.4.4 Bitwise OR

The bitwise OR operator (`|`) performs logical disjunction. For each corresponding pair of bits, it returns 1, if at least one of the bit is switched on. The resulting bit pattern is a union of the operator's arguments. It turned on a bit where either of the operands has a bit as 1. Only a combination of two zeros gives a zero in the final output. See the process listed in Table 4.9.

Table 4.9 Working Process of Bitwise OR Operator

Operand1 (bit)	Operand2 (bit)	Operand1 Operand2
0	0	0
1	0	1
0	1	1
1	1	1

The arithmetic behind it is a combination of a sum and a product of the bit values. To calculate the bitwise OR of numbers **a**, **b**, you need to apply the following formula to their bits at every index **i**, and see the working of bitwise OR, in the given coding example.

$$(b)_i = a_i + b_i - (a_i \times b_i)$$

```
>>> a, b = 16, 8
>>> c = a | b
>>> print(f'{bin(a)} | {bin(b)} = {bin(c)}')
0b10000 | 0b1000 = 0b11000
>>> print(f'{a} | {b} = {c}')
16 | 8 = 24
```

At first both numbers will be represented to same bit length as 8 bits. Then bitwise OR will be applied over each pair of bits, as shown below:

```

  0  0  0  1  0  0  0  0
|  0  0  0  0  1  0  0  0
-----
  0  0  0  1  1  0  0  0
```

4.4.5 Bitwise XOR

The name XOR stands for “exclusive or” since it performs exclusive disjunction on the bit pairs. In other words, every bit pair must contain opposing bit values to produce a one:

Table 4.10 Working of Bitwise XOR Operator

Operand1 (bit)	Operand2 (bit)	Operand1 ^ Operand2
0	0	0
1	0	1
0	1	1
1	1	0

Visually, it's a symmetric difference of the operator's arguments. In Table 4.10, three bits switched on in the result where both numbers have different bit values. Bits in the remaining positions cancel out because they're the same.

Similarly to the bitwise OR operator, the arithmetic of XOR involves a sum. However, while the bitwise OR clamps values at one, the XOR operator wraps them around with a sum modulo two:

$$(a \wedge b)_i = (a_i + b_i) \bmod 2$$

Modulo is a function of two numbers—the dividend and the divisor—that performs a division and returns its remainder. In Python, there's a built-in modulo operator denoted with the percent sign (%).

```
>>> a, b = 34, 12
>>> c = a ^ b
>>> print(f'{bin(a)} ^ {bin(b)} = {bin(c)}')
0b100010 ^ 0b1100 = 0b101110
>>> print(f'{a} ^ {b} = {c}')
34 ^ 12 = 46
```

At first both numbers will be represented to same bit length as 8 bits. Then bitwise XOR applied over each pair of bits, as shown below:

$$\begin{array}{r} 0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1 \ 0 \\ \wedge \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 0 \ 0 \\ \hline 0 \ 0 \ 1 \ 0 \ 1 \ 1 \ 1 \ 0 \end{array}$$

4.4.6. Bitwise NOT

The last bitwise logical operator is the bitwise NOT operator (~), which takes only one operand, making it the only unary bitwise operator. It performs logical negation on a given number by flipping all of its bits:

$$\begin{array}{r} \sim \ 0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1 \ 0 \\ \hline 1 \ 1 \ 0 \ 1 \ 1 \ 1 \ 0 \ 1 \end{array}$$

The inverted bits are a complement to one, which turns zeros into ones and ones into zeros. It can be expressed arithmetically as the subtraction of individual bit values from one:

$$\sim a_i = 1 - a_i$$

While the bitwise NOT operator seems to be the most straightforward of them all, you need to exercise extreme caution when using it in Python. Everything you've read so far assumes that numbers are represented with unsigned integers.

Note: Unsigned data types don't let you store negative numbers such as -273 because there's no space for a sign in a regular bit pattern. Trying to do so would result in a compilation error, a runtime exception, or an integer overflow depending on the language used.

Although there are ways to simulate unsigned integers, Python doesn't support them natively. That

means all numbers have an implicit sign attached to them whether you specify one or not. This shows when you do a bitwise NOT of any number:

```
>>>a = 156
>>>bin(a)
'0b10011100'
```

Now if we flip all the bits of **a** using bitwise NOT, the bits of a will be something like '0b01100011', which is 9910 in decimal, but Python tells different story:

```
>>>~a
-157
```

Remember, one's complement of any positive number will give a negative number, because of change in sign bit. In a computer system, the left most bit is reserved for preserving the sign of any number. If the sign is '+' then left most bit will be 0, else 1 for negative numbers. To store any negative value computer systems uses two's complement of the number and follows the steps given below:

- Extracts all the bits except the sign bit from the resultant binary string, that we received after applying one's complement. So, we have '0b01100011', we didn't consider the sign bit, which is already 1.
- In the next step, we need to find the one's complement of '0b01100011', which will be '0b10011100'.
- In the last step, we will perform binary 1 addition with the binary number received in previous step, which will be '0b10011101' means 15710.
- In the complete process the sign bit remains unchanged and the output is produced after converting the binary string to decimal with a prefixed negative sign.

```
>>>print(0b10011101)
157
```

4.4.7 Bitwise Shift Operators

Bitwise shift operators are another kind of tool for bit manipulation. They let you move the bits around, which will be handy for creating bitmasks later. In the past, they were often used to improve the speed of certain mathematical operations.

Left Shift

The bitwise left shift operator (<<) moves the bits of its first operand to the left by the number of places specified in its second operand. It also takes care of inserting enough zero bits to fill the gap that arises on the right edge of the new bit pattern. Let's see an example,

```
>>> a = 12
>>> a << 1
24
```

In the above code we are shifting the bits of **a** by 1 position towards left and the value 12 will be multiplied by 2 and 24 will be saved into b. So, the print statement will print 24. Let's see a pictorial view of the above example:

Bit position	7	6	5	4	3	2	1	0
Operand 1	0	0	0	0	1	1	0	0
After left shift	0	0	0	1	1	0	0	0

Now the second operand holds the value 1. So, every bits will change their position by 1 towards left. So the new number after shifting of 1 bit is 11000 and the decimal of this number is 24. Shifting a single bit to the left by one place doubles its value. Moving bits in two places to the left will quadruple the resulting value. In general, shifting bits to the left corresponds to multiplying the number by a power of two, with an exponent equal to the number of places shifted:

$$a \ll n = a \times 2^n$$

The left shift used to be a popular optimization technique because bit shifting is a single instruction and is cheaper to calculate than exponent or product. Today, however, compilers and interpreters, including Python's, are quite capable of optimizing your code behind the scenes.

Right Shift

The bitwise right shift operator (>>) is analogous to the left one, but instead of moving bits to the left, it pushes them to the right by the specified number of places. The rightmost bits always get dropped. Let's take an example:

```
>>> a = 39
>>> bin(a)
'0b100111'
>>> b = a >> 1
>>> bin(b)
'0b10011'
```

The above code demonstrates the bitwise right shift (>>) operation. Unlike arithmetic division, a right shift by 1 position on a binary number is equivalent to dividing by 2, its decimal value. The code assigns the shifted value to b. However, it's important to note that the right shift operation discards the least significant bits (those on the right). This effectively truncates the division result, discarding any fractional component.

Bit position	7	6	5	4	3	2	1	0
Operand1	0	0	1	0	0	1	1	1
after right shift	0	0	0	1	0	0	1	1

Every time you shift a bit to the right by one position, you halve its underlying value. Moving the same bit by two places to the right produces a quarter of the original value, and so on. When you add up all the individual bits, you'll see that the same rule applies to the number they represent:

Expression	Binary Value	Decimal Value
a	10011101	157
a >> 1	1001110	78
a >> 2	100111	39
a >> 3	10011	19

Halving an odd number such as 157 would produce a fraction. To get rid of it, the right shift operator automatically floors the result. It's virtually the same as a floor division by a power of two:

$$a \gg n = \lfloor \frac{a}{2^n} \rfloor$$

The bitwise right shift operator and the floor division operator both work the same way, even for negative numbers. However, the floor division lets you choose any divisor and not just a power of two. Using the bitwise right shift was a common way of improving the performance of some arithmetic divisions. You might be wondering what happens when you run out of bits to shift. For example, when you try pushing by more places than there are bits in a number:

```
>>> a = 3
>>> a >> 5
0
```

In this example **a**, is right shifted by 5 bits. But the binary of 3 is 11_2 , which has two bits only. So in this case result will be always zero. Once there are no more bits switched on, you're stuck with a value of zero. Zero divided by anything will always return zero. However, things get trickier when you right shift a negative number because the implicit sign bit gets in the way:

```
>>> a = -2
>>> a >> 5
-1
```

The rule of thumb is that, regardless of the sign, the result will be the same as a floor division by some power of two. The floor of a small negative fraction is always minus one, and that's what you'll get.

4.5 Assignment Operator

You have seen that a single equal sign (**=**) is used to assign a value to a variable. It is, of course, perfectly viable for the value to the right of the assignment to be an expression containing other variables:

```
>>> a = 10
>>> b = 20
>>> c = a * 5 + b
>>> c
70
```

In fact, the expression to the right of the assignment can include references to the variable that is being assigned to:

```
>>> a = 10
>>> a = a + 5
>>> a
15
>>> b = 20
>>> b = b * 3
>>> b
60
```

The first example is interpreted as “**a** is assigned the current value of **a** plus 5,” effectively increasing the value of **a** by 5. The second reads “**b** is assigned the current value of **b** times 3,” effectively increasing the value of **b** threefold. Of course, this sort of assignment only makes sense if the variable in expression **a + 5** or **b * 3** has already previously been assigned a value:

```
>>> var = var / 3
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'var' is not defined.
```

At first line, **var** has been referred to in the expression **var / 3**, which has not defined yet. So doing this is illegal in Python, Else raised a **NameError**.

4.5.1 Shorthand Assignment

A shorthand assignment operator, also called a compound assignment operator, is a way to combine an assignment operation (using the equals sign '=') with another arithmetic or bitwise operator. Python supports a shorthand augmented assignment notation for these arithmetic and bitwise operators as listed below:

Table 4.11 List of Arithmetic and Bitwise Operators Supports shorthand assignment

Arithmetic	Bitwise
+	&
-	
*	^
/	>>
%	<<
//	
**	

For the above listed operators following two expressions are equivalent:

variable operator= expression

variable = variable operator expression

In the above syntax “operator” represent any of the operators from arithmetic and bitwise listed in Table 4.11. Look at the next example, where we have highlighted three types of assignment statement and produce same result for **x** and **y**.

```
>>> x = y = 12 #augmented assignment
>>> x += 2 #compound assignment
>>> y = y + 2 #standard assignment
>>> x, y
(14, 14)
```

4.6 Membership Operator

Python offers two membership operators to check or validate the membership of a value. It tests for membership in a sequence, like a list, tuple, set or string.

Table 4.12 Details of Membership Operators

Operator	Description	Example
in	Returns True if a sequence with the specified value is present in the object	X in Y
not in	Returns True if a sequence with the specified value is not present in the object	X not in Y

The ‘in’ operator is used to check if a character/ substring/ element exists in a sequence or not. Evaluate to True if it finds the specified element in a sequence otherwise, False. For example,

```
>>> l = [2, 3, 4]
>>> 2 in l
True
>>> 10 in l
False
>>> s = (2, 3)
>>> 2 in s
True
>>> 4 in s
False
```

The ‘not in’ operator is the opposite of ‘in’. It returns True, if the specified value is not found in the sequence. Otherwise, it returns False.

```
>>> colors = ["red", "green", "blue"]
>>> "pink" not in colors
True
>>> letters = "Python"
>>> "p" not in letters
True
>>> "P" not in letters
False
```

4.7 Identity Operator

Identity operators compare the memory locations of two objects. There are two Identity operators explained below:

Operator	Description	Example
is	Evaluates to true if the variables on either side of the operator point to the same object and false otherwise.	x is y, here is results in 1 if id(x) equals id(y)
is not	Evaluates to false if the variables on either side of the operator point to the same object and true otherwise.	x is not y, here is not results in 1 if id(x) is not equal to id(y).

In python every object has an identity, a type and a value. An object's identity never changes once it has been created; you may think of it as the object's address in memory. The 'is' operator compares the identity of two objects; the id() function returns an integer representing its identity.

```
>>> a = 20
>>> b = 20
>>> a is b
True
>>> id(a), id(b)
(140715626501512, 140715626501512)
```

This code snippet checks if two variables, a and b, refer to the same object in memory and not just having the same value. The line **a is b**, uses the is operator, checks if a and b refer to the exact same object in memory. In this case, since both variables are assigned the same small integer value (20), Python might optimize by storing only one copy of the value and making both a and b point to that single copy. Both a and b refer to the same memory block as confirmed using **id()**.

For small integers like 20, Python might have an internal pool of pre-allocated memory for these values to improve performance. Assigning the same value to multiple variables allows Python to simply reference the existing object instead of creating a new one in memory for each variable. This behavior depends on the data type, value, and potentially even Python implementation details. Here are some scenarios where **a is b**, might return False:

- **Larger integers or other data types:** For larger integers or other data types like strings or lists, Python might not pre-allocate memory and would likely create separate objects for each variable assignment.
- **Explicit object creation:** If you explicitly create objects using functions or constructors (`a = list(range(5))`, `b = list(range(5))`), even if they have the same content, they will be separate objects.

In the next example we are demonstrating that two objects will refer different memory address.

```
>>> a = 20.5
>>> b = 20
>>> a is b
False
>>> id(a), id(b)
(2424282604688, 140716333962632)
```

In the above example, `a` and `b` are two different type objects, `a` as float and `b` as int. The identity operator return False, as `a` and `b`, referring to different memory blocks. It is also true for large integers. Let us try `is` operator with mutable data types such as list.

```
>>> list1, list2 = [], []
>>> list3 = list1
>>> list1 is list2
False
>>> list1 is list3
True
```

In this code, we have created two empty list `list1` and `list2` and then assigned `list1` to `list3`. The identity of `list1` and `list2` is different even though they are referring empty list. But the identity of `list3` and `list1` are same, because we have assigned the value of `list1` to `list3`. If we used chained assignment instead of multiple assignment then the result will change, and all the list objects will refer to the same memory block.

```
>>> list1 is list2
True
>>> list1 is list3
True
>>> list3 is list2
True
```

`is not` operator evaluates True, if both variables are not the same object. See the example given below:

```
>>> l1 = [1,2,3]
>>> l2 = [1,2,3]
>>> l3 = l1
>>> l1 is not l2
True
>>> l1 is not l3
False
```

The behavior of **is** and **is not** operator also depends on the type of object. Python has two types of objects (i) mutable and (ii) immutable. Details of the mutable and immutable types are discussed in Chapter 3. Types affect almost all aspects of object behavior. Even the importance of object identity is affected in some sense: for immutable types, operations that compute new values may return a reference to any existing object with the same type and value, while for mutable objects this is not allowed.

For example, **a = 1** and **b = 1**, **a** and **b** may or may not refer to the same object with the value one, depending on the implementation of python, but after **c = []**; **d = []**, **c** and **d** are guaranteed to refer to two different, unique, newly created empty lists.

4.8 Operator Precedence and Associativity

While dealing with operators in Python, we must know about the concept of Python operator precedence and associativity as these determine the priorities of the operator otherwise, we'll see unexpected outputs.

4.8.1 Operator Precedence

Imagine an expression with multiple operators like addition, multiplication, and division. Precedence determines which operations are evaluated first. Operators with higher precedence are evaluated before those with lower precedence. This ensures the expression is evaluated in the intended order. Check the example given below:

```
>>> a = 4 + 5 * 3
>>> a
19
```

Here multiplication operator is higher in precedence. So, $5 * 3$ will be evaluated first and 15 will be added with 4. We can also change the order of evaluation using a pair of brackets, as shown below:

```
>>> a = 4 / (5 + 3)
>>> a
0.5
```

In the above code addition is performed over division, because the expression **5+3** surrounded by a pair of parentheses. Here is the order of precedence of the Python operators you have seen so far, from lowest to highest:

Table 4.13 Order of Operator Precedence and Associativity

Precedence	Operator	Description	Associativity
Highest	()	Parenthesis	Left to Right
	x[index], x[index : index]	Subscripting, slicing	Left to Right
	**	Exponentiation	Right to Left
	+x, -x, ~x	unary positive, unary negation, bitwise negation	Right to Left
	*, /, //, %	multiplication, division, floor division, modulo	Left to Right
	+, -	addition, subtraction	Left to Right
	<<, >>	bitwise shift operators	Left to Right
	&	Bitwise AND	Left to Right
	^	Bitwise XOR	Left to Right
		Bitwise OR	Left to Right
	In, not in, ==, !=, <, <=, >, >=, is, is not	Comparisons and identity operators	Left to Right
	not x	Logical not	Right to Left
	And	Logical and	Left to Right
	Or	Logical or	Left to Right
Lowest			

Operators at the top of the Table 4.13 have the highest precedence, and those at the bottom of the table have the lowest. Any operators in the same row of the table have equal precedence. That's why multiplication is performed first, as it is higher precedence than addition.

Similarly, in the next example, 3 raised to the power of 4, will be evaluated first, and then multiplications are carried out in order from left to right ($2 * 81 * 5 = 810$):

```
>>> 2 * 3 ** 4 * 5
810
```

Operator precedence can be overridden using parentheses. Expressions in parentheses are always performed first, before expressions that are not parenthesized. Thus, the following happens:

```
>>> 20 + 4 * 10
60
>>> (20 + 4) * 10
240
```

The two expressions generate different results because of operator precedence in Python. Operator precedence defines the order in which operations are evaluated within an expression. In the first expression, multiplication (*) has higher precedence than addition (+). Therefore, $4 * 10$ is evaluated first (40). Then, 40 is added to 20, resulting in 60. In the second expression parentheses () force the evaluation of $20 + 4$ first (24), then 24 is multiplied by 10, resulting in 240.

There is nothing wrong with making liberal use of parentheses, even when they aren't necessary to change the order of evaluation. In fact, it is considered good practice, because it can make the code more readable, and it relieves the reader of having to recall operator precedence from memory. Consider the following:

```
>>> a, b = 30, 10
>>> (a < 40) and (b > 5)
True
```

Use of parentheses is fully unnecessary, as the comparison operators have higher precedence than **logical and** does and would have been performed first anyhow. But some might consider the intent of the parenthesized version more immediately obvious than this version without parentheses:

```
>>> a < 40 and b > 5
True
```

On the other hand, there are probably those who would prefer the latter; it's a matter of personal preference. The point is, you can always use parentheses if you feel it makes the code more readable, even if they aren't necessary to change the order of evaluation.

4.8.2 Operator Associativity

If an expression contains two or more operators with the same precedence, then operator associativity is used to evaluate the expression. It can either be Left to Right or from Right to Left. Next example shows multiple operators with same precedence. To solve this expression, we have to understand how operator associativity works. Details order of operator associativity is listed in Table 4.13. Some of the coding examples are listed next to understand the working of operator associativity.

```
>>> 4 / 5 * 3
2.4000000000000004
```

In the above expression division operation is evaluated first, which results 0.8 and multiplied with 3. Due to the rule of associativity this expression is evaluated from left to right. Associativity of operators are considered when in a given expression more than one operator are used with same precedence. The right to left associativity can be seen in the next example.

```
>>> 2 ** 3 ** 2
512
```

In the above expression, $3 ** 2$ will be evaluated first, which results in 9 followed by $2 ** 9$ is calculated, using right to left associativity. In python, all the unary (operates over single operand) operators followed right to left associativity, as listed in Table 4.13.

4.8.3 Non-Associative Operators

In Python, non-associative operators are operators where the order of evaluation within an expression doesn't follow the left-to-right or right-to-left rule that applies to most operators. This means you cannot chain them together and expect a well-defined result. Here are the key points about non-associative operators in Python:

- **No defined order of evaluation:** Unlike operators with precedence (like addition and multiplication), non-associative operators have no set order in which they are evaluated. This can lead to unexpected results if you try to chain them together.
- **Limited examples:** There are only a few non-associative operators in Python, primarily assignment operators and some comparison operators used in a specific way.

Operators like `=`, `+=`, `-=`, etc., are non-associative. You cannot combine them like `x = y = z += 1`. This would result in a syntax error.

```
a = b = c += 1
File "<stdin>", line 1
    a = b = c += 1
                ^^
```

SyntaxError: invalid syntax

While comparison operators (`<`, `>`, `<=`, `>=`, `==`, `!=`) themselves are associative (evaluated left-to-right), using them in a chain without parentheses is non-associative. For example, `a < b < c` is not equivalent to `(a < b)` and `(b < c)`. It's generally discouraged to chain.

Key Takeaway's

- Logical operators used to combine conditional statements: `and`, `or`, and `not`.
- Membership Operators are used to test if a sequence is presented in an object: `in`, `not in`.
- Identity Operators are used to compare the memory locations of two objects: `is`, `is not`.
- Operators with higher precedence are evaluated before operators with lower precedence.
- When operators have the same precedence, associativity determines the order of evaluation. Most operators in Python have left-to-right associativity, meaning they are evaluated from left to right.
- Use parentheses `()` to override the default precedence and associativity.
- The exponentiation operator `**` has right-to-left associativity, which is an exception to the common left-to-right rule.
- Unary plus `+` and minus `-` (as in `+x`, `-x`) have higher precedence than the standard arithmetic operators.
- Multiplicative operators (`*`, `/`, `//`, `%`) have higher precedence than additive operators (`+`, `-`).
- All comparison operators have the same precedence, which is lower than that of arithmetic and bitwise operators.
- The logical `not` has higher precedence than `and`, which has higher precedence than `or`

*****END OF CHAPTER*****

5 STRINGS

In Python a string is a sequence of Unicode code points. These code points are converted into a sequence of bytes for efficient storage. This process is called character encoding. There are many encodings such as UTF-8, UTF-16, ASCII etc. and Python uses UTF-8 encoding by default.

5.1 The Story of Unicode

ASCII can only represent a limited number of characters at one time. It isn't very useful to represent any language that isn't based on a Latin character set. ASCII only defines 128 characters. So only 7 bits is normally used in ASCII which is not sufficient to support characters that required more bits to represent. At present every computer program must be scalable enough to take care various international characters and an application needs to be internationalized to display the same output or warning message in a variety of languages in English, French, Japanese, Hebrew, or Russian. Web content can be written in any of these languages and can also include a variety of emoji symbols. Python's string type uses the Unicode Standard for representing characters, which lets Python programs work with all these different possible characters.

However, UTF-8 which is an encoding standard for UCS-4 (Unicode) can represent almost any language. It does this by chaining multiple bytes together to represent one character (or glyph to be more correct). A character is represented on a screen or on paper by a set of graphical elements that's called a glyph. The glyph for an uppercase A, for example, is two diagonal strokes and a horizontal stroke, though the exact details will depend on the font being used. Most Python code doesn't need to worry about glyphs; figuring out the correct glyph to display is generally the job of a GUI toolkit or a terminal's font renderer.

5.1.1 Encoding

Unicode string is a sequence of code points, which are numbers from 0 through 0x10FFFF (1,114,111 decimal). This sequence of code points needs to be represented in memory as a set of code units, and code units are then mapped to 8-bit bytes. The rules for translating a Unicode string into a sequence of bytes are called a character encoding, or just an encoding.

The first encoding you might think of is using 32-bit integers as the code unit, and then using the CPU's representation of 32-bit integers. In this representation, the string "Python" might look like this:

P				y				t				h				o				n			
0x50	00	00	00	79	00	00	00	74	00	00	00	68	00	00	00	6f	00	00	00	6e	00	00	00
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23

This representation is straightforward but using it presents a number of problems. Such as:

- It's not portable; different processors order the bytes differently.
- It's very wasteful of space. In most texts, most of the code points are less than 127, or less than 255, so a lot of space is occupied by 0x00 bytes. The above string takes 24 bytes compared to the 6 bytes needed for an ASCII representation. Increased RAM usage doesn't matter too much (desktop computers have gigabytes of RAM, and strings aren't usually that large), but expanding our usage of disk and network bandwidth by a factor of 4 is intolerable.
- It's not compatible with existing C functions such as `strlen()`, so a new family of wide string functions would need to be used.

Therefore, this encoding isn't used very much, and people instead choose other encodings that are more efficient and convenient, such as UTF-8. UTF-8 is one of the most used encodings, and Python often defaults to using it. **UTF** stands for **"Unicode Transformation Format"**, and the '8' means that 8-bit values are used in the encoding. (There are also UTF-16 and UTF-32 encodings, but they are less frequently used than UTF-8.) UTF-8 uses the following rules:

- If the code point is < 128, it's represented by the corresponding byte value.
- If the code point is >= 128, it's turned into a sequence of two, three, or four bytes, where each byte of the sequence is between 128 and 255.

UTF-8 has several convenient properties:

- It can handle any Unicode code point.
- A Unicode string is turned into a sequence of bytes that contains embedded zero bytes only where they represent the null character (**U+0000**). This means that UTF-8 strings can be processed by C functions such as `strcpy()` and sent through protocols that can't handle zero bytes for anything other than end-of-string markers.
- A string of ASCII text is also valid UTF-8 text.
- UTF-8 is compact; most used characters can be represented with one or two bytes.
- If bytes are corrupted or lost, it's possible to determine the start of the next UTF-8-encoded code point and resynchronize. It's also unlikely that random 8-bit data will look like valid UTF-8.
- UTF-8 is a byte oriented encoding. The encoding specifies that each character is represented by a specific sequence of one or more bytes. This avoids the byte-ordering issues that can occur with integer and word oriented encodings, like UTF-16 and UTF-32, where the sequence of bytes varies depending on the hardware on which the string was encoded.

5.2 String Literals

A string literal is a sequence of characters enclosed in quotes that represents text data in a programming language. It's a fixed value that cannot be changed during the program's execution. String literals are used for various purposes, such as displaying messages, storing text data, and performing string

manipulations. Python offers several ways to create string literals, providing flexibility for different use cases.

5.2.1 Single line string literals

String literals in python are surrounded by either single quotation (‘ ’) marks, or double quotation marks (“ ”). In python single and double-quoted strings are the same and return the same type of object. This type of objects supports only single line string only, as shown below:

```
>>s = 'Hello' # a single quoted string literal
>>print(s)
Hello
>>type(s)
<class 'str'>
>>s = "Hello" # a double quoted string literal
>>print(s)
Hello
>>type(s)
<class 'str'>
```

Python supports triple quoted string literals and a line break is part of the string and using triple quotes string literals are known as a multiline string.

```
>>s = '''This is a multiline
string surrounded by single quotes'''
>>print(s)
```

```
This is a multiline
string surrounded by single quotes
```

```
>>s = """This is a multiline
string surrounded by double quotes"""
>>print(s)
```

```
This is a multiline
string surrounded by double quotes
```

5.2.2 Raw String

A raw string is a type of string that can also include one or many escape characters. A raw string is formed by prefixing an 'r' or 'R', a character following a backslash is included in the string without change, and all backslashes are left in the string. For example, the string literal `r"\n"` consists of two characters: a backslash and a lowercase 'n'.

```
>>> s = r'\n'
>>> print(s)
\n
```

String quotes can be escaped with a backslash, but the backslash remains in the string; for example, `r"\\"` is a valid string literal consisting of two characters: a backslash and a double quote.

```
>>> s = r"\\"
>>> print(s)
\"
```

Any raw string contains only one backslash, `r"\\"` is not a valid string literal (even a raw string cannot end in an odd number of backslashes). Specifically, a raw string cannot end in a single backslash (since the backslash would escape the following quote character). Note also that a single backslash followed by a newline is interpreted as those two characters as part of the string, not as a line continuation.

```
>>> s = r'\
... Hello'
>>> s
'\\nHello'
```

5.3 Escape Characters

Escape characters are used to solve the problem of using special characters inside a string declaration. It directs the interpreter to take suitable actions mapped to that character. We denote escape characters with a backslash (`\`) at the beginning. For example, to put a new line in the string we can add a linefeed (`\n`). Table 5.1, provides information about all the escape characters supported by Python.

Table 5.1 List of Escape Characters and their purpose

Escape Character	Meaning
<code>\'</code>	Represents a single quote
<code>\n</code>	Feeding a newline
<code>\r</code>	Carriage return
<code>\t</code>	Considers a horizontal tab space
<code>\b</code>	A added a backspace
<code>\f</code>	A form feed
<code>\ooo</code>	Representing string by octal value
<code>\xhh</code>	Representing a hex value
<code>\\</code>	Add a backslash
<code>\uxxxx</code>	Represents a 16-bit hex value
<code>\xxxxxxxx</code>	Represents a 32-bit hex value

To add a horizontal tab space between two words '`\t`' need to be inserted between two characters, as follows:

```
>>> print('space between a \t b') #a horizontal space between a, b
space between a      b
```

String containing a `\n` shifted the characters to the next line. In the given example, words after `'\n'` i.e., `'next line'` displayed to the next line.

```
>>print('After new line \n next line')# a newline feed
After new line
next line
```

A string containing a backslash (`\`) can be formed by inserting two backslashes. A string can also be formed using the octal Unicode values for each character. Illustration given below shows an example of forming string literal `'Python'` using octal values:

```
>>ord('P'),ord('y'),ord('t'),ord('h'),ord('o'),ord('n') #Unicode vaules
(80, 121, 116, 104, 111, 110)
>>oct(80), oct(121), oct(116), oct(104), oct(111), oct(110) # Unicode to Octal nums
('0o120', '0o171', '0o164', '0o150', '0o157', '0o156')
>>print('\120\171\164\150\157\156') #Froming a string using octal characters
Python
```

5.4 Modifying String Object

In a nutshell, you can't. Strings are one of the immutable types in Python, meaning not able to be changed. Trying to modify a part of string through indexing (discussed later in this chapter) will generate a `TypeError`.

```
>>str1 = 'Python'
>>str1[0] = 'p'
Traceback (most recent call last):
  File "<pyshell#16>", line 1, in <module>
    str1[0] = 'p'
TypeError: 'str' object does not support item assignment
```

In general, python offers many ways to modify a string. You can usually easily accomplish what you want by generating a copy of the original string that has the desired change in place.

```
>>> str1 = 'Python'
>>> str1 = 'p' + str1[1:]
>>> str1
'python'
```

In the above code at `'p'+str1[1:]`, `str1` has been modified by slicing and concatenation operation (+). Using slicing (`str1[1:]`), we can access any subpart of a string, by proving its boundary. There is also a built-in string method to accomplish this:

```
>>> str1 = 'Python'
>>> str1.replace('P', 'p')
'python'
```

You have already seen the use of plus (+) and multiplication (*) operators with numeric operands in Chapter 4. These two operators can be applied to strings as well. The plus '+' operator concatenates two strings.

```
>>>S1 = 'Python'
>>>S2 = ' Language'
>>>S1 + S2
'Python Language'
>>>'This is String in ' + S1 + S2
'This is String in Python Language'
```

Two or more strings can also concatenate without using '+' operator, as follows:

```
>>>s = 'a' 'b' 'c'
>>>s
'abc'
```

The * operator creates multiple copies of a string. If **S** is a string and **n** is an integer, either of the following expressions returns a string consisting of **n** concatenated copies of **S**:

```
>>> 'abc'*3
'abcbcbcb'
>>> 3*'abc'
'abcbcbcb'
```

The multiplier operand **n** must be an integer and if the multiplier is zero or any negative number then result will be an empty string:

```
>>> s = 'Hello'
>>> s * 0
''
>>> s * -2
''
```

5.4 Comparing Strings

Many operators are used in performing the comparison of strings, such as equality operator (==), and comparison operators like (<, >, <=, >=, !=). Strings are compared, by comparing characters of each string at same position. It will check the character with a lower Unicode and is considered a smaller value character or larger value character, or equal value character. In Python, strings are compared by the comparing of ordinal value of characters. Python uses objects with the same values in memory which makes comparing objects faster. Some basic comparison operators are equal to (==) and 'is' operators.

Next some examples are illustrated to understand the string comparison and while comparing following points need to be consider:

- Comparisons are case sensitive, meaning 'G' is not the same as 'g'.

- Each character in a string has an ordinal value, which is what operators look out for, and not the actual character. For example, 'G' and 'g' have an ordinal value 71 and 103 respectively. So, when compared 'g' becomes greater than 'G'.

The equal comparison '==' operator returns **True**, if two strings are equal, else return **False**:

```
>>>'Python' == 'Python'
True
>>>'Python' == 'python'
False
```

The Boolean value **True** returned because both the operands of '==' are same and in the second comparison, False, return, because 'P' and 'p' are not same. Not Equal (!=) operator is opposite of equal (==) operator and checks for non-equal string objects:

Note: Remember that using = would make the interpreter assume you want to assign one value to another, make sure you are using == operator for comparison.

The less than (<), operator compares the string based on their ordinal values of each character. The less than (<) and other comparison operators requires two operands in the form **str1<str2**.

```
>>> string1 = "Hello"
>>> string2 = "hello"
>>> if string1 < string2:
...     print("string1 is less than string2")
... else:
...     print("string2 is less than string1")
...
string1 is less than string2
```

Consider the outcome of the code snippet, which states that **string1** is less than **string2**. This comparison is performed on the ordinal value of each character. To make this comparison more understandable further two string objects are taken by taking different case in the last character only: See the ordinal values of corresponding characters of each string objects:

```
>>> string1 = "Hello"
>>> string2 = "Hell0"
>>> for i in range(0, len(string1)):
...     print(f"{string1[i]} = {ord(string1[i])}, {string2[i]} = {ord(string2[i])}")
...
H = 72, H = 72
e = 101, e = 101
l = 108, l = 108
l = 108, l = 108
o = 111, 0 = 79
```

From the output, we can conclude that, all the characters are same except the last one i.e., 'o' vs. '0'. The ordinal value of 'o' is 111, greater than uppercase 0, whose ordinal value is 79. It means,

string2 is less than **string1**. Like operator `<`, Python also allows other comparison operators for string comparison based on ordinal values.

String objects can also compare with identity operator **is** and **is not**. As compared to the identity operator, the comparison operator `==` compares the values of string objects and checks for value equality. Whereas **is** operator checks whether both the string objects (operands) refer to the same memory reference or not. Consider the examples:

```
>>> str1 = 'Hello'
>>> str2 = 'Hello'
>>> str1 is str2
True
>>> str1 == str2
True
```

Let's test the identity of two string objects with same value and modifying any of them and check their memory reference with `id` for comparison:

```
>>> str1 = 'Python'
>>> str2 = str1
>>> id(str1), id(str2)
(140718039720784, 140718039720784)
>>> str1 is str2
True
```

Here `str1`, `str2` both referring to the same memory and comparing with operator `is` returns `True`. Now, modify the `str1` and concatenate "3.10" with `str1` and assigned "Python 3.10" to `str3` and compare them:

```
>>> str1 = str1 + " 3.10"
>>> str3 = "Python 3.10"
>>> str1 is str3
False
>>> id(str1), id(str2)
(1859705773744, 140718039720784)
>>> str1 == str3
True
```

Here, **str1** and **str3** both have same value as 'Python 3.10', but they are referring different memory address. So `is` operator returns `False`, but comparison operator `==` returns **True**, because operator `==`, compares values of **str1** and **str3**.

5.5 String Indexing and Slicing

5.5.1 Accessing string through index

We can consider a string object as an ordered set of items and all these items can be accessed using an index. Indexing is a process of accessing individual items using a number known as location or index. In Python, strings are ordered sequences of character data, and support indexing. Individual characters in a string can be accessed by specifying the string name followed by a number in square brackets (`[]`).

Strings are zero (0) index-based, which means the first character can be accessed using the number 0 by passing it inside the square brackets (`[]`). For a string object 'python', indexing for each character are shown below and these characters can be accessed using the index:

0	1	2	3	4	5
p	y	t	h	o	n

```
>>> s = 'python'
>>> s[0], s[1], s[2], s[3], s[4], s[5]
('p', 'y', 't', 'h', 'o', 'n')
```

The index of the last character is one less than the length of the string. The `len(s)`, will return the number of characters and subtracting 1 from it will get the last valid index, as follows:

```
>>> s[len(s)-1]
'n'
```

String indices can also be specified with negative numbers, in which case indexing occurs from the end of the string and the last character can be accessed using index -1, and to refer the second-to-last character by -2 and so on.

```
>>> s[-6], s[-5], s[-4], s[-3], s[-2], s[-1]
('p', 'y', 't', 'h', 'o', 'n')
```

Attempting an index beyond the range as positive or negative results an error and the python interpreter will raise an `IndexError`. For example, the string 'Python', -6 is the lowest negative index and 5 will be the highest positive index.

5.5.2 Getting Substring through slicing

Slicing is an indexing technique used to extract a substring from a string. If `s` is a string, an expression in the form `s[start : end]`, returns the portion of `s` starting with position `start`, and up to but not including position `end`.

```
>>> str1 = 'This is python language'
>>> str1[4:10]
' is py'
```

The above indexing returns the substring ' is py' (from space at the index 4 to index 9, i.e. y). The index of character `t` is 10, can get using `str1[10]`. It is obvious that, the indexing didn't include the character at index 10, because the `slicing [4:10]` returns `10-4=6` characters, not including the end index. If we omit the start index python interpreter by default, consider zero (0) as a start index. It means `str1[0:end]` is equivalent to the slicing `str1[:end]`.

```
>>> str1[:10]
'This is py'
```

Like this, if we omit the `end` index it will consider the length of the string as last index. It means `str1[0:len(str1)]` is equivalent to `str1[0:]`. For a given index `n` ($0 \leq n \leq \text{len}(n)$) the expression `str1[:n]+str1[n:]` will return the string object as same as `str1`. The first operand

`(str1[:n])` of the expression returns a substring from index `0` to `(n-1)` and the second operand `(str1[n:])` will pick the characters from index `n` to `len(info)`.

```
>>> str1[:5] + str1[5:]
'This is python language'
```

For this example, the value of `n` is considered as `5`. So, the string `str1` is sliced at `'This '` and `'is python language'`. Later the plus `+` operator performs the concatenation of these two sliced strings. Omitting both the index will return the same string, not as a copy, it's a reference to the original string:

```
>>> str2 = str1[:]
>>> id(str1), id(str2)
(1348591302928, 1348591302928)
>>> str2
'This is python language'
```

During slicing, if the start index is equal or greater than the end index then interpreter returns an empty string. Negative indices can be used with slicing as well. `-1` refers to the last character, `-2` the second-to-last, and so on, just like indexing.

```
>>> str1[3:3]
''
>>> str1[10:3]
''
>>> str1[-8:]
'language'
```

While slicing a string, a third value as `step` part of the index `str[start : end : step]`, representing a step count or a stride value can be used to get a substring. To add the step, count an extra colon `:` is added after the stop index. This step value represents the number of character(s) that need to be jumped between two consecutive characters of a string. See the examples given below:

```
>>> str1[::2]
'Ti spto agae'
```

Negative step value also allowed, in which case Python steps backward through the string. In this case, the starting index should be greater than the ending index. See the examples given below:

```
>>> str1 = 'Python'
>>> str1[::-1]
'nohtyP'
```

Using negative stride value means the sliced string will be a reverse one, which means from the last character of the string object. There is another use of using a negative stride value and specifically used to reverse a given string. A string can be reversed by using any of these three techniques:

- Considering the last index as the start index with -1 as the stride value `str1[5::-1]`
- By providing only the stride value as -1 while skipping other indexes `str1[::-1]`
- By providing -1 as the start index and stride value while skipping the end index `str1[-1::-1]`

5.6 Built-in String Functions

Python supports many built-in type functions that can be used with a string object. This sections highlights all of them with a description and a sample illustration.

Function name	Description
<code>chr(65)</code>	Returns the Unicode character of the given number.
<pre>>>> chr(65) 'A'</pre>	
<code>ord()</code>	Returns the ordinal value of the given character.
<pre>>>>ord('A') 65</pre>	
<code>len(str)</code>	Returns the length of the string
<pre>>>>len('Hello') 5</pre>	
<code>str('str')</code>	Returns a string object
<pre>>>> str1 = str('Hello') >>> str1 'Hello'</pre>	

5.7 String Methods

Methods are similar to functions. A method is a specialized type of callable procedure that is tightly associated with an object. Like a function, a method is called to perform a distinct task, but it is invoked on a specific object and has knowledge of its target object during execution. The syntax for invoking a method on an object is as follows:

`object.method_name(argument(s))`

In the above syntax, **object** represents any string object and **method_name()** will be any of the method discussed in this section. Few of them may require single or more number of arguments or may not and this list of methods further categorized based of their outcome.

5.7.1 Case Conversion

Methods related to case conversion such as upper to lower or title case or swapping the letter cases are categorized into this group.

Method	Description
<code>str.capitalize()</code>	Converts the first character to uppercase
<pre>>>'python programming'.capitalize() 'Python programming'</pre>	
<code>str.lower()</code>	Converts the string to lowercase letter
<pre>>>'PYTHON'.lower() 'python'</pre>	
<code>str.upper()</code>	Converts the string to uppercase letter
<pre>>>'python'.upper() 'PYTHON'</pre>	
<code>str.swapcase()</code>	Interchange the uppercase to lowercase or vice versa
<pre>>>'Python Programming'.swapcase() 'pYTHON pROGRAMMING'</pre>	
<code>str.title()</code>	Covert the first character of each word to uppercase
<pre>>>'python programming'.title() 'Python Programming'</pre>	

5.7.2 Searching within a String

Python provides various string methods for searching a substring within a given string. Each method in this group supports an optional **start** and **end** arguments. These are interpreted as string slicing and the action of the method is restricted to the portion of the target string starting at position **start** and proceeding up to but not including character position at **end**. If **start** is specified but **end** is not, these methods apply to the portion of the target string from start to the end of the string.

Method	Description
<code>count(sub_string)</code>	Returns an integer specifying the number of occurrences of the given sub_string in a given string. Returns zero if sub_string is not found
<pre>>>'aab bba aab aab ba rta'.count('aa') 3 >>'aab bba aab aab ba rta'.count('bc')</pre>	

0

count(sub_string, start, end) – start and end both are optional and specifying the start and end index of sub_string within a given string. If end index is given then start index become compulsory.

```
>>'aab bba aab aab ba rta'.count('aa',0,10) # with start and end index
```

2

```
>>'aab bba aab aab ba rta'.count('aa',5) # with start index
```

2

startswith(suffix, star, end) Returns True, if the given string starts with **suffix**, else return False. Other two parameters are optional and specifies the search area.

```
>>'Python'.startswith('Py')# Searching in complete string
```

True

```
>>'Hello Python'.startswith('Py',6)# searching within 'Python'
```

True

```
>>'Hello Python'.startswith('Py',0,6) # searching within 'Hello '
```

False

endswith(suffix, star, end) Return True, if the given string ends with **suffix**, else return False. Other two parameters are optional and specifies the search area

```
>>'Python'.endswith('on') #Searching in complete string
```

True

```
>>'Hello Python'.endswith('Py',6) #Searching within Python
```

False

```
>>'Hello Python'.endswith('thon',0,6) #searching within 'Hello '
```

False

find(sub_string, star, end) Returns the start index of the **sub_string**, if it is present in the string. On multiple occurrences returns the lowest index. On absence of optional parameters complete string is considered.

```
>>'sun sun gun gun'.find('sun') # return 0 as lowest index
```

0

```
>>'sun sun gun gun'.find('un',4,10) # searching within 'sun gun gun'
```

5

```
>>'sun sun gun gun'.find('abc',4,10) # return -1 if not found
```

-1

rfind(sub_string, star, end) Returns the index of the **sub_string**, if it is present in the given string. On multiple occurrences returns the highest index.

```
>>'sun sun gun gun'.rfind('sun') # returns 4 as highest index
4
```

<code>index(sub_string, end)</code>	<code>star,</code>	Search the sub_string within a given string and returns the lowest index, same as find() .
-------------------------------------	--------------------	--

```
>>'sun sun gun gun'.index('sun') # return 0 as lowest index
0
```

<code>rindex(sub_string, end)</code>	<code>star,</code>	Search the sub_string within a given string and returns the highest index, same as rfind() .
--------------------------------------	--------------------	--

```
>>'sun sun gun gun'.rindex('sun') # returns 4 as highest index
4
```

The only difference between `index()` and `rindex()` with `find()` and `rfind()` is, through an exception as **ValueError** instead of -1, if the given sub_string is not found in the string, as follows:

```
>>'sun sun gun gun'.index('abc')
Traceback (most recent call last):
  File "<pyshell#17>", line 1, in <module>
    'sun sun gun gun'.index('abc')
ValueError: substring not found
```

Out of `find()` and `index()` method following points need to keep in mind when to prefer one over another:

- If you are sure about the presence of a substring in the given string, you can use either of the methods.
- If you are not sure about the presence of substring use `find()`.
- In case, you are using the `index()` method to find the index of the substring, use exception handling to avoid runtime crashes. The exception mechanism is discussed later.

5.7.2 Matching pattern within a String

This set of methods checks whether a given strings formed with a specific type of characters or not, such as all alphabetic characters, all digits, all upper or lowercase characters or not. Like this, Python also provide many constants defined in a module namely **string**, can be used using **import** statement.

Method	Description
<code>isalnum()</code>	Returns True if the string contains characters from A-Z or a-z or 0-9.

```
>>'Alp45'.isalnum()# characters from A-Z, a-z, 0-9
True'
```

```
>>'abgr'.isalnum()# characters from a-z
True
>>'123'.isalnum()# characters from 0-9
True
>>'are 123'.isalnum()# white_space is included
False
```

isalpha() - Returns True if the given string contains only alphabets.

```
>>'Alp45'.isalpha()#contains numeric values
False
```

isdecimal() - Return True if all characters in the string are decimal characters of base 10. Return False for subscript and superscript numbers.

```
>>> '123'.isdecimal()
True
>>> two = '\u2082'
>>> "H"+two+"O"
'H2O'
>>> two.isdecimal()
False
```

isdigit() - Returns True if the characters are from 0 – 9 only, irrespective of subscript or superscript digits.

```
>>> '2.45'.isdigit()
False
>>> two.isdigit()
True
```

isnumeric() - Used to determine whether a string consists entirely of numeric characters. It covers basic decimal digits, subscript and superscript digits, character with Unicode numeric value and currency numerators

```
>>> '123abc'.isnumeric()
False
>>> text = "\u0661\u0662\u0663" # Arabic-Indic digits
>>> text.isnumeric()
True
```

str.isidentifier() - Returns True if **str**, is a valid python identifier name, means satisfies naming

convention listed in Chapter 3.

```
>>> '_abc23'.isidentifier()
True
>>> '23'.isidentifier()
False
>>> 'else'.isidentifier()
True
>>> '@name'.isidentifier()
False
```

islower() Returns true if, the given string contains all the lowercase characters.

isupper() Returns true if, the given string contains all the uppercase characters.

isprintable() Returns true if the given string, contains all the printable characters.

A printable character is generally considered to be any character that can be displayed on the screen or printed on a physical device. This includes: alphanumeric characters (a-z, A-Z, 0-9), punctuation characters (!, @, #, \$, %, etc.), space character. Nonprintable characters are those characters defined in the Unicode character database as “Other” or “Separator”, excepting the ASCII space (0x20) which is considered printable.

```
>>> '\t'.isprintable()
False
>>> ' '.isprintable()
True
>>> '\n'.isprintable()
False
>>> 'aA01@'.isprintable()
True
```

isspace() – Return True if there are only whitespace characters in the string and there is at least one character, False otherwise.

istitle() – Return True if the string is a titlecased string and there is at least one character, for example uppercase characters may only follow uncased characters and lowercase characters only cased ones. Return False otherwise.

5.7.2 Formatting an output String

This set of methods used to manipulate the whitespace surrounding the text within the string. They are particularly useful for formatting output or creating aligned columns of text.

center(width, fill) - Returns a string containing the given string as centred of the given width and filled with default ASCII space character, else padded with **fill** character. Returns the given string, if the width

is same as the length of the given string.

```
>>'abc'.center(10) # filled with ASCII space
'  abc  '
>>'abcde'.center(3) # returns the same string
'abcde'
>>'abc'.center(10,'*') # center padded with '*'
'****abc*****'
```

expandtabs(tabsize) – Return a copy of the string where all tab characters are replaced by one or more spaces, depending on the current column and the given tab size. It has an optional keyword argument **tabsize** with default value 8.

```
>>>s = 'a\tab\tabc\t'.expandtabs()#Create a string of length 24
>>>s
'a      ab      abc      '
>>>len(s)
24
```

The **expandtabs()** method replaces tab characters (**\t**) in a string with spaces. It considers the tab positions to occur every **tabsize** characters (default is 8, meaning tab positions are at columns 0, 8, 16, and so on). To expand the string, it works as follows:

- The current column position is set to zero.
- The string is examined character by character.
- If the character is a tab (**\t**), one or more space characters are inserted into the resulting string until the current column position reaches the next tab position. (The tab character itself is not copied.)

```
>>>s = 'a\n\r\tab'.expandtabs()
>>>len(s)
13
>>>s
'a\n\r      ab'
```

If the character is a newline (**\n**) or return (**\r**), it is copied and the current column is reset to zero. Any other character is copied unchanged and the current column is incremented by one regardless of how the character is represented when printed.

```
>>>s = '01\t012\t0123\t01234'.expandtabs(4)
>>>len(s)
21
>>>s
'01  012 0123   01234'
```

In the above example, **tabsize** is set to 4 and till encounters a **'\t'**, considering the characters '01' and next two spaces are considered, because tab position is 4. After that, out of 4 spaces 3 are taken by '012' so only one space is added and before the next **'\t'** already 4 or more characters are there and complete **tabsize** 4 is added.

join(iterable) – Return a string which is the concatenation of the strings inside an iterable object.

```
>>>' '.join(['Hello','Python'])#given string is space
'Hello Python'
>>>''.join(['1','2','3'])#no space so concatenate to a number
'123'
```

ljust(width, fillchar) – Return a string left justified and padded with the **fillchar**. Default is ASCII space. Original string will return if width is less than or equal to the length of given string.

```
>>>'abc'.ljust(5)#left aligned with space
'abc  '
>>>'abc'.ljust(5, '*')#left aligned with *
'abc**'
```

rstrip(char) – Return a copy of the string after removing the leading characters. If no **char** passed then leading spaces are removed, else all the occurrences of given characters are removed.

```
>>>'  spacious  '.rstrip()#leading spaces removed
'spacious  '
>>>'www.example.com'.rstrip('cmowz.')#removed all w with first period(.)
'example.com'
```

5.8 String Module

String module provides many constants and can be used to verify string contains all the alphabets, digits, punctuations, and whitespace etc. Table 5.2 listed all such constants with their purpose.

Table 5.2 Various constants available within string module

String Constants	Description
ascii_letters	Returns a string containing all ASCII lower and uppercase
ascii_lowercase	Returns a string containing all ASCII lowercase characters
ascii_uppercase	Returns a string containing all ASCII uppercase characters
digits	Returns a string containing all the digits from 0-9

hexdigits	Returns a string containing all the hex digits 0-9 and a-f and A-F
octdigits	Returns a string containing all the octal digits 0-7
punctuation	String of ASCII characters which are considered punctuation characters
printable	String of ASCII characters which are considered printable by combining <i>digits</i> , <i>ascii_letters</i> , <i>punctuation</i> and <i>whitespace</i>
whitespace	A string containing all ASCII characters that are considered whitespace. This includes the characters space, tab, linefeed, return, form feed, and vertical tab.

The `import string` statement facilitate us to use any of these constants, as follows:

```
>>import string
>>string.ascii_letters
'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ'
>>string.digits
'0123456789'
>>string.printable
'0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ! " #$ % & \
' ( ) * + , - . / : ; < = > ? @ [ \ ] ^ _ ` { | } ~ \t \n \r \x0b \x0c '
>>string.punctuation
'! " #$ % & \ ' ( ) * + , - . / : ; < = > ? @ [ \ ] ^ _ ` { | } ~ '
```

5.9 Formatting a String

Formatting a string is the process of infusing things in the string dynamically and presenting the string. Python supports three ways to format a given string, as follows:

- Using string formatting expression with `%` operator
- Using string formatting method `str.format()` and
- Formatting through **f-string** since ver. 3.6

5.9.1 String formatting using Formatting Expression (%)

Formatting a string, requires to inject a format specifier based on the datatype listed in Table 5. 3.

Table 5. 3 List of Format Specifiers used to format a string

Formatting Type	Format Specifiers	Description
Integer	%d	Print an Integer in decimal 10 base
Character	%c	Convert an integer to its corresponding character

Float-point	<code>%f</code>	Prints a floating-point number in standard decimal format
Exponential	<code>%e</code> or <code>%E</code>	Prints a floating-point number in scientific notation
String	<code>%s</code>	To print a string
Hexadecimal	<code>%x</code> or <code>%X</code>	Prints an integer in lowercase or uppercase hexadecimal format (base 16), respectively.
Octal	<code>%o</code> or <code>%O</code>	Prints an integer in lowercase or uppercase octal format (base 8), respectively.

While using `%` for string formatting is still supported in Python, it's considered an older approach. Here's how to use it with the `print` function:

```
print("Formatted string %s %d" % (value1, value2))
```

The above syntax can be interpreted as follows:

- The string you want to print is enclosed in double quotes.
- Within the string, placeholders for the values you want to insert are marked with `%s` for strings or `%d` for integers (and other format specifiers for different data types).
- After the closing quotation mark, you use the `%` operator again, followed by a parenthesis containing the values you want to insert into the placeholders.
- The order of values in the parenthesis must match the order of placeholders in the string.

```
>>> name = "Bob"
```

```
>>> age = 30
```

```
>>> print("Hello, my name is %s and I am %d years old." % (name, age))
```

```
Hello, my name is Bob and I am 30 years old.
```

Instead of using the placeholders within `print()`, Python also allows us to format a string first and print it next. See the example given below:

```
>>message = 'Hello, %s you are %d years old' % (name, age)
```

```
>>print(message)
```

```
Hello, Bob you are 30 years old
```

Use of parenthesis become compulsory, while passing multiple values to format a string. For single value it become optional.

```
>>> print('Hello, %s'% name)
```

```
Hello, Bob
```

Next example shows how to format a string including numbers in octal and hexadecimal format.

```
>>> a = 0xafe45
```

```
>>> b = 0o3456
```

```
>>> print('%d is equivalent to %x in hexadecimal' %(a, a))
```

```
720453 is equivalent to afe45 in hexadecimal
```

```
>>> print('%d is equivalent to %o in octal' %(b, b))
```

```
1838 is equivalent to 3456 in octal
```

String formatting with floating-point format-specifiers can be displayed in decimal and exponential form.

```
>>> a = 0.000023
>>> b = 111111111111111111.56
>>> 'a = %f, b = %f' % (a, b)
'a = 0.000023, b = 1111111111111111104.000000'
>>> 'a = %e, b = %e' % (a, b)
'a = 2.300000e-05, b = 1.111111e+17'
```

In the exponential form there will be a negative (-) or a positive (+) sign after **e**. A plus (+) sign indicates a large value and minus (-) means represents a very small value. Further we can decide in advance the width of the output, by specifying it with **%** sign. Field width is the width of the entire number and precision is the width towards the right. One can alter these widths based on the requirements. The default **precision** width is set to 6, for floating point numbers.

```
>>> a = 2.34
>>> print('a = %f' % a)
a = 2.340000
```

The default precision width for floating-point numbers using **%f** is indeed 6, as it shown in the output. Based on our requirement the width of the precision can be controlled, as follows:

```
>>> print('a = %.3f' % a)
a = 2.340
```

In the **print()** statement a value 3 is set, after the decimal point (**.**), that controls the width or length of the precision. That's why in the output **a = 2.340** is printed, instead of **a = 2.340000**. The width value should be placed in between **%** and **f**, and always be a positive integer. If the field width, is set to, more than the default or required width, than the data will be right aligns itself to adjust to the specified values.

```
>>> print('a = %f' % a)
a = 2.340000
>>> print('a = %5.3f' % a) #width 5 = as a's length and precision, (3 < 6)
a = 2.340
>>> print('a = %5.9f' % a) #width 5 = as a's length and precision, (9 > 6)
a = 2.340000000
>>> print('a = %6.3f' % a) #width 6 > a's length and precision, (3 < 6)
a = 2.340
>>> print('a = %10.9f' % a) #width 10 > a's length and precision, (9 > 6)
a = 2.340000000
```

Whenever we have used the precision value the output will be aligned either right or left. The first output, aligned to left, because we have not used field or precision width. But the second output, a value 5.3 is set, it means field width = 5 and precision = 3. So the value will be printed up to 3rd decimal place and the field width equal to the formatted numbers total length i.e. 5. That's why aligned to left.

In the 3rd `print()` statement, field width = 5 and precision = 9. Because the width is same as length then output will be left aligned but number of decimals placed is increased to 9 from 6. In the next line, field width = 6 and precision = 3 and the output is right aligned by single space after equal sign. Because, **field width - precision length (3) - length of decimal part (1) - 1 (decimal point) = 1**. In the last print, **field width (10) - precision (9) - length of decimal part (1) - 1 (decimal point) = -1**, output will be left aligned.

For proper alignment, a space can be left blank in the field width so that when a negative number is used, proper alignment is maintained.

```
>>> b = -2.340
>>> print('% .3f' %b)
-2.340
>>> print('%+.3f' %a)
+2.340
```

Like a minus (-) sign get printed for a negative value, similar to this we can also print the plus (+) sign for positive values, by inserting a plus (+) sign after the % sign.

Note: % formatting is not useful for larger string, because once you start using several parameters and longer strings, your code will quickly become much less easily readable and things are starting to look a little messy. To format long formatted string Python offers `str.format()` method.

5.9.2 String formatting using `str.format()` method

A string can be formatted using `str.format()` method and formatted string contain “replacement fields” surrounded by curly braces `{ }`. Anything that is not contained in braces is considered literal text, which is copied unchanged to the output. The syntax for a replacement field contains a field name, conversion character and a format specifier, as shown below:

```
replacement_field = "{"[field_name]!"[conversion]":["format_spec]}"
```

In less formal terms, the `replacement_field` can start with a `field_name`, that specifies the object whose value is to be formatted and inserted into the output of the replacement field. The `field_name` is optionally followed by a `conversion field`, which is preceded by an exclamation point `!`, and a `format_spec`, which is preceded by a colon `:`. These specify a non-default format for the replacement value.

The `field_name`, itself begins with an `arg_name`, that is either a number or a keyword. If it's a number, it refers to a positional argument, and if it's a keyword, it refers to a named keyword argument. Consider examples given below:

```
>>> '{lan} has ver {ver}'.format(lan = 'Python', ver = '3.x')
'Python has ver 3.x'
```

The string is formatted using **arg_name** only and inside the braces **lan** and **ver** is representing the argument name. Instead of **arg_name**, position of the arguments can be used by specifying their position inside the curly braces (**{}**) and by default an empty pair of braces **{}**, represents positional only.

```
>>> '{} has ver {}'.format('Python', '3.x')
'Python has ver 3.x'
>>> '{1} has ver {0}'.format('3.x', 'Python')
'Python has ver 3.x'
```

Position index starts from 0 and using argument positions can be passed in any order. If the numerical **arg_name** in a format string are **0**, **1**, **2** in a sequence, they can all be omitted (not just some) and the argument numbers will be automatically inserted in that order. Mixing of default and manual numbering will raised a **ValueError**.

```
>>> '{} has ver. {1}' .format('Python', '3.x')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: cannot switch from automatic field numbering to manual field specification
```

Further dictionary keys can be used as the position of the argument for formatting a string. See the example given below:

```
>>> language = {'lan': 'Python', 'ver': '3.x'}
>>> '{lan} has ver. {ver}' .format(**language)
'Python has ver. 3.x'
```

The conversion field causes the type correction before formatting the given string. Formatting of any value is done by the **__format__()** method of that value itself. However, in some cases, it is desirable to force a type to be formatted as a string, by overriding its own definition of formatting. This ensures the value is treated as a plain string and formatted according to the format specifiers you provide in the string (like **.2f** for two decimal places). Details of all conversion flag that can be used to convert the string to a specific type is listed in Table 5.4.

Table 5.4 List of flags used in conversion field

Conversion Flags	Description
!s	Calls the str() method
!r	Calls the repr() function, returns a printable character
!a	Calls the ascii() function, return a printable representation of an object. Ignores the non-ascii characters using \x , \u , \U

Using these flags, we can check further a string is in printable form or not. See the coding example listed below:

```

#Converts the string in printable form
>>'{s!r}'.format(s = '¡¢£ Pyðþðþthon')
" '¡¢£ Pyðþðþthon'"
#Converts all non-ascii characters to ascii code
>>'{s!a}'.format(s = '¡¢£ Pyðþðþthon')
"'\xa1\xa2\xa3 Py\xbc\xbd\xbethon'"
#Strings of non-ascii characters
>>' \xa1\xa2\xa3'
'¡¢£'

```

The last **format_spec**, in **replacement_field**, contains a specification of how the value should be presented, including such details as field width, alignment, padding, decimal precision and so on. Each value type can define its own “formatting mini-language” or interpretation of the **format_spec**.

A **format_spec** field can also include nested replacement fields within it. These nested replacement fields may contain a field name, conversion flag, and format specification, but deeper nesting is not allowed. The replacement fields within **format_spec** are substituted before the **format_spec** string is interpreted. This allows the formatting of a value to be dynamically specified. A general form of format specifier is given below and in Table 5.5, listed every part of a standard format specifier with description.

```
format_spec = [[fill] align] [sign] [z] [#][0][width] [grouping_option]
[.precision][type]
```

Table 5.5 Various types of Field can be used in format specifier

Field Value	Description
fill	Filled the string with any character when passed with valid align value
align	Explained in Table
sign	‘+’ or ‘-’ or space with numeric values only
width	A positive integer, controlling the length of the formatted string
grouping action	‘-’ or ‘,’
precision	A decimal integer, indicating number of digits after the decimal point
type	Determines how the data should be represented

With an align value any filled character can be preceded that can be any character and defaults to a space if omitted. It is not possible to use a literal curly brace (“{” or “}”) as the fill character in a formatted string literal or when using the **str.format()** method. Various alignment options are listed in Table 5.6:

Table 5.6 Details of alignment options with meaning

Alignment Options	Description
‘<’	Forces the field to be left-aligned within the available space (this is the default for most objects).

'>'	Forces the field to be right-aligned within the available space (this is the default for numbers).
'='	Forces the padding to be placed after the sign (if any) but before the digits. This is used for printing fields in the form '+000000120'. This alignment option is only valid for numeric types. It becomes the default for numbers when '0' immediately precedes the field width.
'^'	Forces the field to be centered within the available space.

Some examples of string formatting based on **align** value with predefined **fill** character and width value (the length of the formatted string) is illustrated next, using keyword argument:

```
#Text left aligned with fill character and specified width with
>>'{text:{fill}{align}20}'.format(fill='#',text = 'AB',align='<')
'AB#####'
#Text centre aligned with fill character and specified width with
>>'{text:{fill}{align}20}'.format(fill='#',text = 'AB',align='^')
'#####AB#####'
```

Note: unless a minimum field width is defined, the field width will always be the same size as the data to fill it, so that the alignment option has no meaning in this case.

```
#width is less than the length of the text
>>'{text:{fill}{align}}'.format(fill='#',text = 'AB', align='^')
'AB'
>>'{text:{fill}{align}3}'.format(fill='#',text = 'AB',align='^')
'AB#'
>>'{text:{fill}{align}4}'.format(fill='#',text = 'AB',align='^')
'#AB#'
```

Formatting any string of numbers with a positive or negative sign can be done by passing the **sign** value just before the **width** of the string. All the possible signs with their details are listed in Table 5.7.

Table 5.7 Details of sign fields

Option	Meaning
'+'	indicates that a sign should be used for both positive as well as negative numbers.
'-'	indicates that a sign should be used only for negative numbers (this is the default behavior).
Space	indicates that a leading space should be used on positive numbers, and a minus sign on negative numbers.

```
>>>'{:+} {: -} {: }'.format(3.56,-2.52,-56)
'+3.56 -2.52 -56'
```

```
>>>'{: -} {: } {: -}'.format(3.56, -2.52, -56)
'3.56 -2.52 -56'
```

The optional literal **z** forces negative zero floating-point values to positive zero after rounding to the format precision. This formatting option is added in version 3.11. In many fields of mathematics negative zero is surprising or unwanted minus (**-**), especially in the context of displaying an (often rounded) numerical result. Using this optional **z**, for floating-point presentation types (f, g, etc., as defined by the format specification documentation), normalized to a negative zero to positive zero.

```
#with z
>>'{:z.2f}'.format(-0.0001)
'0.00'

#without z
>>'{: .2f}'.format(-0.0001)
'-0.00'
```

The **#** option causes the “alternate form” to be used for the conversion. The alternate form is defined differently for different types. This option is only valid for integer, float and complex types. For integers, when binary, octal, or hexadecimal format is used, this option adds the respective prefix **'0b'**, **'0o'**, **'0x'**, or **'0X'** to the output value. See the examples given below:

```
#with # before a type
>>'20 in binary {:#b} {:b}'.format(20, 20)
'20 in binary 0b10100 10100'

#without #
>>'20 in binary {:b}'.format(20)
'20 in binary 10100'
```

In this above illustration a type **'b'** is used for binary number and using **#** just before it prefixes **'0b'** before the number. If the type is not preceded by a **'#'**, then simply returns the conversion without any prefix. The comma (,) option signals the use of a comma for a thousand separator.

```
#with width and thound separator
>>'{: {fill}{align}12,}'.format(1234.8, fill='#', align='<')
'1,234.8#####'

#Thousand separator in integer
>>'{: ,}'.format(1023456)
'1,023,456'
```

The underscore option signals the use as a placeholder to improve the readability for large numbers. It is used as a thousand separator for integer (decimal type) and floating point literals. For other integer presentation types **'o'**, **'x'**, and **'X'**, underscores will be inserted after every 4 digits.

```
>>> '{:_b}'.format(1023456)
'1111_1001_1101_1110_0000'
>>> '{:_d}'.format(1023456)
'1_023_456'
>>> '{:_o}'.format(1023456)
'371_6740'
>>> '{:_X}'.format(1023456)
'F_9DE0'
```

Use of **precision** value with 'f' format specifier for fixed-point notation for displaying a specific number of decimal places. While 'g' format specifier for general notation (choosing between fixed-point or scientific notation for best readability), specifies number of significant digits (leading zeros don't count). In this case, it chooses fixed-point notation rounding to 3 significant digits.

```
>>> '{:.3f}'.format(2.456890000)
'2.457'
>>> '{:.3g}'.format(2.456890000)
'2.46'
```

For string **precision** field indicates the maximum field size - in other words, how many characters will be used from the field content. The **precision** is not allowed for integer presentation types.

```
>>> '{:.3s}'.format('Python') # Taking 3 characters
'Pyt'
>>> '{:0s}'.format('Python') # 0-length string
''
```

Finally, the **type** determines how the data should be presented. The available presentation types for string and integer type are listed in Table 5.8.

Table 5.8 List of available string and integer presentation type

Type	Meaning
's'	String format. This is the default type for strings and may be omitted.
B	Binary format. Outputs the number in base 2.
C	Character. Converts the integer to the corresponding unicode character before printing.
D	Decimal Integer. Outputs the number in base 10.
'o'	Octal format. Outputs the number in base 8.
'x'	Hex format. Outputs the number in base 16, using lower-case letter.
'X'	Hex format. Outputs the number in base 16, using upper-case letters.
N	Number. This is the same as 'd', except that it uses the current locale setting (defined within current system settings) to insert the appropriate number separator characters.

```
>>> "int: {0:d}; hex: {0:x}; oct: {0:o}; bin: {0:b}".format(42)
'int: 42; hex: 2a; oct: 52; bin: 101010'
```

In addition to the above presentation types, integers can be formatted with the floating-point presentation types listed in (except 'n' and **None**) Table 5.9. When doing so, **float()** is used to convert the integer to a floating-point number before formatting.

Table 5.9 Available presentation types for float and Decimal values

Type	Meaning
'e'	Floating point
'E'	Scientific notation. Same as 'e' except it uses an upper case 'E' as the separator character.
'f'	Fixed-point notation
'F'	Fixed-point notation. Same as 'f', but converts nan to NAN and inf to INF.
'g'	General format
'G'	General format. Same as 'g' except switches to 'E' if the number gets too large. The representations of infinity and NaN are uppercased, too.
'n'	Number. This is the same as 'g', except that it uses the current locale setting to insert the appropriate number separator characters.
'%'	Percentage. Multiplies the number by 100 and displays in fixed ('f') format, followed by a percent sign.
None	For float this is the same as 'g', except that when fixed-point notation is used to format the result, it always includes at least one digit past the decimal point.

In addition to the string's **format()** method, a single object may also be formatted, using the syntax **format(object, formatspec)**, built-in function (which the method uses internally). The format specifications listed in Table 5.3 need to pass in place of **formatspec** inside **format()**, to get the formatted string. Next some of the examples are illustrated to help you understand the working of **format()** function.

```
>>format(13, 'o')
'15'
>>format(2345.6, '.2f')
'2345.60'
>>format(10000000, ',')
'10,000,000'
>>format(75, '#b')
'0b1001011'
>>format('wer', '>20')
'                wer'
>>format('wer', '#>20')
'#####wer'
>>format(-0., 'z')
'0.0'
>>format(12, '%')
'1200.000000%'
```

```
>>format(23.687,'e')
'2.368700e+01'
>>format(23.687,'g')
'23.687'
```

5.9.3 String formatting using f-string

Strings in Python are usually enclosed within double quotes (“”) or single quotes (‘’). To create **f-strings**, you only need to add character ‘**f**’ or ‘**F**’ before the opening quotes of the string. For example, “**This**” is a string, whereas **f“This**” is an **f-String**.

When using **f-strings** to display strings, you only need to specify the names of the variables inside a placeholder (**{}**). During the execution, all the names are replaced with their respective values. If you have multiple variables in the string, you need to enclose each of the variable names inside the placeholder. Suppose you have two variables namely **language**, and **school**, and a string can be formed as follows:

```
>>language = 'Python'
>>school = 'Computer Science & Engineering'
>>message = f'We are learning {language} from {school}'
>>message
'We are learning Python from Computer Science & Engineering'
```

Notice how the variables **language** and **school** are replaced with their values. As **f-Strings** are evaluated at runtime, you can evaluate valid Python expressions on the fly. In the next illustration, **num1** and **num2** are two variables. To calculate their product, you may insert the expression **num1 * num2** inside a placeholder, and the expression will be evaluated, as follows:

```
>>num1, num2 = 3, 4
>>print(f'The product of {num1} and {num2} is {num1 * num2}')
The product of 3 and 4 is 12
```

In an **f-string**, **{var_name}**, **{expression}** serve as placeholders for **variables** and **expressions**, and are replaced with the corresponding values at runtime. Any string can be formatted using multiple **f-strings**, remember that you need to place an **f** in front of each line of a multiline string.

```
>>name = "Eric"
>>profession = "comedian"
>>affiliation = "Monty Python"
>>message = (f"Hi {name}. "f"You are a {profession}. "f"You were in {affiliation}.")
>>message
'Hi Eric. You are a comedian. You were in Monty Python.'
```

If we don’t put an **f** in front of each individual string, then placeholder **{}**, is considered as part of the string. A string can also spread over multiple lines, by escaping a return with a backslash ‘\’:

```
>>msg = f'Hello {name} '\
```

```
f'you are a {profession} '\
f'you were in {affiliation}'
>>msg
'Hello Eric you are a comedian you were in Monty Python'
```

The **'f'**, in f-strings may as well stand for “fast.”, f-strings are faster than both **%**-formatting, and **str.format()**. Let’s see how to take care quotation mark within an f-string. While working with quotations just make sure that, similar type of quotation mark on the outside of the f-string as you are using in the expression, as follows:

```
>>f"{'Hello Python'}"
'Hello Python'
>>f'{"Hello Python"}'
'Hello Python'
>>f'''{"Hello Python"}'''
'Hello Python'
```

While formatting a dictionary within a string using f-string the double quotes needs to be surrounded and keys must be retrieve using single quote. Using of single quote will leads to a **SyntaxError**, as follows:

```
>>language = {'name': 'Python', 'ver': 3.11}
>>f'{language['name']} has verion {language['ver']}'
SyntaxError: f-string: unmatched '['
```

The correct f-string will be:

```
>>language = {'name': 'Python', 'ver': 3.11}
>>f'{language['name']} has verion {language['ver']}'
'Python has verion 3.11'
```

There is a small difference between **str.format()** and f-strings. i.e., the difference is in how index lookups are performed. In **str.format()**, index values that do not look like numbers are converted to strings:

```
>>d = {'a': 10, 'b': 20}
>>'a={d[a]}'.format(d=d)
'a=10'
```

In **.format()** method, you need to use keyword arguments that match the placeholder names in the string. Non-numeric keys (like variable names) are treated as literal strings within the curly braces. The f-strings enable direct attribute access using string keys within curly braces **{}**. This means you can use a variable name that corresponds to an object's attribute.

```
>>f'a={d["a"]}'
'a=10'
```

This difference is required because otherwise you would not be able to use variables as index values:

```
>>a = 'b'
#using variable as key
>>f'a={d[a]}'
'a=20'
```

The `.format()` method allows you to insert values into strings using curly braces `{}`. However, it has limitations on the types of expressions you can use within those braces. It's generally recommended to use valid Python expressions within `.format()` to avoid unexpected behavior. If you need more flexibility in formatting expressions, consider using f-strings.

Note: If you need more flexibility in formatting expressions, consider using f-strings n deep dive further to PEP 498²², which covers other details about the `f`-string and `str.format()`.

Key Takeaway's

- Strings in Python are immutable, meaning once created, they cannot be changed.
- Use the `+` operator to concatenate strings.
- Access characters in a string using indexing with square brackets `[]`. Indexing starts at 0.
- Retrieve parts of strings using slicing with the syntax `[start : stop : step]`.
- During slicing, if `start` is omitted, it defaults to 0. If `stop` is omitted, it defaults to the length of the string. If `step` is omitted, it defaults to 1.
- Negative values can be used for start, stop, and step. A negative index counts from the end of the string.
- Slicing will not throw an error for indexes out of range; it will simply return an empty string or the available characters.
- Use backslashes `\` to insert special characters into strings, like `\n` for a newline or `\\` for a backslash.
- Python has many built-in string methods for common tasks like `.upper()`, `.lower()`, `.strip()`, `.split()`, and `.replace()`.
- Use `.format()` or `f`-strings (formatted string literals) to embed expressions inside string literals.
- Prefix strings with `r` to treat backslashes as literal characters and not escape characters.
- Use triple quotes `"""` or `'''` for strings that span multiple lines.
- Python strings are Unicode by default, which means they can include characters from most of the world's writing systems.
- To reverse a string use a negative step to reverse a string, like `[::-1]`

*****END OF CHAPTER*****

²² <https://peps.python.org/pep-0498/#format-specifiers>

6 BRANCHING AND LOOPING

In this chapter we will learn about types of statements supported by Python statement set. Table 6.1 summarizes the types of statements supported in Python and you have already seen top two i.e. assignment and print in Chapter 3. Table 6.1 also highlights the respective Section of this book, where we have explained the details about individual statement types. In your programs, these units of code can perform actions, repeat tasks, to break out or continue with loop and so on.

This chapter deals with the entries listed in Table 6.1 and explains their working process based on data type need to work with or depends on the situation. Statements lower in Table 6.1 i.e., **def**, **class**, **try/catch**, and **import** that have to do with larger program units—functions, classes, modules, and exceptions—lead to larger programming ideas, so we have not included here. But, **def** statements are discussed separately in Chapter 8 and rest of them included in the Part 2 of this book. More focused statements (like **del**, which deletes various components) are covered in Chapter 7.

Table 6.1 Python Statement Types

Statement	Role	Example
assignment	Creating Reference	Discussed in Chapter 3, 4
print calls	Printing Object	print(var)
if/elif/else	Selection	Discussed in Section 6.1
for/else	Iteration	Discussed in Section 6.3 and 6.5
while/else	Iteration	Discussed in Section 6.3 and 6.5
pass	Empty placeholder	Discussed in Section 6.6
break	Exit from loop	Discussed in Section 6.4
continue	Loop continue	Discussed in Section 6.4
match	Branching with multiple case	Discussed in Section 6.2
del	To remove individual items from collections	Discussed in Chapter 7
def	Defining a function	Discussed in Chapter 8
class	To define a user-defined type	Not included in this book
import	To import an external module	Not included in this book
try/catch	To handle exception	Not included in this book

Everything you have seen so far has consisted of sequential execution, in which statements are always performed one after the next, in exactly the order specified. But the world is often more complicated

than that. Frequently, a program needs to skip over some statements, execute a series of statements repetitively, or choose between alternate sets of statements to execute. That is where control structures come in. A control structure directs the order of execution of the statements in a program.

6.1 Selection Statement

In the real world, we commonly evaluate information around us and then choose one course of action or another based on what we observe. If the weather is nice, then I'll mow the lawn. (It's implied that if the weather isn't nice, then I won't mow the lawn.)

In a Python program, we have used **if** statement to perform decision-making. It allows for conditional execution of a statement or group of statements based on the value of an expression. This section highlighted the working of **if** statement in following ways:

- First, you'll get a quick overview of the if statement in its simplest form.
- Next, using the if statement as a model, you'll see why control structures require some mechanism for grouping statements together into compound statements or blocks. You'll learn how this is done in Python.
- Lastly, you'll tie it all together and learn how to write complex decision-making code.

6.1.1 The if Statement

The simplest form of **if** statement is,

```
>>> if expr:
...     statement(s)
```

The above form can be broken down as follows:

- `expr` is an expression evaluated in a Boolean context, either as `True` or `False`, as we have discussed in Section 4.2 and 4.3 of Chapter 4.
- `Statement(s)` refers one or many valid Python statement, starts after indentation.
- If `expr` is `True` (evaluates to a value that is “truthy”), then `statement(s)` executed. If `expr` is `False`, then `statement(s)` is skipped over and not executed.
- The colon (`:`) following `expr` is required, following this body of the **if** statement will start. Some programming languages require `expr` to be enclosed in parentheses, but Python does not.

Next, we have listed some examples to understand the working process of **if** statement.

```
>>> if 5 > 4:
...     print('Yes 5 > 4')
...
Yes 5 > 4
>>> if 5 < 4:
...     print('Yes 5 < 4')
...

```

The condition of the **if** statement `5 > 4`, evaluates to `True` and the `print` statement inside if gets executed. The outcome of the above execution is displaying a message `'Yes 5 > 4'`. But the second if condition, `5 < 4`, evaluates to `False` and that's why we did not received any output. In the next example, two conditions are combined with logical operators in `expr` of the **if** statement.

```
>>> if True or True:
...     print('Truthy Condition')
...
Truthy Condition
>>> if False or False:
...     print('Falsy Condition')
...
```

For this example, we have used Boolean values `True` and `False` with logical **OR** operator. The first if expression evaluates to `True` and the print statement 'Truthy Condition' gets executed. But the logical **OR** returns `False`, when both the operands evaluate to `False` and that is the reason, body of second if statement is not executed means skipped or stepped over by the interpreter. Now let's see a bit complex expr and mix comparison and logical operator to form an expression within an if statement:

```
>>> if 4 == 4 and 5 > 3:
...     print('Truthy Condition')
...
Truthy Condition
```

In the above example, two conditions `4 == 4` and `5 > 3` are combined using logical **and** operator. Both the conditions are evaluating to `True` and the body of if, gets executed and displayed the message. There is no such limit, how many such conditions we can combine using logical operators. Let's solve some problem to understand the working of if statement. All these are executed through Python script file (.py). See Chapter 2, how to create and execute Python script files using IDLE.

Problem 6.1: Write a program to find the largest of two integers using if statement only and numbers must be taken as input from user.

```
1 #Step-1: Taking Input
2 num1 = int(input('Enter number:'))
3 num2 = int(input('Enter number:'))
4 #Step-2: Display both numbers
5 print(f'Numbers are : num1 = {num1}, num2 = {num2}')
6
7 #Step-3: Condition for num1 is largest or step-4
8 if num1 > num2:
9     print(f'{num1} is largest')
10 #Step-4: Condition for num2 is largest
11 if num2 > num1:
12     print(f'{num2} is largest')
```

```
Enter number:23
Enter number:12
Numbers are : num1 = 23, num2 = 12
23 is largest
```

Two numbers 23 and 12 is taken as input and the expr in first if, **num1 > num2** line 8, evaluates to

True and displayed '**23 is largest**'. The expr, at line 8, written to test num1 (23) is larger than num2 (12), and the expr, evaluates to True. The second expr **num2 > num1**, at line 11, written for num2 is larger than num1, becomes False for this pair of input. Till this discussion we have found that body of if statement only executed if the expression of if evaluates to True. All the above examples have two if statements, one written for Truthy condition and other one for Falsy condition. One can change the pair of input, as shown below, and test the expr at line 11. So that **num2** become largest.

```
Enter number:5
Enter number:6
Numbers are : num1 = 5, num2 = 6
6 is largest
```

In the above program multiple if statements are written for same purpose (to check the largest) and the if statement, **num1 > num2** at line 8, evaluates to True. From this point one can understand this, out of these two if statements, one will be True and other will be False and need to skip. But Python interpreter, evaluates both, because we have not written the code to do this. It can be achieved by introducing **else** (works for False condition) clause with **if** statement. Before introduction to the else clause let us understand the importance of indentation for if statement. Consider the coding sample illustrated next:

```
1 num1 = int(input('Enter number:'))
2 num2 = int(input('Enter number:'))
3 print(f'Numbers are : num1 = {num1}, num2 = {num2}')
4
5 if num1 == num2:
6     print('Start of If')
7     print('Both numbers are same')
8     print('End of If')
9 print('After If')
```

In the above code, if the condition **num1 == num2**, is True, then all the indented statements from line 6 to 8 get executed and then non-indented statement at line 9 will be executed. If the condition become False, then program control will skip all the indented lines from 6 to 8 and directly executed line 9. See the program output:

```
Enter number:4
Enter number:4
Numbers are : num1 = 4, num2 = 4
Start of If
Both numbers are same
End of If
```

For the same input, the condition at line 5, evaluates to True and all the indented lines inside if, gets executed and then non-indented lines. Now execute the script again and give two different numbers as input, as shown below:

```
Enter number:4
Enter number:5
```

Numbers are: num1 = 4, num2 = 5

After If

For different numbers as input and the condition at line 5, evaluates to False and program control skipped all indented line inside if, and directly executes the non-intended lines after if. In both the scenarios we have understood that, either way, execution proceeds with line 9 afterward. Did you notice that there is no way to know the end of if block? Rather, put a print statement at the last intended line inside if.

Note: In the Python documentation, a group of statements defined by indentation is often referred to as a suite. This book uses the terms block instead of suite.

6.1.2 The else Clause

We have learnt, how to use an if statement to conditionally execute a single statement or a block of statements. It's time to find out what **else** can do. Sometimes, you want to evaluate a condition and take one path of execution, if the condition with **if** statement it is true, and specify an alternative path if it is not. This is accomplished with an **else** clause after an **if**. Let see the general syntax:

```
if expr:
    statement(s)
else:
    statement(s)
```

If the **expr** is true, the blocks of statements inside **if**, will be executed, and the statements inside **else** will be skipped. If **expr** is false, then the first block is skipped, and the second one is executed. Either way, execution then resumes after the second block, means **else**. Both blocks are defined by indentation, as described above. Now let's re-write the above program (Program 6.1) with if and else.

Problem 6.2: Write a program to find the largest of two integers using if-else statement and numbers must be taken as input from user.

```
1 #Step-1: Taking Input
2 num1 = int(input('Enter number:'))
3 num2 = int(input('Enter number:'))
4 #Step-2: Display both numbers
5 print(f'Numbers are : num1 = {num1}, num2 = {num2}')
6
7 #Step-3: A True Condition for num1 is largest
8 if num1 > num2:
9     print('Inside If')
10    print(f'{num1} is largest')
11 #Step-4: Automatically executes, if step-3 Condition become False
12 else:
13    print('Inside Else')
14 print(f'{num2} is largest')
```

Enter number:8

Enter number:9

Numbers are : num1 = 8, num2 = 9

```
Inside Else
9 is largest
```

See the solution carefully and the **else** clause at line 12, does not have any **expr** like **if**. It means else not designed to test any condition. After running of the above program with 8 (for num1) and 9 (for num2) as input, the if condition, **num1 > num2**, at line 8, evaluates to False, as we have entered largest value for num2. So, it is obvious that the expression of if False and else block will be executed, thereafter statements written after else will be executed. Earlier we have solved the same problem using if and now we have re-written with else clause, which improves the code efficiency. Let us answer one question, can we use an else clause without if? Answer is very straight a big NO; interpreter will raise a **SyntaxError**.

Now let's add another condition to the above program. Previously we have checked the largest of two numbers using if and else clause. Now suppose we have given two of the same numbers as input and let's test the program is ready to tackle this. Now re-execute the above code and consider 5 as input for both variables and analyze the code.

```
Enter number:5
Enter number:5
Numbers are : num1 = 5, num2 = 5
Inside Else
5 is largest
```

The output is '5 is largest', but which one, num1 or num2. This happened because we did not write the required logic to test the equality. Now question is that how to add the third condition in the program. We can do this by replacing the else clause with an if and add third if statement to code, see the modified code:

```
num1 = int(input('Enter number:'))
num2 = int(input('Enter number:'))
print(f'Numbers are : num1 = {num1}, num2 = {num2}')

#Condition 1: A True Condition for num1 is largest
if num1 > num2:
    print(f'{num1} is largest')
#Condition 2: A True condition for num2 is largest
if num2 > num1:
    print(f'{num2} is largest')
#Condition 3: A true condition for both numbers are equal
if num1 == num2:
    print('Both numbers are equal')
```

In the above code we have written three individual conditions based on their different outcome of the problem. Means, if we take input **num1** as the largest value, condition1 will become True and for num2 as largest and equality condition2 and 3 will be True. There is nothing wrong about the output of the code, but the main drawback of this approach is all three conditions will be checked by the interpreter. It is wasting of execution time and to improve we can use an else clause after any of the if statement, as follows:

```
1 num1 = int(input('Enter number:'))
```

```

2 num2 = int(input('Enter number:'))
3 print(f'Numbers are : num1 = {num1}, num2 = {num2}')
```

4

```

5 #Condition 1: A True Condition for num1 is largest
6 if num1 > num2:
7     print(f'{num1} is largest')
8 #If, condition 1 False, Else clause will execute automatically
9 else:
10    print(f'{num2} is largest')
11 #Condition 2: A true condition for both numbers are equal
12 if num1 == num2:
13    print('Both numbers are equal')
```

The second if statement, at line 8 is replaced by an else clause, it means if the first if statement is True, interpreter will skip the else clause. So, we have saved execution time, but, what if, the first if statement is False, then else at line 9 and if statement at line 12 also evaluated (for same number). Have a look at the output screen:

```

Enter number:5
Enter number:5
Numbers are : num1 = 5, num2 = 5
5 is largest
Both numbers are equal
```

We get two outputs instead of just 'Both numbers are equal'. This happened because we have placed the else clause after the first if. So, it makes the if statement at line 12 is an individual condition and that's why we get two outputs. To stop this one thing we can do, place the else clause after the second if, as follows:

```

num1 = int(input('Enter number:'))
num2 = int(input('Enter number:'))
print(f'Numbers are : num1 = {num1}, num2 = {num2}')
```

#Condition 1: A True Condition for num1 is largest

```

if num1 > num2:
    print(f'{num1} is largest')
```

#Condition 2: A true condition for num1 is largest

```

if num1 < num2:
    print(f'{num2} is largest')
```

#If, condition 2 False, Else clause will execute automatically

```

else:
    print('Both numbers are equal')
```

Now we haven't put any condition for equality, instead the else clause will take care the equality. The modified code still can produce two outputs, num1 is taken as largest value as input. It means if the first **if**, evaluates to True still the second **if** will be considered for testing and else clause gets executed. Means, to get only one output based on multiple conditions, we need to write the logic with multiple ifs, if program execution is not the concern. But in case of more than two conditions, using off one else clause, will not produce the expected output. To get this along with the reduction of execution time, we need to re-write the logic using **if-elif-else** (if-else ladder) statements.

6.1.3 The if-elif-else Statements

For branching execution based on several alternatives we can use one or more **elif** (short for else if) clauses. Python evaluates each expression in turn and executes the block corresponding to the first that is True. If none of the expression is True, and an else clause is specified, then its body is executed. Consider the required syntax for this:

```
>>> if expr:
...     statement(s)
... elif expr:
...     statement(s)
... elif expr:
...     statement(s)
... .. .
... .. .
... else:
...     statement(s)
```

We can place any number of **elif** (else if) clause, after if statement with an optional else clause at the end. The actual benefit is we can interlink many conditions together, and on execution of any expr as True, interpreter will skip the remaining **elif** and **else** (if provided) clause. At most, one of the code blocks specified will be executed. If no else clause is included, then you might see no output, if all the conditions are False, then none of the blocks will be executed.

Problem 6.3: Write a program to find the largest of two integers using if-elif-else statement and write the required conditions to test the equality of two input numbers. Execute your code 3 times with following pair of inputs (4, 5), (5, 4) and (5, 5) and the code shouldn't produce more than one output.

```
1 num1 = int(input('Enter number:'))
2 num2 = int(input('Enter number:'))
3 print(f'Numbers are : num1 = {num1}, num2 = {num2}')
4
5 #Condition 1: A True Condition for num1 is largest
6 if num1 > num2:
7     print('num1 is largest')
8 #Condition 2: A true condition for num2 is largest
9 elif num2 > num1:
10    print(f'num2 is largest')
11 #For equality
12 else:
13    print('Both numbers are equal')
```

Execution1	Execution2	Execution3
Enter number:4	Enter number:5	Enter number:5
Enter number:5	Enter number:4	Enter number:5
Numbers are : num1 = 4, num2 = 5	Numbers are : num1 = 5, num2 = 4	Numbers are : num1 = 5, num2 = 5
num2 is largest	num1 is largest	Both numbers are equal

The above code is executed 3 times with different pair of inputs, as follows:

- num1 = 4, num2 = 5, output 'num2 is largest'. For this pair of input, condition written at line 9 with elif clause evaluates to True and the first if expression is evaluating to False. Further, else clause did not consider for execution.
- num1 = 5, b = 4, output is 'num1 is largest'. For this pair of input, the if expression at line 6, evaluates to True and interpreter skipped the elif and else clause.
- num1 = 5, b = 5, output is 'Both numbers are equal'. Here all the conditions written with if and elif clause is evaluates to False and block associated within else get executed.

Program 6.4: Write a Python program that takes a student's numerical grade as input and assigns a corresponding letter grade based on the following criteria: a) 90 or above: 'A'; b) 80 to 89: 'B'; c) 70 to 79: 'C'; d) 60 to 69: 'D'; e) Below 60: 'F'.

```
grade = float(input("Enter student's grade: "))

if grade >= 90:
    letter_grade = 'A'
elif grade >= 80:
    letter_grade = 'B'
elif grade >= 70:
    letter_grade = 'C'
elif grade >= 60:
    letter_grade = 'D'
else:
    letter_grade = 'F'
print(f"The letter grade is: {letter_grade}")
```

In the above code, total 5 conditions are listed and out of these, 1 condition is tested using if (for grade 90 or more), 3 conditions are tested using elif statements (for grade ≥ 60 and < 90) and grade less 60 will be consider by else clause. For any given grade as input, program will produce the desired output and rest of the conditions will skip by the interpreter.

In the above examples, all the if and elif expr tested only single condition and based on this any blocks get executed. Sometimes it become necessary, to build a complex logic, where a decision is taken based on combined conditions or based alternative of multiple conditions. The next Section is highlighted about this fact.

6.1.4 Logical Operators with if-elif Statements

To test multiple conditions that must be met together or to check for alternative conditions, we can combine them in a single if or elif expression using logical (**and**, **or**) operators. This can improve code readability and maintainability by avoiding the need for separate condition checks. The need of logical operators can be summarized as follows:

- **Combines Necessary Conditions:** The **and** operator lets you check if all specified conditions are met before executing a code block. This ensures that only intended actions are triggered when all requirements are fulfilled.
- **Checks for Alternatives:** The **or** operator allows you to handle scenarios where at least one of the combined conditions needs to be true. This simplifies the logic if multiple possibilities lead to the same outcome.

- **Improved Readability:** Logical operators help write concise if-elif expressions, making the code more readable and easier to maintain compared to having separate checks for each condition.
- **Complex Decision-Making:** By combining conditions, you can create more intricate decision flows, making your programs more adaptable to various situations.

You can use logical (and, or) operators to combine multiple conditions or logical not to reverse the decision, within an if or elif statement. The logical operators with if-elif work as follows:

- **and operator:** The combined condition is only True if all individual conditions connected by **and** are True. For e.g. `if age >= 18 and has_id == True:` (checks both age and ID for access)
- **or operator:** The combined condition is True if at least one of the individual conditions connected by **or** is True. For e.g. `if is_weekend == True or is_holiday == True:` (checks for either weekend or holiday)
- **not operator:** Used to reverse the logical state of a single condition. **not** True is False, and **not** False is True.

Program 6.5: Write a Python program that takes a student's numerical grade (floating-point number between 0.0 and 100.0) as input. The program should then assign a corresponding letter grade based on the following criteria:

- A: 90.0 to 100.0 (inclusive)
- B: 80.0 to 89.9 (inclusive)
- C: 70.0 to 79.9 (inclusive)
- D: 60.0 to 69.9 (inclusive)
- F: Below 60.0 (including invalid grades)

```
grade = float(input("Enter your grade (0.0-100.0): "))
```

```
if grade >= 90 and grade <= 100:
    letter_grade = 'A'
elif grade >= 80 and grade < 90:
    letter_grade = 'B'
elif grade >= 70 and grade < 80:
    letter_grade = 'C'
elif grade >= 60 and grade < 70:
    letter_grade = 'D'
else:
    letter_grade = 'F' # Includes invalid grades (< 60)

print(f"Your letter grade is: {letter_grade}")
```

The first if statement checks if the grade is both greater than or equal to 90 (≥ 90) and less than or equal to 100 (≤ 100) using logical **and**. This ensures a valid grade range for an 'A'. Subsequent elif statements follow the same pattern, checking for grade ranges for 'B', 'C', and 'D' using **< and >** to define the upper and lower bounds. If none of the if or elif conditions are met (including invalid grades below 60), the else block assigns 'F' to the letter_grade variable. Finally, the program prints the assigned letter grade.

Problem 6.6: Write a Python program that takes a year as input and determines if it's a leap year. Developed your logic to satisfies the following rules:

- If a year is divisible by 4 but not by 100 (e.g., 2004) or
- A year divisible by 400 (e.g., 2000)

```
year = int(input("Enter a year: "))
```

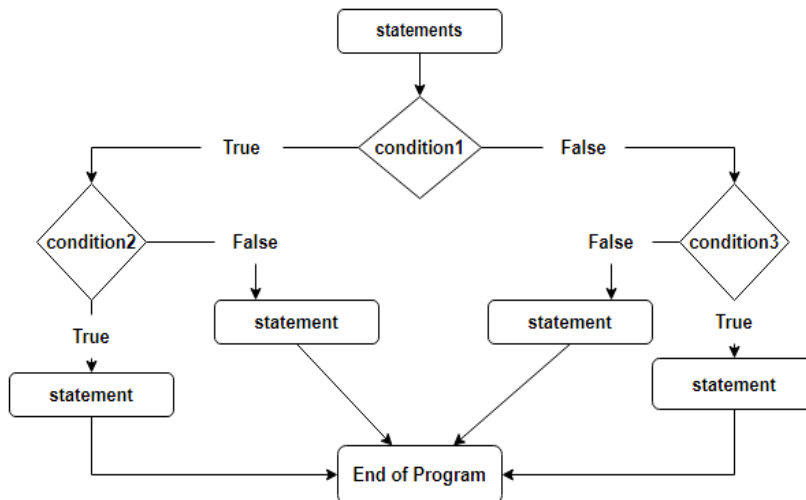
```
if (year % 4 == 0 and year % 100 != 0) or year % 400 == 0:
    print(f"{year} is a leap year!")
else:
    print(f"{year} is not a leap year.")
```

The if statement uses parentheses for proper evaluation order. The first condition (using or) checks if the year is divisible by 4 but not by 100 ($\text{year \% 4} == 0$ and $\text{year \% 100} != 0$). The second condition checks if the year is divisible by 400 ($\text{year \% 400} == 0$). If either condition is true, the year is a leap year. Otherwise, it's not. These examples showcase how the logical OR operator (or) can be effectively combined with if-elif statements to create more complex decision-making logic based on multiple alternative conditions.

6.1.5 Nested if-elif-else Statements

Sometimes we need to build complex logic, where multiple conditions need to be tested. Depend on True outcome program execution will follow different path, and based on False, it goes in other direction. To understand this, let's consider a flow-chart of nested if-else statement.

In the flow chart you can see based on the outcome of condition, the program execution will follow two different paths. If the condition1 become True, it will go for condition2, which will be nested inside the condition1 if or statement else it will go for condition3. In Python programming we can implement the same by nesting of an if, inside another if or inside an elif, or inside an else clause. This section highlights the working of nested if-elif-else statements based on some examples.



A nested if-else statement is an if-else statement that is nested (meaning, inside) another if, elif or else statement. With those statements we evaluate True or False conditions and make our program respond appropriately. See the syntax of nested if-else structure:

```
>>> if expr:
```

```

...     if expr:
...         statement(s)
...     else:
...         statement(s)
... elif expr:
...     if expr:
...         statement(s)
...     ... ..
...     ... ..
... else:
...     if expr:
...         statement(s)
...     else:
...         statement(s)

```

In the above syntax, we have nested if-else inside if and else block. Like this we can nest any number if-else statements or only if inside an elif clause. Let's rewrite the Leap Year Program using nested if-elif-else statements.

Problem 6.7: Write a Python program that takes a year as input and determines if it's a leap year or not. Developed your logic to satisfies the following rules:

- The given year must be divisible by 4 and then
 - If the year is divisible by 100 such as 1700 is not a leap year
 - If the year not divisible by 100 such as 2004 is a leap year
 - If the year divisible by 100 and 400 such as 2000 is a leap year
- If the year is not divisible by 4 such as 2011 is not a leap year

```

1 year = int(input('Enter a Year: '))
2
3 if year % 4 == 0:
4     if year % 100 != 0:
5         print(f'{year} is a leap year')
6     else:
7         if year % 400 == 0:
8             print(f'{year} is a leap year')
9         else:
10            print(f'{year} is not a leap year')
11 else:
12     print(f'{year} is not a leap year')

```

In the above solution, code line number (4 – 10) are nested inside the if block written at line 4. Now, these lines are part of the if block and these lines will be evaluated if the condition `year % 4 == 0`, evaluates to True. This condition will divide the given `year` by 4, if the remainder of the division operations returns zero, program control enters inside the if statement.

Next, interpreter have two situations, if the year for e.g. 1996, is not divisible by 100 and the condition at line 4, evaluates to True, that year will be displayed as leap year. Further, the condition `year % 400 == 0`, nested inside the else statement.

Problem 6.8: Write a Python program to find the largest number out of three integers. Write your logic to satisfy the equality of two and three numbers as well.

```

1 num1 = int(input('Enter a number: '))
2 num2 = int(input('Enter a number: '))
3 num3 = int(input('Enter a number: '))
4 print(f'Values are num1 = {num1}, num2 = {num2}, num3 = {num3}')
5
6 if num1 == num2 == num3:
7     print('Numbers are equal')
8 else:
9     # Find the largest number
10    if num1 > num2 and num1 > num3:
11        print(f'num1 = {num1} is largest')
12    elif num2 > num3:
13        if num2 == num1:
14            print('num1, num2 are same')
15        else:
16            print(f'num2 = {num2} is largest')
17    else:
18        if num3 == num1:
19            print('num1, num3 are same')
20        elif num3 == num2:
21            print('num3, num2 are same')
22        else:
23            print(f'num3 = {num3} is largest')

```

The above code can be broken down into two segments as follows:

- Initial Equality Check (lines 6-7): Correctly states that the first segment tests for equality using `num1 == num2 == num3`. Captures the essence of displaying "Numbers are equal" when all three numbers are the same.
- Handling Number Differences (lines 8-26): Accurately identifies the **else** clause handling cases where numbers are not equal. Clearly outlines the nested conditional structure for determining the largest number.

Breakdown of the logic within the **else** Clause, as follows:

- Checking for **num1** as Largest: Correctly identifies the condition `num1 > num2 and num1 > num3`, at line 10. Appropriately states that **num1** is declared largest when this condition holds.
- Checking for **num2**: Correctly mentions the **elif** clause at line 12, for prioritizing **num2**. Identifies the nested if-else checks for **num2**'s equality with **num1** or its designation as largest.
- Checking for **num3**: Accurately locates the final **else** clause at line 17 for handling cases where neither **num1** nor **num2** is largest. Appropriately describes the nested **if-elif-else** structure within this clause:
 - Nested **if** at line 18 checks for **num3**'s equality with **num1**.
 - Nested **elif** at line 20 checks for **num3**'s equality with **num2**.
 - Final nested **else** at line 22 declares **num3** as largest when previous conditions fail.

6.1.6 One line if statement

It is permissible to write an entire if statement in one line. The syntax is given below:

```
>>> if expr: statement
```

We can put more statements in same line, must be separated by semi-colon (;). Careful statements fall in between colon (:) and last statement must be separated by semi-colon only.

```
>>> if expr: statement_1; statement_2; statement_3
```

If the given expression after **if** is **True** then all the statements followed by colon (:) will be executed otherwise don't execute any of them. All the statements after colon (:) considered as block of if. See the example given below:

```
>>> if 5 > 4: print(1); print(2); print(3)
...
1
2
3
```

Note: While all of this works, and the interpreter allows it, it is generally discouraged on the grounds that it leads to poor readability, particularly for complex if statements.

6.1.7 Ternary Operator

Since Python 2.5, it supports one additional decision-making entity called a conditional expression (because the C language has a similar tool). In python, conditional expressions also known as conditional operator or ternary operator. See the syntax given below:

```
>>> statement1 if expr else statement2
```

Python executes statement1, if the conditional expression (expr) evaluates to **True**, or executes statement2, if **expr**, turns out to be **False**. We can also assign the result of conditional operator to a variable. In ternary operator the **else** part is compulsory, and if we forget to put, interpreter will raise a **SyntaxError**. Let's see an example,

```
>>> age = 18
>>> s = 'minor' if age < 21 else 'adult'
>>> print(s)
minor
```

The above code tested a person is minor or adult based on his/her age. The variable **age** is holding a value 18 and the condition **age < 21**, after if is evaluates to **True**. So, the statement1 i.e. 'minor' is assigned to **s** and the statemen2 as part of else is ignored.

A common use of the conditional expression is to select variable assignment. For example, suppose you want to find the smallest of two numbers. Of course, there is a built-in function, **min()**, which can solve our purpose. But suppose you want to write your own code from scratch, as follows:

```
>>> a, b = 11, 12
>>> min = 0
>>> if a < b:
...     min = a
... else:
...     min = b
...
>>> print(min)
11
```

With comparison to the above code a conditional expression will be shorter and more readable, as follows:

```
>>> min = a if a < b else b
>>> print(min)
11
```

Note: The conditional expression behaves like an expression syntactically. It can be used as part of a longer expression. The conditional expression has lower precedence than virtually all the other operators, so parentheses are needed to group it by itself.

```
>>> a = b = 30
>>> c = 1+a if a < b else b+2
>>> c
32
```

Here $(1 + a)$ and $(b + 2)$ will be evaluated first then followed by the conditional expression **(31 if 30 < 30 else 32)** and 32 is assigned to the variable **c**. If we want to evaluate the conditional expression first, then we need to put the conditional expression **(a if a < b else b)** inside parenthesis, as follows:

```
c = 1 + (a if a < b else b) + 2
>>> c
33
```

The conditional expression **a<b** evaluates to **False** and the value of b i.e. 30 is replaced the conditional expression. Then the expression become **1 + 30 + 2** evaluates to 33 and assigned to the variable **c**.

6.2 match Statements

A match statement takes an expression and compares its value to successive patterns given as one or more case blocks. A general syntax of match statement is given below:

```
match expression:
    case match_exp1:
        pass
    case match_exp2:
        pass
    case _:
        pass
```

At first, the **expression** will be evaluated and compared with every successive case expression written as **match_exp1** to **match_exp2**. On matching the body of respective case get executed. Note that, the last case: is always written as **_** (an underscored) acts as a wildcard and gets executed, if no case matches and none of the branches is executed.

There are many situations where **match** expression become very useful. Like you want to perfume a mathematical operation of any two number given by a user and the operation he wants to perform. See the next programming example,

Problem 6.8: Write a program to take two numbers and an operator as input. Build your logic with

match statement, to execute the desired mathematical operation, based on user choice.

```
a, b = eval(input('Enter two numbers:'))
operator = input('Enter an operator from +, -, *, /:')
match operator:
    case '+':
        print(a + b)
    case '-':
        print(abs(a-b))
    case '*':
        print(a * b)
    case '/':
        print(a / b)
    case _:
        print('Wrong operator selected')
```

```
Enter two numbers:4,5
Enter an operator from +, -, *, /: +
9
Enter two numbers:6,7
Enter an operator from +, -, *, /: 3
Wrong operator selected
```

In the above code sample two scenario has been considered. At first, an addition operation between 4 and 5 is performed, based on the user selected operator. Once the first case expression matched, statement(s) written as part of its body gets executes and prints 9 and all other case expressions are ignored. Secondly, we have entered a wrong operator intentionally, which is not part of the any of the case expression. Due to this the last case expression gets executed and prints 'Wrong operator selected'. We can also combine multiple literals in one case expression together using | ("or") as stated below:

```
a, b = eval(input('Enter two numbers:'))
match a+b:
    case 10|15:
        print('First Case Executed')
    case 20|25:
        print('Second Case Executed')
    case _:
        print('The Special case get Executed')
```

Two points related to match statements are highlighted in the above example. At first, any mathematical expression like 'a+b' can be used as part of match expression. Secondly, using "or" multiple case expression can be combined together, which may be helpful when, we want to execute a group of statement for different types case expression. Case expression look like unpacking assignments can be used to bind variables in a match statement:

```
point = (3,0)
match point:
    #for point = (0, 0)
    case (0, 0):
        print("Origin")
    #for point(0, 3)
    case (0, y):
        print(f"Y={y}")
```

```

#for point(3,0)
case (x, 0):
    print(f"X={x}")
#for point = (3, 3)
case (x, y):
    print(f"X={x}, Y={y}")
case _:
    raise ValueError("Not a point")

```

The first pattern (0,0) has two literals and can be thought of as an extension of the literal pattern shown above. But the next two patterns (0, y) and (x, 0) combine a literal and a variable, and the variable binds a value from the subject (point). The fourth pattern captures two values, which makes it conceptually similar to the unpacking assignment (x, y) = point.

An **if** clause can be added with a pattern, known as a “guard”. If the guard is false, match goes on to try the next case block. Note that value capturing in case patterns happens before the guard is evaluated:

```

match point:
    case point(x, y) if x == y:
        print(f"Y=X at {x}")
    case point(x, y):
        print(f"Not on the diagonal")

```

6.2.1 match vs. if-else Statements

Both **match** and **if-else** statements are used for conditional branching in Python, but they offer different approaches:

- **if-else Statements**, uses Boolean expressions to evaluate conditions and execute a specific code block based on whether the condition is True or False. It can also have chained **elif** statements for handling multiple conditions.
- **match Expressions** (since Python 3.10+), performs pattern matching on an expression's value and execute code blocks based on the matched pattern(s). It offers more flexibility and readability for complex conditional logic.
- For error handling, **if-else** might require additional checks to ensure all conditions are covered. **match** can optionally include a wildcard pattern (**_**) to handle unmatched cases for better error handling.
- For simple condition, **if-else** is sufficient and familiar. But for complex pattern matching or multiple conditions, **match** offers better readability and maintainability.

6.3 Looping Statements

This section highlights about two Python’s looping constructs statements that repeat an action over and over. At first, while loop is discussed to understand the basic mechanism of general loop constructs. The second one is for loop statement, which is used to iterate over a collection of items and executes a block of statement for each item.

While we are studying the loops, we will also cover some other statements such as **break** and **continue** commonly used with loop. Along with this we will also cover some basic built-ins such as

range, **zip** and **map** are commonly used with loop. In computer programming there are two types of iteration, known as, definite and indefinite:

- The definite iterations mean, at the start of loop the number of iterations (number of times designated block will execute) is defined.
- With indefinite iteration it is not known how many times the block of code will execute. Rather the block of code is executed repeatedly as long as the condition is **True**.

6.3.1 The while loop

Python while loop is the most basic construct of looping statement. As long as the condition at top evaluates a true value the block (normally indented) will executes repeatedly. Once the top condition become false the block of indented code will be skipped and interpreter proceed with other available statements. Now let's see the basic while loop statement.

```
>>>while expr:
```

```
    statement(s)
```

The **statement(s)** generally refers block of statements, often referred as body of the loop, should be indented just like the if statement. The control expression **expr** may contain any boolean expression evaluates to **True** or **False**. If the given **expr** evaluates to **True** then the statements as part of block or body of while will executes else loop terminates. Let's consider the following while statement:

```
>>> n = 1
>>> while n <= 5:
...     print(f'value of n is {n}')
...     n = n + 1 #updating loop variable
...
value of n is 1
value of n is 2
value of n is 3
value of n is 4
value of n is 5
```

Here is what's happening in the given code:

- The variable **n** is initialised to 1. The expr **n<=5**, at line 2, evaluates to **True**, and the body of while from line 3 – 4 executes. Inside the body the value of **n** is printed and its value gets incremented by 1, through the statement **n = n + 1**.
- The statement at line 4 is called an updation of the looping variable **n**. Once all the statements from line 3 – 4 is executed, then program execution returns to the top of the loop at line 2, and the condition **n<=5** is evaluated again. This time the condition is tested over the updated value of **n**, and it's still **True** (because value of **n** is 2 which is less than or equal to 5). So, the body of while executes again, and program control moves to top.
- This process will continue till the value of **n** become 6, which turns the expression evaluates to **False** and loop terminates. Immediately program control moves to the statements followed the loop body.

Some important points need to remember while working with while loop:

- If we forget to update the loop control variable (**n** in our case), who's value always tested before entering to the body of the loop, then loop can be executed for infinite amount of time. Consider the following while statement:

```
>>> n = 1
>>> while n <= 5:
...     print(f'value of n is {n}')
```

Here the updation statement **n = n + 1**, is removed from the body of while. So, the expression **n<=5**, always be True and due to this loop body will executes infinite amount of time.

- If we miss the initialization (**n=1**) statement then a NameError will raised, if we have used a variable in the Boolean expression, which is not declared previously.

```
>>> while n <= 5:
...     print(f'value of n is {n}')
```

Traceback (most recent call last):
File "<stdin>", line 1, in <module>
NameError: name 'n' is not defined

- It is also not necessary to update the loop control variable always incremented by 1. It can be incremented or decremented by any value, as follows:

```
>>> n = 5
>>> while n > 0:
...     print(n)
...     n = n-1
```

Here the value of **n** is initialised to 5, then at every iteration its value gets decremented by 1 (**n = n - 1**). See, the next example which prints all the even numbers from 2 to 20, where loop control variable gets updated by 2, after every iteration.

```
>>> n = 2
>>> while n <= 20:
...     print(n)
...     n = n+2
```

Now let's see another example of while loop, with list, rather than a numeric comparison:

```
>>> numbers = [1,2,3,4,5]
>>> while numbers:
...     print(numbers.pop(-1))
```

In the above code, we didn't test any expression for True or False, rather the condition at line 2, will be True, till the list become empty. Once all the items are removed using the **.pop()** function the condition become False and loop terminates. We can also put a string as part of the expression and an empty string will be evaluated to False. Details about truthy and falsy python types are already discussed in Chapter 4. You can use all of them and test with while loop as part of expression.

Problem 6.9: Write a program that asks the user for a positive integer (greater than 0). Keep prompting the user until they enter a valid positive integer.

```

status = True
while status:
    number = int(input("Enter a positive integer: "))
    if number > 0:
        status = False
    else:
        print("Invalid input. Please enter a positive integer.")
print("You entered:", number)

```

```

Enter a positive integer: -9
Invalid input. Please enter a positive integer.
Enter a positive integer: 5
You entered: 5

```

Problem 6.10: Develop a program that plays a guessing game with the user. The program thinks of a random number between 1 and 100 (inclusive). The user has to guess the number within a limited number of attempts (e.g., 7 tries). After each guess, the program provides feedback like "Too high," "Too low," or "Correct!"

```

import random

secret_number = random.randint(1, 100)
guesses_left = 7

while guesses_left > 0:
    guess = int(input("Guess a number between 1 and 100: "))
    guesses_left -= 1

    if guess == secret_number:
        print("Congratulations! You guessed the number!")
        break # Exit the loop if the guess is correct
    elif guess < secret_number:
        print("Too low. Try again.")
    else:
        print("Too high. Try again.")

if guesses_left == 0:
    print("Sorry, you ran out of guesses. The number was", secret_number)

```

```

Guess a number between 1 and 100: 70
Too high. Try again.
Guess a number between 1 and 100: 30
Too low. Try again.
Guess a number between 1 and 100: 45
Too high. Try again.
Guess a number between 1 and 100: 35
Too low. Try again.
Guess a number between 1 and 100: 38
Too low. Try again.
Guess a number between 1 and 100: 40
Too low. Try again.
Guess a number between 1 and 100: 42

```

Congratulations! You guessed the number!
 Sorry, you ran out of guesses. The number was 42

6.3.2 The for loop

A for loop is a generic iterator type in Python, used to step through the items in any ordered sequence. We can use the for loop to iterate over a string, list, dictionary, set and other built-in iterables. Let us see the general syntax of for loop:

```
>>> for var in iterable_object:
...     statement(s)
```

In the given syntax **iterable_object** is a collection of objects of same type or different type such as a list or set. The **statement(s)** inside the loop body is defined by the indentation just like if-else or while statement. The loop body will execute once for each item **var** of the **iterable_object**.

Here we can say that, every item of an **iterable_object** will be assigned to **var** at the start of the iteration. Later in the body of loop we can use the value of var, if it is necessary. Let's see an example:

```
>>> numbers = [1,2,3,4,5]
>>> for number in numbers:
...     number += 1
...     print(number)
```

Here is what's happening in the given code:

- A list **numbers** is created at line 1 with 5 items. The **for** loop header at line 2 picks the first item 1 from the list **numbers** and assigned it to **number**.
- Thereafter the body of loop from line 3 – 4 will be executed and the value of number get incremented by 1.
- After that, value of number get printed and program control backs to the loop header at line 2, to iterate over the second item of the list. This process will continue till the length of the numbers. Once all the items are iterated then program control will jump outside of the loop. This program prints the output as 2, 3, 4, 5, 6 and Outside of for loop.

In Python iterable means an object can be used in iteration. If an object is iterable, it can be passed to the built-in Python function **iter()**, which returns something called an iterator. Consider the statement given below:

```
>>> iter(numbers)
<list_iterator object at 0x000002636D2665F0>
```

In the previous example we have used a list as part of iterable object, and python supports many other iterables, such as set, tuple, dictionary, and strings etc. Some of them are provide sequential access to its items and some of them are non-sequential.

Accessing a list, set, string or a tuple are similar, means at every iteration, we are getting the items directly and for string we will get individual characters. But for dictionary, we are getting the keys only, because in dictionary items are always stored in **key:values** pair. In dictionary, to access any value we need the respective key. But for the other iterators such as string, list, tuple and set, we are directly

getting the item. Use of any non-iterable object within for loop will raise a `TypeError`. A sample code to accessing dictionary items are shown below:

```
>>> my_dict = {"name": "Alice", "age": 30, "city": "New York"}
>>> for key in my_dict:
...     print(key, ":", my_dict[key])
...
name : Alice
age : 30
city : New York
```

This for loop that iterates through the keys in the `my_dict` dictionary. The `key` variable defines a variable that can be used within the loop. This variable will store the key of each value of the dictionary item for each iteration. Further, inside the for loop, we are accessing the value associated with the key, `my_dict[key]` by passing the required key inside the square bracket (`[]`). This is not the only one way to accessing a dictionary through for loop, the detail content is discussed in Chapter 7.

When to prefer for loop over while?

Whatever the examples we have seen so far in this chapter, it represents counter-style type for loop. In comparison to while, for loop is quicker than while, so it should be our first-choice tool whenever we need to iterate through a sequence or iterable objects.

Still there are some situations when we need to iterate over second or third items in a list or to update the list. Sometimes we will see it also necessary to traverse more than one iterable object at the same time. What if we need to access all the even or odd indexed item, where we need the index. To fulfil all such requirement Python provides many built-ins for specialized iteration with for loop. These are as follows:

- **range(start, stop, step):** Generates a sequence of numbers within a specified range. Useful for creating the iteration variable in for loops.
- **enumerate(iterable, start=0):** Allows you to loop over both the items and their indices in an iterable object, discussed in Chapter 7.
- **zip(iterable1, iterable2, ...):** Combines multiple iterables element-wise, allowing you to loop through them together.
- **sorted(iterable, key=None, reverse=False):** Returns a sorted version of an iterable. You can customize the sorting behavior with the `key` and `reverse` arguments, discussed in Chapter 7.
- **filter(function, iterable):** used to create a new iterator containing elements from an existing `iterable`, for which a certain condition is true. It applies a filter based on a user-defined `function`.
- **map(function, iterable):** Applies a function to each element of an iterable and returns an iterator of the results.

6.3.3 The range() Function

The `range()` function in Python3 is just the rename of `xrange()` function of Python2. Originally, both `range()` and `xrange()` produced numbers that could be used to iterated over with for-loops. But the

former generated a list of those numbers all at once, while the latter produced numbers lazily, meaning numbers were returned one at a time as they were needed.

Now let's talk about how the **range()** function works and its totally depends how many values we have passed to it. We can tell the **range()** function, about the starting and ending value and also differences between one number and the next. There are three ways we can call the **range()** function:

- **range(stop)** takes one argument and stop informs about when to stop the iteration
- **range(start, stop)** takes two arguments and start informs about the starting value
- **range(start, stop, step)** takes three arguments, step control the differences between two values generated by **range()**.

The range(stop)

It will give range of values starts from zero (0), includes every number up to **stop**, but not including. Values generated by range object can be used as index to iterate over any object, such as string, list, set etc. Consider the statements given below:

```
>>> for i in range(5):
...     print(i, end = ' ')
...
0 1 2 3 4
```

In the output we have all the whole numbers starting from zero (0) up to but not including the **stop** (5).

The range(start, stop)

With two arguments, we are controlling the numbers range from starts to stops. It means we don't have to **start** at 0 (default start value) all the time. We can use **range()** to generate a series of numbers from predefined **start** to **stop** using **range(start, stop)**. Let's find out how to generate a range starting at 1.

```
>>> for i in range(1,5):
...     print(i, end = ' ')
...
1 2 3 4
```

In the output, we have all the whole numbers from 1 (the number is provided as the start) up to but not including 5 (the number you provided as the stop).

The range(start, stop, step)

With three arguments, we can choose not only where the series of numbers will start and stop, but also how big the difference will be between one number and the next. If you don't provide a **step**, then **range()** will automatically consider 1 as step value between two numbers.

```
>>> for i in range(1,6,2):
...     print(i, end = ' ')
...
1 3 5
```

This **range()** function produces the numbers from 1 up to 6 (not including) and difference is 2, controlled by the value of **stop**.

Note: The value of stop can be a positive or negative whole number, but not zero (0). If you try, Python interpreter raised a ValueError.

Increment with range()

To generate a sequence of numbers incrementally using the **range()** function in Python, you need to provide a positive step value as the third argument. By default, the step value is 1, which creates a sequence of numbers increasing by 1. Specifying a different step value allows you to generate sequences with larger or smaller increments. To get an idea of what this means in practice, type in the following code:

```
>>> for i in range(3,100, 25):
...     print(i, end = ' ')
...
3 28 53 78
```

Here we got a range of numbers that were each greater than the preceding number by 25, the step we have provided. Now that you've seen how you can step forwards through a range, it's time to see how you can step backwards.

Decrement with range()

If your step is positive, then you move through a series of increasing numbers and are incrementing. If your step is negative, then you move through a series of decreasing numbers and are decrementing. This allows you to go through the numbers backwards.

```
>>>for i in range(10,-5,-3):
    print(i, end=' ')
10 7 4 1 -2
```

In the above example, the step value is -3. That means that you'll be decrementing by 3, the new value than its preceding value. Most importantly the start value must be greater than stop value. The most Pythonic way to create a range that decrements is to use **range(start, stop, step)**. But Python does have a built-in reversed function, if you wrap **range()** inside **reversed()**, then you can print the integers in reverse order.

```
>>>for i in reversed(range(5)):
    print(I, end=' ')
4 3 2 1 0
```

range() makes it possible to iterate over a decrementing sequence of numbers, whereas **reversed()** is generally used to loop over a sequence in reverse order.

Problem 6.11: Write a program to find the sum **n** natural number start from 1 upto a given number (inclusive) **n**, as input.

```

n = int(input("Enter a positive integer: "))

if n <= 0:
    print("Invalid input. Please enter a positive integer.")
else:
    # Initialize sum to 0
    sum = 0

    # Loop from 1 to n (inclusive)
    for i in range(1, n + 1):
        print(f'{sum}+{i}=',end='')
        sum += i
        print(f'{sum}')

    print(f"The sum of natural numbers from 1 to {n} is: {sum}")

```

```

Enter a positive integer: 10
0+1=1
1+2=3
3+3=6
6+4=10
10+5=15
15+6=21
21+7=28
28+8=36
36+9=45
45+10=55
The sum of natural numbers from 1 to 10 is: 55

```

Problem 6.12: Write a program to print the multiplication table of a given number by user.

```

num = int(input("Enter a number: "))

# Loop from 1 to 10 (inclusive) to print the multiplication table
for i in range(1, 11):
    product = num * i
    print(f"{num} x {i} = {product}")

```

```

Enter a number: 5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50

```

Problem 6.13: Write a program to take input 10 integers as list and build your own logic to find the minimum and maximum number of this list.

```

1 numbers = []

```

```

2
3 # Get 10 integers as input
4 for i in range(10):
5     number = int(input("Enter an integer: "))
6     numbers.append(number)
7
8 # Initialize min and max to the first element of the list
9 min_num = numbers[0]
10 max_num = numbers[0]

11 # Iterate through the list starting from the second element
12 for number in numbers[1:]:
13     if number < min_num:
14         min_num = number
15     if number > max_num:
16         max_num = number
17
18 # Print the minimum and maximum numbers
19 print("Minimum number:", min_num)
20 print("Maximum number:", max_num)

```

This program first creates an empty list **number**, uses a loop to get 10 integers as input from the user and appends them to the list (line 4-6). Next, it initializes two variables, **min_num** and **max_num**, to the first element of the list. This is because we can't be sure if the first element is the minimum or maximum yet. The program then iterates through the list starting from the second element (since we already used the first element to initialize **min_num** and **max_num**). Inside the loop (line 12-16), it compares each element with **min_num** and **max_num**. If the element is smaller than **min_num**, it updates **min_num** with the new value. Similarly, if the element is greater than **max_num**, it updates **max_num**. Finally, after iterating through the entire list, the program prints the minimum and maximum numbers.

6.3.4 Nesting of loop

Now let's look at a for loop that's a bit more sophisticated than those we've seen so far. The next example illustrates statement nesting a loop inside another loop. Given two lists of names with some names are common in the lists. We need to create a third list containing the names present in both the lists.

```

1 list1 = ['Ram', 'Rahul', 'Jyoti', 'Shima']
2 list2 = ['Rohit', 'Ram', 'Rahul', 'Shruti', 'Jyoti']
3
4 list3 = []
5 for name1 in list1:
6     for name2 in list2:
7         if name1 == name2:
8             list3.append(name1)
9
10 print(list3)

```

The above coding example can be summarized as follows:

- The outer loop at line 5 picked a name from **list1** and compared with each name of **list2** using the condition written at line 7.
- Each names of **list2** is assigned to **name2** variable using the inner **for** loop written at line 6. Here we can see that, the inner **for** loop is indented to the outer **for** loop, means it is part of the body of outer **for** loop.
- Once the given condition **name1==name2** at line 7, evaluates to True (means **name1** of **list1** is also present in **list2**), **name1** is appended to the **list3** using **.append()** list method.
- In the above code for one iteration of outer loop, inner loop get executes 5 times which is equal to the length of **list2**. Means at one single iteration of outer loop all the names of **list2** is compared with a name of **list1**. At last, all the names common in both list are printed at line 10.

6.4 break and continue Statements

The break and continue statements are used within looping statements (such as for and while loops) to control the flow of the loop and alter its behavior. These statements serve distinct purposes in looping environment.

6.4.1 The break Statement:

Purpose: The break statement is used to terminate the execution of a loop prematurely when a certain condition is met. It allows you to exit the loop before it naturally completes all its iterations.

Usage: When a break statement is encountered within a loop, the program immediately exits the loop, and control flows to the statement immediately following the loop.

```
for i in range(1, 11):
    if i == 5:
        break # Exit the loop when i equals 5
    print(i)
```

This code will print numbers from 1 to 4 and then exit the loop when **i** is equal to 5. In this code, **range()** is used to generate sequence of numbers from 1 to 10 and break statement gets executed, once the condition with if becomes True.

Use Cases: break is commonly used when you need to exit a loop prematurely based on a specific condition, such as finding a target element in a list or stopping the loop when an error condition is met.

6.4.2 The continue Statement

Purpose: The continue statement is used to skip the current iteration of a loop and move on to the next iteration. It allows you to skip certain iterations of the loop based on a condition without exiting the loop entirely.

Usage: When a continue statement is encountered within a loop, the program skips the remaining code in the current iteration and proceeds to the next iteration.

```
for i in range(1, 12):
```

```

if i % 2 == 0:
    continue # Skip even numbers
print(i)

```

This code will print only the odd numbers from 1 to 12, on meeting the condition, at line 2, `i % 2 == 0`, `continue` statement gets executed and program control will skip all the source line from part of the for loop, will move to the next iteration of the loop.

Use Cases: `continue` is used when you want to skip certain iterations of a loop based on a condition, such as filtering or processing specific elements in an iterable.

In summary, the `break` statement is used to prematurely exit a loop, while the `continue` statement is used to skip the current iteration and move on to the next one. These statements provide control flow mechanisms that allow you to tailor the behavior of loops based on specific conditions, making your code more flexible and efficient.

6.5 The else Clause with loop

The `else` keyword with loops serves a specific purpose: it provides a way to execute a block of code after a loop has completed all its iterations without being interrupted by a `break` statement. The primary purposes of using the `else` clause with loops are as follows:

- **Checking for Completion:** You can use the `else` clause to determine whether a loop completed all its iterations without any premature termination using `break`. This can be useful for situations where you want to act or make a decision based on whether a loop ran to completion or was interrupted.
- **Error Handling:** It allows you to handle situations where a loop was expected to find something or perform some action but didn't. You can use the `else` block to raise an exception, log a message, or take other appropriate error-handling actions.

This behavior can be useful in certain situations to perform additional actions or checks after a loop has finished running. The basic programming syntax to use `else` clause at the end of any looping statement is given below:

```

for item in iterable:
    # Loop body
    if condition:
        # Condition met, break out of the loop
        break
else:
    # This block is executed if the loop completes without a break
    # Additional actions or checks can be performed here

```

The `else` clause must be written at the end of any looping construct. A loop can have any number of if-else statements nested inside it, but the outer `else` clause (written at the end of loop) doesn't have any link with if-else written as part of the loop body as indented statement. The `else` clause at the end of loop will be executed, if the condition for `break` statement didn't meet, means for complete iteration of the loop. The above claim about the working of `else` clause with loop will be better understand from the next programming example.

Problem 6.14: Take a list of 10 numbers and take input a number from user to search it. Write your logic and you are free to use any looping construct, but loop must be end with an `else` clause and print

the message “Number is not found”. Else the loop body must terminate, if the given input is present in the list and displayed “Number is present at index”.

```
numbers = [11,23,45,8,2,9,8,67,3,5]

# Get the number to search for
search_number = int(input("Enter the number to search: "))

for index, number in enumerate(numbers): # Iterate with index
    if number == search_number:
        print(f"{number} is present at index {index}")
        break # Exit the loop if found

else: # Execute only if loop completes without a break (number not found)
    print("Number is not found")
```

In this example a number is searched and for that it is compared with all the numbers present in a list object. If the **search_number** is present, then else block will not be executed. In this example, the else block is executed only if the loop completes all iterations without finding the **search_number**. It provides a clear way to communicate whether the search was successful or not.

Problem 6.15: Build a required logic to test a password is satisfying the following criteria or not. Password must be taken as input from user and maximum three attempts are allowed to form a correct password. Write the required else clause with while loop and print “Password creation successful”, if user forms a password within 3 attempts, else display “Maximum attempted reached”. The condition for password is given below:

- The length of the password minimum 8 and maximum 15.
- Password must contain one lowercase, uppercase, special case, and number from 0-9.

```
import string

# Define password criteria
min_length = 8
max_length = 15
special_chars = "!@#$%^&*()-_+={}[]|\:; '<, >. ? /"

max_attempts = 3
attempts = 0

while attempts < max_attempts:
    password = input(f"Enter your password (attempt {attempts + 1}/{max_attempts} ):")

    # Check password length
    if len(password) < min_length or len(password) > max_length:
        print(f"Password length must be between {min_length} and {max_length} characters.")
        attempts += 1
        continue # Skip to the next iteration

    # Check for lowercase, uppercase, special character, and number
    has_lowercase = any(char.islower() for char in password)
    has_uppercase = any(char.isupper() for char in password)
    has_special = any(char in special_chars for char in password)
```

```

has_number = any(char.isdigit() for char in password)

if not (has_lowercase and has_uppercase and has_special and has_number):
    print("Password must contain at least one lowercase letter, uppercase letter,
special character, and number.")
    attempts += 1
    continue # Skip to the next iteration

# Password meets all criteria
print("Password creation successful!")
break # Exit the loop if successful

else:
    print("Maximum attempts reached. Password creation failed.")

```

```

Enter your password (attempt 1/3 ): 45Ab67
Password length must be between 8 and 15 characters.
Enter your password (attempt 2/3 ): 123ABC$r12487
Password creation successful!
while loop with else clause

```

This code incorporates the following elements of python:

- imports statement to import the string module
- to match password minimum and maximum length variables `max_length` and `min_length` is used
- variable `max_attempts` is used to keep track the maximum number of attempts
- `len(password)` checks the password length falls within the allowed range
- `any()` is used to check the requirement of password characters for lowercase, uppercase, special characters, and numbers.
- if statement inside the while loop checks if all character criteria are met using logical `and` operators. If any criteria are not met, it prints an error message and increments attempts.
- The `continue` statement inside if uses to skip to the next iteration. On successful creation it prints a success message and `breaks` the loop using break.
- `else` clause executes only if the loop finishes without a break (meaning all attempts are exhausted), prints a message indicating failed password creation due to exceeded attempts.

Using the else clause with loops can make your code more expressive and help you handle situations where you want to check if a certain condition was met during the loop's execution. In summary, the else clause in loops in Python is a way to handle situations where you want to execute code after a loop has completed all its iterations without interruption, and it helps improve the readability and maintainability of your code in such scenarios.

6.6 pass statement

The `pass` keyword is a placeholder statement that does nothing when executed. It is often used as a temporary placeholder for code that you plan to write in the future or to create empty code blocks when required by Python's syntax. You can use pass within loops for various reasons, including code structure, readability, or as a placeholder for future code. Here are some common use cases for pass within loops:

- **Stubbing out loop bodies:** You can use `pass` to define an empty loop body when you're in the process of designing your code and haven't implemented the logic yet. This allows you to create the loop structure without writing the actual code at that moment.

```
>>> for item in iterable:
...     #Add your logic
...     pass
```

- **Placeholder for Future Code:** When you're working on a codebase collaboratively or planning to add specific functionality to a loop later, you can use `pass` as a placeholder to indicate that this is an intentional empty section that will be filled in later.

```
for item in iterable:
    if condition:
        # Code to handle the condition
    else:
        pass # Placeholder for future code
```

- **Maintaining Code Structure:** In some cases, you may want to maintain the structure of your code even if you don't currently need to perform any action within a loop. Using `pass` in such cases keeps the loop intact and doesn't break the code's structure.
- **Simplifying Conditional Logic:** `pass` can be used in conditional blocks inside loops to maintain consistent indentation and structure, especially when the true or false branches of the condition don't require any additional code.

Key Takeaway's

- Python uses indentation to define blocks of code. Ensure that the code inside the `if` and `else` blocks is properly indented.
- After the `if` or `else` statement, a colon (`:`) is required to indicate the start of the block.
- The condition after `if` must evaluate to a Boolean value (`True` or `False`). You can use comparison operators (`==`, `!=`, `<`, `>`, `<=`, `>=`) to form these expressions.
- Use `elif` (short for else if) to check multiple conditions in sequence.
- In Python, non-zero numbers, non-empty strings, and non-empty lists are considered `True`. Zero, empty strings/lists, and `None` are considered `False`.
- For simple conditions, you can use a one-line `if-else` statement, known as a ternary operator: `x = 'True' if condition else 'False'`.
- `for` loops are used for iterating over a sequence (like a list, tuple, dictionary, set, or string).
- Use the `range()` function to generate a sequence of numbers which can be iterated over
- Use `break` to exit the loop prematurely if a condition is met.
- Use `continue` to skip the current iteration and continue with the next one.
- An `else` block after a `for` loop will execute after the loop completes normally (i.e., not exited with a `break`).
- `while` loops continue to execute as long as a condition remains true.

Practicing Question

1. Write a program that takes the current temperature as input and gives advice on whether to wear a jacket, a t-shirt, or a sweater based on the temperature range. You can take following ranges of values, or you can go with your own conditional values:
 - If temperature less than 10 print "It's cold! Wear a jacket.
 - If temperature is in between 10 and 20 "It's cool. You should wear a sweater"

- Else, It's warm enough for a t-shirt.
- 2. Simulate a traffic light using case statements. The program should display “Stop”, “Ready”, or “Go” based on the color input (Red, Yellow, Green).
- 3. Build a simple calculator that can perform basic operations (addition, subtraction, multiplication, division) using if-else or case statements to handle different operation requests.
- 4. Write a program that calculates the price of a movie ticket based on age and time of day using nested if-else statements. Include special prices for seniors and children. Use the following list of conditions:
 - If the time is before 5 PM (matinee show), apply a discount.
 - If the time is after 5 PM (evening show), charge the regular price.
 - Within each time block, check the age of the customer:
 - If the customer is a child (age < 12), apply a child discount.
 - If the customer is a senior (age >= 60), apply a senior discount.
 - Everyone else pays the regular price.
- 5. Write a python program to use case statements to suggest activities based on the day of the week input by the user (e.g., Monday - Gym, Tuesday - Swimming).
- 6. Implement a guessing game where the user must guess a secret number. Use a while loop to allow multiple attempts and an else clause to congratulate the user if they guess correctly. Use random module to generate a random number.
- 7. Create a program to calculate the factorial of a given number using a for loop. Use an else clause to print the result after the loop completes.
- 8. Write a program that checks if a number is prime. Use a for loop to check for factors and an else clause to confirm that the number is prime if no factors are found.
- 9. Write a program that calculates the sum of all natural numbers up to a given limit using a while loop. Use an else clause to display the total sum after the loop finishes
- 10. Write a program that prints numbers from 1 to 50, but skips any number divisible by 7 using continue. If the number is greater than 35, use break to exit the loop.
- 11. Take a number as input from user and find all the prime factors of the number.

*****END OF CHAPTER*****

7 EXPLORING PYTHON COLLECTIONS

7.1 List

In short, a list is a collection of arbitrary objects, somewhat like an array in many other programming languages but more flexible. Lists are defined in Python by enclosing a comma-separated sequence of objects in square brackets (`[]`), as shown below:

```
>>>nums = [3,30,6,7,23]
>>>nums
[3, 30, 6, 7, 23]
```

It is also possible to create an empty list by assigning a pair of empty square brackets (`[]`) or using `list()` constructor method, as follows:

```
>>>nums = []
>>>len(nums)
0
>>>nums = list()
>>>len(nums)
0
```

`len()` returns the number of items present in a list and returns 0, on passing an empty list. Out of these two approach to create an empty list, the second approach `list()` is slower because it requires looking up the name of the function, calling it, and then creating the list object in memory. In contrast, `[]` is like a "shortcut" that doesn't require so many intermediate steps to create the list in memory.

7.1.1 Why list?

Before we deep dive into the working principle of list let us first understand why list data structure is necessary and its purpose. In general, you should use lists when you need to and the purpose are listed below:

- **To keep your data in ordered form:** it is possible to store the data in a list in ordered form either as ascending or descending order or arbitrary.

- **Store a sequence of values:** Lists are a great choice when you need to store a sequence of related values. For example: list of names, mobile numbers, etc.
- **To update your records:** lists are mutable by nature means items of the list can be modified any number of times.
- **To access items randomly:** Lists allow quick and easy access to elements based on their index. List index started from 0, means the location of first item and supports negative indexing also.

There are sometimes we need to ignore list object to store some specific type of data:

- Don't use to store immutable data, as list is mutable in nature. Prefer tuple for immutable data and tuples are more memory efficient.
- If list is used to represent database records, then data can be easily modifiable. So, consider using a tuple or a data class.
- As list allows duplicate data, so it is not preferable to store unique and unordered values.
- Run many membership tests where item doesn't matter: In this case, consider using a set. Sets are optimized for this type of operation.
- To work with advanced array and matrix operations consider using NumPy's specialized data structures.
- For manipulate your data as a stack or queue consider using deque or Queue from collection module.

7.1.2 Accessing and Updating list items

You can access individual items from a list using the item's associated index. What's an item's index? Each item in a list has an index that specifies its position in the list. Indices are integer numbers that start at 0 and go up to the number of items in the list minus 1. The following syntax is used to access the list item using index:

```
>>>l1 = [4, 4, 2, 1, 4]
>>>l1[0]
4
```

Instead of accessing just one item, we can index multiple items by passing range of indexes. Some of the examples related to range of indexing is listed in Table 7.1. Consider a list `my_list = [1, 2, 3, 4, 5]`.

Table 7.1 Example of range of indexing with list slicing

Range of index	Explanation
<code>>>>mylist[1:4]</code>	Get elements from index 1 (inclusive) to 4 (exclusive)
<code>>>>my_list[:3]</code>	By leaving out the start value, the range will start at the first item. Get the first three elements
<code>>>>my_list[3:]</code>	Get the last two elements. By leaving out the end value, the range will go on to the end of the list
<code>>>>my_list[::2]</code>	Get elements at even indices
<code>>>>my_list[::-1]</code>	Reverse the list

<code>>>>my_list[::-2]</code>	Get elements at even indices in reverse order
<code>>>>my_list[1:4:2]</code>	Get elements from index 1 to 4 with a step of 2
<code>>>>my_list[2:4] = [6, 7]</code>	Replace elements at index 2 and 3 with 6 and 7
<code>>>>my_list[3:0:-1]</code>	Get elements from index 3 to 1 in reverse order
<code>>>>copy_list = my_list[:]</code>	Create a copy of the original list
<code>>>>my_list[-1:-4:-1]</code>	Get the last 3 elements in reverse order
<code>>>>my_list[1:-1]</code>	Get all elements except the first and last
<code>>>>my_list[:] + [6, 7]</code>	Create a new list by appending elements 6 and 7
<code>>>>my_list[2:7]</code>	Slicing beyond the end of the list is safe
<code>>>>my_list[-3:]</code>	Get the last three elements using a negative start index
<code>>>>my_list[1:4] = []</code>	Clear elements at index 1, 2, and 3
<code>>>>my_list[1:2] = [10]</code>	Change the second value by replacing it with two new values
<code>>>>my_list[1:3] = [3]</code>	Change the second and third value by replacing it with one value

7.1.3 Traversing list items

We can also access all the items of list using a loop till the length of the list. An example is shown below:

```
1 l1 = [4, 4, 2, 1, 4]
2 for i in range(0,len(l1)):
3     print(l1[i],end=' ')
```

`range(0, len(l1))` generates the indexes from 0 to (length -1) of the list. `l1[i]` accessing the items of `l1` using index the value of '`i`'. List items are also accessible using negative index. Negative indexing means start from the end, -1 refers to the last item, -2 refers to the second last item etc.

```
1 l1 = [4, 4, 2, 1, 4]
2 for i in range(-1,-len(l1)-1,-1):
3     print(l1[i],end=' ')
```

```
4 1 2 4 4
```

`range()` in line 2 generates the negative index starting from -1 to `-len(l1)` and `len[i]` accessing items from back of the list. Without index also items are accessible using a loop, as shown below:

```
1 my_list = [1, 2, 3, 4, 5, 6, 7, 8, 9]
2 for x in my_list:
3     print(x, end=' ')
1 2 3 4 5 6 7 8 9
```

But the above style of accessing, doesn't allow us to update the list item. You can loop through the list

items by using a while loop as well. Use of `len()` function to determine the length of the list, then start at 0 and loop your way through the list items by referring to their indexes. Remember to increase the index by 1 after each iteration.

```
1 my_list = [1, 2, 3, 4, 5, 6, 7, 8, 9]
2 i = 0
3 while i < len(my_list):
4     print(my_list[i], end=' ')
5     i = i + 1
```

In many situations, you'll need to know the index of the current item to be able to perform some computations. It's especially true when you're dealing with complex algorithms that operate on indices. In those cases, Python has you covered as well. It offers you the built-in `enumerate()` function, which you can use as in the following example:

```
fruits = ["banana", "Orange", "Kiwi", "cherry"]
for i, fruit in enumerate(fruits):
    print(f"{i} is the index of '{fruit}'")
```

Python provides many other tools that you can use when you're iterating through a list of values. For example, you can use `reversed()` to iterate over the list in reverse order:

```
1 fruits = ["banana", "Orange", "Kiwi", "cherry"]
2 for fruit in reversed(fruits):
3     print(fruit, end=' ')
```

cherry Kiwi Orange banana

The `reversed()` function to traverse your list of fruits in reverse order, which might be a common requirement in your code. Another common need is to traverse the list in sorted order. For this, you can use `sorted()` as in the code below:

```
1 numbers = [2, 9, 5, 1, 6]
2 for number in sorted(numbers):
3     print(number, end=' ')
```

1 2 5 6 9

The `zip()` function, which allows you to iterate over multiple lists in parallel. The `zip()` function returns an iterator of tuples. The elements of each tuple come from the input iterables. In this example, the tuples combine items from three lists integers, letters, and floats.

```

1 integers = [1, 2, 3]
2 letters = ["a", "b", "c"]
3 floats = [4.0, 5.0, 6.0]
4
5 for i, l, f in zip(integers, letters, floats):
6     print(i, l, f)

```

```

1 a 4.0
2 b 5.0
3 c 6.0

```

7.1.4 Concatenating and Repeating

In this Section you'll learn how these two operations are performed with lists and how you can use them in your code. It is an useful feature of Python's list is that it supports the following two operations:

- Concatenation, which uses the plus operator (+)
- Repetition, which uses the multiplication operator (*)

Concatenation consists of joining two things together. In this case, you'd like to concatenate two lists, which you can do using the plus operator (+).

```

>>>[0, 1, 2, 3] + [4, 5, 6] + [7, 8, 9]
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

```

Whenever you use the concatenation operator, you get a new list object as a result. Consider the following example and keep an eye on the identity of **digits** list, that after every concatenation its identity get changed:

```

>>>digits = [0, 1, 2, 3, 4, 5]
>>>id(digits)
3028851826880
>>>digits = digits + [6, 7, 8, 9]
>>>id(digits)
3028851824384

```

You can only concatenate a list with another list. If you try to concatenate a list with something else, then Python will raise a `TypeError` exception:

```
>>>[0, 1, 2, 3, 4, 5] + (6, 7, 8, 9)
Traceback (most recent call last):
  File "<pyshell#5>", line 1, in <module>
    [0, 1, 2, 3, 4, 5] + (6, 7, 8, 9)
TypeError: can only concatenate list (not "tuple") to list
```

The concatenation operator has an augmented variation, which uses the `+=` operator. The augmented concatenation operator works on an existing list. It takes a second list and adds its items, one by one, to the end of the initial list. The operation is a shortcut to something like `digits = digits + [6, 7, 8, 9]`. However, it works a bit differently. Unlike the regular concatenation operator, the augmented variation mutates the target list in place rather than creating a new list:

```
>>>digits = [0, 1, 2, 3, 4, 5]
>>>id(digits)
3028851821952
>>>digits += [6, 7, 8, 9]
>>>id(digits)
3028851821952
```

In this example, the `id()` function returns the same value in both calls, meaning that you have a single list object instead of two.

Repeating the Content of a List

Repetition consists of cloning the content of a given list a specific number of times. You can achieve this with the repetition operator (`*`), as follows:

```
>>>["A", "B", "C"] * 3
['A', 'B', 'C', 'A', 'B', 'C', 'A', 'B', 'C']
```

In the above example, you repeat the content of a list three times and get a new list as a result. The right-hand operand is an integer representing the number of times that you want to repeat the list's content.

7.1.5 Comparing Two List

You can also face the need of comparing two lists and it can be done using the standard comparison operators. All these operators work by making item-by-item comparisons within the two involved lists, as follows:

```
>>>[2, 3] == [2, 3]
True
>>>[5, 6] != [5, 6]
```

```

False
>>>[5, 6, 7] < [7, 5, 6]
True
>>>[5, 6, 7] > [7, 5, 6]
False
>>>[4, 3, 2] <= [4, 3, 2]
True
>>>[4, 3, 2] >= [4, 3, 2]
True

```

Lists with different lengths can also be compared and in this case all the items of the first list (left operand) will be compared with the right-hand side operand, based on the given operator.

```

>>>[5, 6, 7] < [8]
True
>>>[5, 6, 7] == [5]
False

```

In the first expression, you get `True` as a result because 5 is less than 8. That fact is sufficient for Python to solve the evaluation. In the second example, you get `False`. This result makes sense because the lists don't have the same length, so they can't be equal.

7.1.6 Built-in Methods

Python has a set of built-in methods that you can use on lists. To add an item to a list the `append()` and `insert()` method can be called on a list object.

```

>>>nums = [1,2,3]
>>>nums.append(6)
>>>nums
[1, 2, 3, 6]
>>>nums.insert(1,7)
>>>nums
[1, 7, 2, 3, 6]

```

`nums.append(6)`, appends an object at last and `nums.insert(1,7)` requires two parameters. The first parameter is '1', that specifies the index or location where to be inserted the given object '7' as second parameter. To remove an item or all from a list object, we can use `clear()`, `pop()`, and `remove()` methods, as follows:

```
>>>nums.remove(2)
>>>nums
[1, 7, 3, 6]
```

To remove any specific item from a list, we can pass any value to `nums.remove(2)`, if the item is present, else Python will raise a `ValueError`. To handle this issue one can first check the membership of an item, if present then it can be removed.

```
#10 not present, no error is raised
>>>if 10 in nums: nums.remove(10)
>>>nums
[1, 7, 3, 6]

#7 remove
>>>if 7 in nums: nums.remove(7)
>>>nums
[1, 3, 6]
```

To remove any item from a specific location `pop()` method is used, by passing the index to it. If the given index is equal or more than the length of a list, it will raise an `IndexError`. To handle this, you can first check the length of the list and then pass an index value lesser than the length.

```
>>>index = 9
>>>nums.pop(index) if index < len(nums) else print('Invalid index')
Invalid index
>>>index = 2
>>>nums.pop(index) if index < len(nums) else print('Invalid index')
6
>>>nums
[1, 3]
```

Using `.clear()` all the items of a list can be removed and the list object will be empty.

```
>>>nums.clear()
>>>nums
[]
```

List `index()` method searches for a given element from the start of the list and returns the index of the first occurrence. Like `remove()`, `index()` can also raise a `ValueError`, if the given value is not present in the list object.

```
>>>nums = [2,4,5,6,7]
>>>nums.index(3)
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    nums.index(3)
ValueError: 3 is not in list
>>>nums.index(4)
1
```

ValueError can be taken care by using an extra check that we did earlier for `remove()`. Instead of searching all over the complete list, we can also pass a range of indexes using start and end value to the `index()` method, as follows:

```
>>>nums = [1, 2, 3, 4, 1, 1, 1, 4, 5]
>>>nums.index(4,3,6)
3
```

The code uses Python's built-in `.index()` method to find the index of the number 4 within a specific range of a list. the index method is searching for the first occurrence of the number 4 between indices 3 and 5 (because 6 is exclusive). Since 4 is found at index 3, which is within the specified range, the method returns 3 as the output. If the value 4 wasn't found within the specified range (3 to 5), a ValueError exception would be raised.

The `.count()` method in Python is a built-in function used to count the number of times a specific element appears in a list.

```
>>>points = [1, 4, 2, 9, 7, 8, 9, 3, 1]
>>>points.count(9)
2
```

The above code uses the `.count()` method to find the frequency of 9 within points. The count method then searches the entire points list and returns the number of times it finds the value 9.

The `.copy()` method returns a shallow copy of the original list. It creates a new list object that contains the same elements as the original list, but any modifications made to the new list will not affect the original list.

```
>>>l1 = [4, 4, 2, 9, 7, 8]
>>>l2 = l1.copy() #creates a shallow copy
>>>l2[0] = 1
>>>l2
[1, 4, 2, 9, 7, 8]
>>>l1
[4, 4, 2, 9, 7, 8]
```

After the copy operation, first item at index 0, in list `l2` is updated, but original list `l1` didn't get effected. Instead of `.copy()`, if the assignment operator is used, then it creates a deep copy of the original list. Means changes made inside the new list, will modifying the actual list. See the illustration below:

```
>>>l1 = [4, 4, 2, 9, 7, 8]
>>>l2 = l1 #creates a deep copy
>>>l2[0] = 1
>>>l1
[1, 4, 2, 9, 7, 8]
```

```
>>>l2
[1, 4, 2, 9, 7, 8]
```

The `.extend()` method adds the specified list elements (or any iterable) at the end of the current list. In the illustration, items of `l2` added at the end of `l1`, on which `.extend()` is called.

```
>>>l1 = [4, 4, 2]
>>>l2 = [1, 4]
>>>l1.extend(l2)
>>>l1
[4, 4, 2, 1, 4]
```

The `reverse()` method is used to reverse the order of elements in a list. It is an in-place reversal method, meaning it modifies the original list and does not create a new one, making it memory-efficient.

```
>>>l1.reverse()
>>>l1
[4, 1, 2, 4, 4]
```

The `sort` method is used to sort elements in a list. By default, it sorts the list in ascending order. You can optionally customize the sorting behavior using two arguments:

- `reverse`: This is a boolean value (True or False). If set to True, the list will be sorted in descending order.
- `key`: This is a function that takes one element from the list as input and returns a value to be used for comparison during sorting. This allows you to define custom sorting criteria.

```
>>>l1 = [4, 4, 2, 1, 4]
>>>l1.sort() #ascending order
>>>l1
[1, 2, 4, 4, 4]
>>>l1.sort(reverse=True) #descending order
>>>l1
[4, 4, 4, 2, 1]
```

The `key` parameter allows you to specify a function that computes a value for each element in the list, and the list is then sorted based on the computed values rather than the elements themselves. See the illustration:

```
>>>cars = ['Ford', 'BMW', 'Volvo']
>>>cars.sort(key=len)
>>>cars
['BMW', 'Ford', 'Volvo']
```

In the above example list is sorted based on the length of each string. To do that the `len()` function is used as the `key` parameter, which computes the length of each string. The `sort()` function then uses these computed lengths to sort the list. if you want a case-insensitive sort function, use `str.lower()` as a `key` function

```
>>>fruits = ["banana", "Orange", "Kiwi", "cherry"]
```

```
>>>fruits.sort(key = str.lower)
>>>print(fruits)
['banana', 'cherry', 'Kiwi', 'Orange']
```

Program 7.1: Design a user-defined function to sort a list of fruit names based on the last letter of each fruit name.

```
# Define a function to sort by the last character of the string
def sort_by_last_letter(word):
    return word[-1]

words = ["apple", "banana", "cherry", "date", "apricot"]

# Sort the list by the last character of each word
words.sort(key=sort_by_last_letter)
print(words)

['banana', 'apple', 'date', 'apricot', 'cherry']
```

sort_by_last_letter() function takes a single argument word (which is a string in this case) and returns the last character of the word. This function essentially defines the sorting criteria. **words.sort(key=sort_by_last_letter)** line sorts the words list using the sort method. The key argument specifies the **sort_by_last_letter** function. This means the sorting will happen based on the last letter of each word (as determined by the **sort_by_last_letter** function). As you can see, the words are sorted alphabetically based on their last characters (t, a, y, e, and e).

7.1.7 List with Functional Tools

You can also take advantage of some Python functional programming tools, such as **map()** and **filter()**, to traverse a list of values. These functions have an internal loop that iterates over the items of an input iterable and returns a given result.

The **map()** function takes a transformation function and an iterable as arguments. Then it returns an iterator that yields items that result from applying the function to every item in the iterable.

```
>>>numbers = ["2", "9", "5", "1", "6"]
>>>numbers = list(map(int, numbers))
>>>numbers
[2, 9, 5, 1, 6]
```

In the above illustration, **map()** applies **int()** to every item in numbers in a loop. Because **map()** returns an iterator, to the **list()** constructor to consume the iterator and show the result as a list.

If you need to filter out some values from an existing list, then you can use the built-in **filter()** function. This function takes two arguments: a predicate function and an iterable of data. Then it returns an iterator that yields items that meet a given condition, which the predicate function tests for. Here's how **filter()** works in practice:

```
>>>integers = [20, 31, 52, 6, 17, 8, 42, 55]
>>>even_numbers = list(filter(lambda number: number % 2 == 0, integers))
>>>even_numbers
[20, 52, 6, 8, 42]
```

In the above illustration `map()` is used when you want to transform every item in an iterable using a function and collect the results. `filter()` is used when you want to select and retain specific elements from an iterable based on a condition. The built-in `min()` and `max()` takes a list as argument and returns the smallest and largest of any list.

```
>>>l1 = [3,4,5,12,7]
>>>min(l1)
3
>>>max(l1)
12
```

Sometimes in programming it is necessary to check the truthiness of a list and for that Python offers two built-in functions as `any()` and `all()`. Refer the programming example 7.2 to understand the working of these functions with list object.

Program 7.2: Suppose you want to schedule interviews for all the candidates, who meet any or all of the following criteria, (a) Knows Python; (b) Have experience five or more years; and (c) Have a degree

```
applicants = [
    {
        "name": "Devon Smith", "programming_languages": ["c++", "ada"],
        "years_of_experience": 1, "has_degree": False,
        "email_address": "devon@email.com"},
    {
        "name": "Susan Jones", "programming_languages": ["python", "javascript"],
        "years_of_experience": 5, "has_degree": True,
        "email_address": "susan@email.com"},
    {
        "name": "Sam Hughes", "programming_languages": ["java"],
        "years_of_experience": 4, "has_degree": True,
        "email_address": "sam@email.com"},
]
for applicant in applicants:
    meet_criteria = []
    meet_criteria.append("python" in applicant["programming_languages"])
    meet_criteria.append(applicant["years_of_experience"] >= 5)
    meet_criteria.append(applicant["has_degree"])
    if any(meet_criteria):
        print(f'{applicant["name"]}')
```

Susan Jones
Sam Hughes

At first a list of dictionaries namely `applicants` are created, where each candidate information is

preserved as dictionary with information like, **name**, **programming_languages**, **year_of_experience** and **email_address**. Then record of each candidate are checked for shortlisting them based on the required criteria, if any condition matched then program will print the candidate names.

For loop is used to iterate each candidate records and through membership operator **programming_languages**, information is checked for a candidate having knowledge for python or not. Further **years_of_experience** is checked for the candidate having experience more than or equal to 5. At last candidate degree has been recorded and all these information is kept in a list **meet_criteria**.

The **any()** functions returns true because 'Susan Jones' has 'python' as known language and 'Sam Hughes' have a degree. Now if we simply want to shortlist all the candidates matched with all the criteria, then instead of **any()**, **all()** will be used, in the if expression, as shown below:

```
if all(meet_criteria):
    print(f'{applicant["name"]}')
```

Susan Jones

From the output we can check only 'Susan Jones' has matched all the criteria and **any()** returns True, if all the items of a given list are True.

7.1.8 The del statement

del statement is used to remove an item from a list based on the given index, instead of value. This differs from the **pop()** method which returns a value, that we have seen earlier. The **del** statement can also be used to remove slices (a sub-part of the list) from a list or clear the entire list. **del** statement also useful to erasing a list object from memory, and on referencing such object after del operation, will raise a **NameError**, see the illustration given below:

```
>>>a = [-1, 1, 66.25, 333, 333, 1234.5]
>>>del a[0] #delete an item at index 0
>>>a
[1, 66.25, 333, 333, 1234.5]
>>>del a[2:4] #delete items from index 2 to 4 (not including)
>>>a
[1, 66.25, 1234.5]
>>>del a[:] #to delete all the list items
>>>a
[]
>>> del a #delete the complete list from memory
>>> a
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'a' is not defined
```

7.1.9 List Comprehension

List comprehension is a syntactic construct used to create a new list from an existing list. It follows the form of the mathematical set-builder notation (set comprehension) as distinct from the use of map and filter functions. Consider the following example in set-builder notation:

$$S = \{2 \cdot x | x \in \mathbb{N}, x^2 > 3\}$$

or often

$$S = \{2 \cdot x : x \in \mathbb{N}, x^2 > 3\}$$

This can be read as, **S** is the set of all numbers, 2 times of **x**, where **x** is an element or member of set natural number $\mathbb{N}$, and **x** square is greater than 3. The smallest natural number is 1 and the condition $1^2 > 3$, fail to satisfy the condition so it is not included in the new list, else all other numbers are added. The given set-builder notation can be annotated as follows,

$$S = \left\{ \underbrace{2 \cdot x}_{\text{output expression}} \mid \underbrace{x}_{\text{variable}} \in \underbrace{\mathbb{N}}_{\text{input set}}, \underbrace{x^2 > 3}_{\text{predict}} \right\}$$

where,

- **x** is the variable representing numbers of an input set or a list.
- $\mathbb{N}$ represents the input set, in this example we have taken the set of natural number.
- $x^2 > 3$, is a predictor used as a filter out some items from existing list or set
- $2 \cdot x$ is an output expression producing member for new list or set
- Symbol **{}** indicates that the result is a set
- The vertical bar **|** is read as such that
- Comma (,) separates the predict and read as AND

A list comprehension has the same syntactic components to represent generation of a list in order from an input list or iterator:

```
newlist = [expression for item in iterable if condition == True]
```

In the above syntax of list comprehension,

- The **condition** is similar to the predict rule as shown above and it is optional and used when some selected items are picked
- An **iterable** can be anything from string, list, tuple, dictionary, tuple and set.
- **expression** can have any valid python expression

Let's explore list comprehension with some examples and generate a new list from a given list **fruits**, containing only the fruits with the letter "a" in the name.

```
1 fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
2 newlist = [name for name in fruits if "a" in name]
3 print(newlist)
```

```
['apple', 'banana', 'mango']
```

In line 2 of the above illustration, if letter "a" is in **name** having the value as a fruit names from **fruits** iterator, is the predictor used and worked as filter out some fruits name from the given iterable object **fruits** and fruit names are used as part of the **expression**. If we want to generate the same list without

list comprehension it needs more than one line of source code and a conditional statement need to be put inside for statement, as follows:

```
1 fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
2 newlist = []
3
4 for fruit_name in fruits:
5     if "a" in fruit_name:
6         newlist.append(x)
7
8 print(newlist)
```

It is not necessary to use any predicate rules of expression as part of list comprehension. The next example, generates a new list without the using of any predicate rule. But as part of the expression each item of the given list is squared with the expression `number**2`.

```
>>>numbers = [3,4,5,6,8]
>>>nums = [number**2 for number in numbers]
>>>print(nums)
[9, 16, 25, 36, 64]
```

In list comprehension a condition is act as a filter that only accepts the items, if it evaluates to True. See the next illustration:

```
>>>numbers = [3,4,5,6,8]
>>>nums = [number**2 for number in numbers if number % 2 == 0]
>>>print(nums)
[16, 36, 64]
```

The condition `if number % 2 == 0`, will be True for all the numbers are even, making a new list containing the square of all the even numbers. As part of the iterable object anything can be used for a sequence type. It is also possible to generate a sequence through `range()` function:

```
>>>[x+1 for x in range(10)]
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

You can also use a string as an iterable and generating a list of letters of a given string, using comprehension:

```
>>>[char for char in "Python"]
['P', 'y', 't', 'h', 'o', 'n']
```

The expression `char` is the current item in the iteration, but it is also the outcome, which you can manipulate before it ends up like a list item in the new list. The next example converting all the fruits name to upper case:

```
>>>fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
>>>[x.upper() for x in fruits]
['APPLE', 'BANANA', 'CHERRY', 'KIWI', 'MANGO']
```

You can also generate a new list containing all the same item of equal length of existing one, as shown below:

```
>>>fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
>>>['banana' for x in fruits]
['banana', 'banana', 'banana', 'banana', 'banana']
```

The expression can also contain a condition, not like a filter, but as a way to manipulate the outcome. The given condition, `if x != "banana"` checks if the current element (`x`) is not equal to "banana". If it's True, then the current element (`x`) itself is included in the new list. If it's False (`x == "banana"`), then "orange" is included in the new list instead.

```
>>>fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
>>>[x if x != "banana" else "orange" for x in fruits]
['apple', 'orange', 'cherry', 'kiwi', 'mango']
```

7.1.10 2-d List

Working with 2-d list includes the understanding of the structure, accessing elements, and efficiently manipulating data in a two-dimensional grid. When working with structured data or grids, 2-d lists can be useful. A 2-d list is a collection of a list of lists, which represents a table-like structure with rows and columns. In Python matrices are represented as 2-d list and python offers many ways to create a 2-d list. Let's see our first example of creating a 2-d list with 2 rows and 2 columns:

```
>>>A = [[1,2],[3,4]]
>>>A
[[1, 2], [3, 4]]
```

The 2D list is enclosed within double square brackets `[]`. Inside the outer brackets, there are two inner square brackets `[]`, each containing two elements. The memory view of a 2-d list is like a grid only and to access any item from a 2-d list we need to pass two indexes, one for row and other for column. See the next tabular view to understand the indexing of a 2-d list.

	0	1
0	1 [0][0]	2 [0][1]
1	3 [1][0]	4 [1][1]

In the above grid numbers on top row and left-most column represents column and row indexes. Numbers inside the square `[]` brackets, such as `[0][0]` represents the location of the first item. To access of these items, we need to combine row and column indices with the list name, as follows:

```
>>>A[0][0]
1
>>>A[1][0]
3
```

If a 2-d list is small in size then it is easy to be declared with values. But if we want populate a 2-d list based on user input, we need to do it row wise and keep appending it to the main 2-d list. Here is an example of taking input from user of 3×3 matrix.

```

1 MAT = []
2 rows, cols = (3,3)
3 #Taking input rowwise
4 for i in range(0,rows):
5     row = input(f'Enter data:').split()
6     row = [int(item)for item in row]
7     MAT.append(row)
8
9 #print 2-d list in matrix form
10 for i in range(0,rows):
11     for j in range(0, cols):
12         print(MAT[i][j], end=' ')
13     print()

```

```

Enter data:1 2 3
Enter data:4 5 6
Enter data:7 8 9
1 2 3
4 5 6
7 8 9

```

The various components of the above illustration are described below:

- Initially an empty 1-d list **MAT = []** is declared at line 1, which will eventually hold the 2D data.
- In line 2, **rows, cols = (3, 3)** assigns the values 3 to both rows and cols variables. These represent the dimensions of the 2D list, creating a 3x3 matrix in this case.
- The for loop iterates three times (0, 1, 2) based on the value of rows. prompts the user to enter data for each row. The **.split()** method splits the input string into a list of elements (assuming space-separated values).
- The list comprehension at line 6, converts each item in the row list from string to integer. **MAT.append(row)**, at line 7, adds the processed row (now a list of integers) to the MAT list. This builds the 2D structure row by row.
- The outer loop at line 10, iterates three times again to control the rows when printing. The inner loop at line 11, iterates three times within each outer loop iteration to control the columns when printing.

Like 1-d list a 2-d list can also be created through nested list comprehension. Personally, I didn't prefer working with 2-d list as list comprehension as syntax are complex. But as per anyone interest you can use it while declaring a 2-d list and latter values can be appended by accessing individual locations. The next illustration demonstrates the creation of 2-d list through list comprehension.

```
# Create a 2D list with 3 rows and 4 columns filled with zeros
```

```

rows = 3
cols = 4
matrix = [[0 for _ in range(cols)] for _ in range(rows)]

# Display the 2D list
for row in matrix:
    print(row)

```

The above illustration demonstrates the use of list comprehension for creating a 2D list filled with zeros. Here's a breakdown:

- In the list comprehension, outer **for _ in range(rows)** loop iterates three times (0, 1, 2) to create three rows.
- Within the outer loop, the inner **[0 for _ in range(cols)]** list comprehension creates a sub-list (representing a row) filled with zeros.
- The inner loop iterates four times (0, 1, 2, 3) to create four zeros for each row.
- The entire expression creates a nested structure, building the 2D list with rows containing zeros.

7.2 Tuple

Tuple is one of the built-in data types in Python used to store collections of data like list, set and dictionary. A is an immutable ordered sequence of items, means existing objects in the sequence cannot be modified, and new elements cannot be added once the sequence is created. Two ways we can create a tuple, one using **tuple()** constructor method and other using a pair of empty parentheses “()”, as shown below:

```

>>>t = ()
>>>type(t)
<class 'tuple'>
>>>t = tuple()
>>>type(t)
<class 'tuple'>

```

A tuple consists of several values separated by commas. Tuple items are ordered, unchangeable, and allow duplicate values. Tuple items are indexed like list and the first item is accessed using index **[0]**.

```

>>>t = ("apple", "banana", "cherry")
>>>t[0]
'apple'

```

A tuple object can iterate through a loop using for or while. We can also use **range()** function to generate the index of every item, as shown below:

```

fruits = ("apple", "banana", "cherry")
#Accesing each item
for fruit in fruits:
    print(fruit,end=' ')

```

```
#Accessing items using index
for i in range(0,len(fruits)):
    print(fruits[i], end=' ')
```

Membership of any item can also be tested using membership operators, as follows:

```
1 if 'apple' in fruits:
2     print('Fruits has an apple')
```

Fruits has an apple

Tuple also support slicing and indexing like string and list which includes negative indexing. Some of the examples are listed below:

```
>>>fruits[::]
('apple', 'banana', 'cherry')
>>>fruits[1:3]
('banana', 'cherry')
>>>fruits[::-1]
('cherry', 'banana', 'apple')
```

Trying to update an item of tuple will raise a `TypeError`, as tuple object doesn't support item assignment.

```
>>>fruits[0]='mango'
Traceback (most recent call last):
  File "<pyshell#12>", line 1, in <module>
    fruits[0]='mango'
TypeError: 'tuple' object does not support item assignment
```

To create a tuple with only one item, you have to add a comma after the item, otherwise Python will not recognize it as a tuple.

```
>>>fruits = ('banana',)
>>>fruits
('banana',)
```

If comma is omitted Python won't recognise that object as a tuple instead considered as string class object.

```
>>>fruits = ('banana')
>>>type(fruits)
<class 'str'>
```

7.2.1 Modifying a Tuple

Tuples are ordered collections of elements, similar to lists. However, unlike lists, tuples cannot be

modified after creation. This means you can't change the values of individual elements within a tuple. But there is a workaround and you can convert the tuple into a list, change the list, and convert the list back into a tuple.

```
1 x = ("apple", "banana", "cherry")
2 y = list(x)
3 y[1] = "kiwi"
4 x = tuple(y)
5 print(x)
```

```
('apple', 'kiwi', 'cherry')
```

7.2.2 Adding Items

Since tuples are immutable, directly you can't add an item to it. But there are other ways to add items to a tuple, by converting a tuple to list we can add item to it and later convert it to a tuple, as follows:

```
1 fruits = ("apple", "banana", "cherry")
2 l1 = list(fruits)
3 l1.append("orange")
4 fruits = tuple(l1)
5 print(fruits)
```

```
('apple', 'banana', 'cherry', 'orange')
```

Another way we can do by adding a tuple to another tuple instead of converting to a list.

```
>>>t1 = (2,3)
>>>t2 = (4,)
>>>t1 = t1+t2
>>>print(t1)
(2, 3, 4)
```

7.2.3 Methods

Tuple objects supports only two built-in methods:

- **count()** - Return the number of times that value appears in the tuple
- **index()** - Search for the first occurrence of the value, and return its position. Returns `ValueError`, if the given value is not present.

```
>>> t = (1,2,3,4,4,5)
>>> t.count(4)
2
>>> t.index(4)
3
>>> t.index(110)
```

Traceback (most recent call last):

```
File "<stdin>", line 1, in <module>
ValueError: tuple.index(x): x not in tuple
```

7.2.4 Packing and Unpacking

Packing and unpacking tuples in Python are convenient ways to work with multiple values. Here's a breakdown of both concepts:

Packing refers to creating a tuple by assigning multiple values within parentheses `()` and the order you specify the values becomes the order they are stored within the tuple.

```
>>>a, b, c = 1, 2, 3
>>>t = (a,b,c) #Packing to an object
>>>type(t)
<class 'tuple'>
```

In this example `t` become a tuple containing three integers 1,2,3. While Unpacking assigns individual values from a tuple to separate variables. The number of variables must match the number of elements in the tuple.

```
>>>a1, b1, c1 = t #unpacking an object
>>>print(a1, b1, c1)
1 2 3
```

You can unpack tuples with more or less elements than variables using extended unpacking with an asterisk `*`. Unpacking with `*` before a variable name captures all remaining elements in a list.

```
>>> fruits = ("apple", "banana", "cherry", "mango", "orange")
>>> first_fruit, second_fruit, *other_fruits = fruits
>>> print(first_fruit)
apple
>>> print(second_fruit)
banana
>>> print(other_fruits)
['cherry', 'mango', 'orange']
```

7.2.5 Why tuple?

Tuples are a data structure in Python, and they have various use cases where they are preferable over other data structures like lists or dictionaries. Tuples are similar to lists in that they can store multiple items, but they have some key differences:

- **Immutable Data:** Tuples are immutable, meaning their elements cannot be modified after creation. This immutability makes tuples suitable for use cases where data should not change.
- **Hashable:** Tuples are hashable, which means they can be used as keys in dictionaries, while lists cannot.

Here are some common use cases when tuples are preferable:

- **Return Multiple Values from Functions:** Tuples are often used to return multiple values from a function when you want to ensure that the values are treated as a single unit, discussed in Chapter 8.
- **Unpacking:** Tuples can be used for unpacking values returned from functions or when iterating through sequences, just discussed in Section 7.2.4.
- **Dictionary Keys:** Since tuples are hashable, they can be used as keys in dictionaries when the key is a combination of multiple values, discussed more in details later in this Chapter.

```
>>> coordinates = {(1, 2, 3): "Point A", (4, 5, 6): "Point B"}
>>> coordinates[(1,2,3)]
'Point A'
```

- **Records or Structs:** Tuples can represent a fixed collection of related data fields, similar to a lightweight struct or record.
- **Immutability:** When you need to ensure that data remains constant throughout the program, using tuples can prevent accidental modifications.
- **Performance:** Tuples can be slightly faster than lists when it comes to iteration and indexing since they are immutable and have a fixed structure.
- **Data Integrity:** Tuples can be used to enforce data integrity by preventing the accidental modification of values within a collection.
- **Function Arguments:** Tuples can be used to pass multiple arguments to functions without explicitly defining named parameters. Discussed in Chapter 8.

Remember that the choice between tuples, lists, and other data structures depends on the specific requirements of your program and the nature of the data you are working with. Tuples are particularly useful when you need a fixed, unchangeable collection of items.

7.2.6 Tuple Comprehension

In Python, there is no direct tuple comprehension construct like there is for list comprehensions. However, you can create a tuple using a generator expression and the `tuple()` constructor to convert it into a tuple. A generator expression is similar to a list comprehension, but it returns an iterable generator object instead of a list. Here's the basic structure of a tuple comprehension using a generator expression:

```
tuple_variable = tuple(expression for element in iterable)
```

The meaning of each component in the above syntax is summarised below:

- **expression:** The expression to evaluate for each element in the iterable.
- **element:** The variable that takes on each value from the iterable.
- **iterable:** The iterable you want to loop through to create the tuple.

Here's an example of creating a tuple using a generator expression:

```
>>> numbers = [1, 2, 3, 4, 5]
>>> squared_tuple = tuple(x ** 2 for x in numbers)
>>> print(squared_tuple)
(1, 4, 9, 16, 25)
```

7.3 Set

In Python, a set is a built-in data structure used to store a collection of unique and unordered elements. Items of the set are immutable type objects and sets are defined by enclosing a comma-separated sequence of elements within curly braces `{}` or by using the built-in `set()` constructor method. Once a set is created set items are not modifiable, but you can add new items using `.add(elem)` method. Let us first create a list by using comma-separated sequence of elements:

```
>>>my_set = {1, 2, 3, 4, 5}
>>>my_set
{1, 2, 3, 4, 5}
```

Note that you should not use empty curly braces to create an empty set because Python will interpret it as a dictionary.

```
>>>s = {}
>>>type(s)
<class 'dict'>
```

The `set()` constructor allows you to create an empty set. Items can be added later using the `add()` method. As shown in the next illustration

```
>>>s = set()
>>>type(s)
<class 'set'>
>>>s.add(1)
>>>s.add(2)
>>>s
{1, 2}
```

Remember adding an item is not ordered in a set like a list. It can be placed anywhere of a given object and like list, set doesn't keep duplicate items. Python simply ignored the item and keep the first item. Now a duplicate item to set `s`, that we have created in the previous illustration.

```
>>>s.add(2) #adding a duplicate item
>>>s
{1, 2}
```

While creating a set using `set()` constructor method, you can pass an iterable (e.g., a list or tuple) object as an argument. This will create a set with the unique elements from the iterable. For example:

```
>>>my_list = [1, 2, 2, 3, 4, 4, 5]
>>>my_set = set(my_list)#duplicate items rejected
>>>my_set
{1, 2, 3, 4, 5}
```

Similar to the list comprehensions, you can use set comprehensions to create sets by applying an expression to each element of an iterable. For example:

```
>>>my_list = [1, 2, 3, 4, 5]
>>>my_set = {x * 2 for x in my_list}
>>>my_set
{2, 4, 6, 8, 10}
```

Like list and tuple using **in** and **not in** operators can be used to test the membership of any item and **len()** function returns the number of elements in a set:

```
>>>len(my_set)
5
>>>2 in my_set
True
>>>11 in my_set
False
```

Sets items are not accessible using index like list or tuple objects. Sets items are iterate through loop using for loop as follows:

```
>>> s = {1,2,3}
>>> for item in s:
...     print(item, end=' ')
...
1 2 3
```

7.3.1 Operations on Set

Union of two sets: Set operations in Python can be performed in two different ways: by operator or by method. Let's take a look at how these operators and methods work, using set union as an example. Given two sets, **set1** and **set2**, the union of **set1** and **set2** is a set consisting of all elements in either set. Consider these two sets:

```
>>>set1 = {1,3,4,5,6}
>>>set2 = {1,7,3,10}
```

In Python, set union can be performed with the union (**|**) operator:

```
>>>set1 | set2
{1, 3, 4, 5, 6, 7, 10}
```

Note that elements 1 and 3, which appears in both the sets, appears only once in the union operation. Set union can also be obtained with the **union()** method. The method is invoked on one of the sets, and the other is passed as an argument:

```
>>>set1.union(set2)
```

```
{1, 3, 4, 5, 6, 7, 10}
```

The working of union operator (`|`) and `union()` method is quite different. For union operator `|`, both the operands must be of set type, but `union()` method can accept any iterable type as argument, later converts that iterable as set and perform union operation. See the next illustration:

```
>>>num1 = {1,3,4,5,6}
>>>num2 = tuple([11,2,3])
>>>num1|num2
Traceback (most recent call last):
  File "<pyshell#31>", line 1, in <module>
    num1|num2
TypeError: unsupported operand type(s) for |: 'set' and 'tuple'
>>>num1.union(num2)
{1, 2, 3, 4, 5, 6, 11}
```

Python raised `TypeError` if any of the operand is a different type of object, other than set. But `union()` can take any type of iterable object as argument and perform the union operation. More than two sets may be specified with either the operator or the method:

```
>>>a = {1, 2, 3, 4}
>>>b = {2, 3, 4, 5}
>>>c = {3, 4, 5, 6}
>>>a.union(b, c)
{1, 2, 3, 4, 5, 6}
>>>a | b | c
{1, 2, 3, 4, 5, 6}
```

Intersection between two sets: `a.intersection(b)` and `a & b` return the set of elements common in both sets `a` and `b`. You can also specify multiple sets with the intersection method or operator, just like you can with set union operation. If there are no common elements between the sets, the intersection will be an empty set `set()`.

```
>>>a.intersection(b)
{2, 3, 4}
>>>a & b
{2, 3, 4}
>>> c = {6,7}
>>> a & c
>>> a & c #returns an empty set
set()
```

Like set union method and operator, the `intersection()` method and intersection (`&`) operator can handle intersections of more than two sets at once.

Difference between two sets: Set difference finds elements that are in the first set but not in the second. `a.difference(b)` and `a-b` is the required syntax to get the set of all elements that are in `a` but not in `b`:

```
>>>a.difference(b)
```

```
{1}
>>>a-b
{1}
```

Another way to think of this is that `a.difference(b)` and `a-b` return the set, that results when elements are present in `a` not in `b`. In simpler terms it removes the “overlap” between `a` and `b`. Once again, you can specify more than two sets, when multiple sets are specified, the operation is performed from left to right. In the example below, `a - b` is computed first, resulting in `{1, 2, 3, 300}`. Then `c` is subtracted from that set, leaving `{1, 2, 3}`.

```
>>>a = {1, 2, 3, 30, 300}
>>>b = {10, 20, 30, 40}
>>>c = {100, 200, 300, 400}
>>>a-b-c
{1, 2, 3}
```

Symmetric Difference: In Python, `a.symmetric_difference(b)` and `a^b` are two ways to find the symmetric difference between two sets `a` and `b`. The symmetric difference of two sets contains all elements that are in either of the sets but not in their intersection. In other words, it includes elements that are unique to each set.

```
>>>a = {1, 2, 3, 30, 300}
>>>b = {10, 20, 30, 40}
>>>a^b
{1, 2, 3, 40, 10, 300, 20}
```

The symmetric difference can also be performed using difference operator between the union and intersection operations of two sets as follows:

```
>>>(a | b) - (a & b)
{1, 2, 3, 40, 10, 300, 20}
```

Like other set operators’ symmetric difference (^) operator also allowed with multiple sets to perform symmetric difference. But `symmetric_difference()` method exactly allows only one argument.

Disjoint of two sets: Using `a.isdisjoint(b)` method we can determine `a` and `b` are disjoint set or not. Two sets are disjoint, if they have no elements in common, and if `a.isdisjoint(b)` is True, then `a & b` returns an empty set

```
>>>a = {1, 2, 3, 30, 300}
>>>b = {10, 20, 30, 40}
>>>a.isdisjoint(b)
False
>>>x1 = {1, 3, 5}
>>>x2 = {2, 4, 6}
>>>x1.isdisjoint(x2)
True
>>>x1 & x2
set()
```

Subset of a set: In set theory, a set **a** is considered a subset of another set **b**, if every element of **a** is in **b**. The required logic for subset operation is shown below:

```
a = {1, 2, 3}
b = {1, 2, 3, 4, 5}
flag = True
#For subset length of set a should be less than or equal to b
if len(a)>len(b):
    print('a is not a subset of b')

else:
    #iterating over each item of a
    for item in a:
        #checking membership in set b
        if item not in b:
            #update the flag and break from loop
            flag = False
            break
    if flag:
        print('a is subset of b')
    else:
        print('a is not a subset of b')
```

Python offers a method `issubset()` and a subset operator (`<=`) to check a set is subset of another or not.

```
>>>a.issubset(b)
True
>>>a<=b
True
```

Proper subset: The operation `a<b`, determines whether one set is a proper subset of the other. A proper subset is the same as subset, except that the sets can't be identical. A set **a** is considered a proper subset of **b**, if every element of **a** is in **b**, and **a** and **b** are not equal.

```
>>>a = {1,2}
>>>b = {1,2,3}
>>>a<b
```

```
True
```

The `<` operator is the only way to test whether a set is a proper subset or not. There is no corresponding method.

Superset: Superset operator (`>=`) is used to check a set is superset of another or not and the corresponding method is `issuperset()`. A set **b** is considered a superset of another set **a**, if **b** contains all the element of **a**.

```
>>>b>=a
True
>>>b.issuperset(a)
```

True

`a > b`, determines whether one set is a **proper superset** of the other. A set `a` is considered a proper superset of another set `b`, if `b` contains every element of `a`, and `a` and `b` are not equal.

```
>>>b>a
```

True

```
>>>a.add(3) #both sets are equal
```

```
>>>b>a
```

False

The proper superset (`>`) operator is the only way to test whether a set is a proper superset or not, there is no corresponding method.

7.3.2 Modifying a set

`a.remove(<elem>)` removes an `<elem>` from set `a`, if `<elem>` is not present, Python will raise an `KeyError` exception:

```
>>>a = {1,2,3,4,5}
```

```
>>>a.remove(1)
```

```
>>>a
```

```
{2, 3, 4, 5}
```

If you don't want to handle any exception while removing an element from a set, then use `x.discard(<elem>)`, instead of `a.remove(<elem>)`. This method quietly does nothing if `<elem>` is not in set `a`.

```
>>>a.discard(10)
```

```
>>>a
```

```
{2, 3, 4, 5}
```

```
>>>a.discard(2)
```

```
>>>a
```

```
{3, 4, 5}
```

`a.pop()` removes and returns an arbitrarily chosen element from set `a`. If set `a` is empty, `a.pop()` raises an exception:

```
>>> a.pop(), a.pop(), a.pop()
```

```
(3, 4, 5)
```

```
>>> a.pop()
```

```
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
```

```
KeyError: 'pop from an empty set'
```

Instead of removing items one by one `clear()` method removes all elements from a set. A set can be updated by adding items from other set using `update()` method or `|=` operator.

```
>>> b
```

```
{2, 3, 4, 5}
```

```
>>> b.update({6,7})
```

```
>>> b
{2, 3, 4, 5, 6, 7}
```

An in-place update of a set can be done using `intersection_update(other_set)` method or `&=` operator, that modifies a set by keeping only the elements common to both sets.

```
>>> a = {1,2,3}
>>> b = {2, 3, 4, 5, 6, 7}
>>> id(a)
2309581386496
>>> a.intersection_update(b)
>>> a
{2, 3}
>>> id(a)
2309581386496
```

An in-place modification changes the values or elements within an existing data structure (like a list, set, or object). The original data structure is mutated, and it reflects the changes made. In the above illustration `id()` on `a`, before and after modification returns same memory reference. Like `intersection_update(other_set)`, `difference_update(other_set)` method or `-=` operator modifies a set by removing elements found in the other set as in-place modification.

```
>>> a = {1,2,3}
>>> b
{2, 3, 4, 5, 6, 7}
>>> a.difference_update(b)
>>> a
{1}
```

The last method of this type is `symmetric_difference_update(other_set)` method or `^=` operator, modifies the set by keeping elements that are in exactly one of the sets.

```
>>> a = {1,2,3}
>>> a.symmetric_difference_update(b)
>>> a
{1, 4, 5, 6, 7}
```

Note, the non-operator versions of the `update()`, `intersection_update()`, `difference_update()`, and `symmetric_difference_update()` methods will accept any iterable as an argument.

7.4 Frozen Sets

Python provides another built-in type called a frozenset, which is in all respects exactly like a set, except that a frozenset is immutable. You can perform non-modifying operations on a frozenset. The inbuilt function called `frozenset()` that takes an iterable object as input and makes it immutable. Simply put, the Python Frozenset renders iterable objects unchangeable by freezing them.

```
>>> vowels = {'a', 'e', 'i', 'o', 'u'}
>>> fSet = frozenset(vowels)
>>> fSet
frozenset({'a', 'o', 'u', 'e', 'i'})
```

frozenset() takes only one parameter, that contains the elements that will be used to initialize the frozenset and returns a frozenset with items from the supplied iterable that is untouched. It returns an empty frozenset if arguments are not given.

7.4.1 Operations with frozenset

You can perform set operations such as union, intersection, and difference with other frozensets, sets, or even other iterable types (converted to sets or frozensets).

```
>>> frozen_set1 = frozenset([1, 2, 3])
>>> frozen_set2 = frozenset([2, 3, 4])

>>> frozen_set1 & frozen_set2 #intersection
frozenset({2, 3})
>>> frozen_set1 | frozen_set2 #union
frozenset({1, 2, 3, 4})
>>> frozen_set1 - frozen_set2 #difference
frozenset({1})
```

- Membership is allowed in frozenset and can be checked using **in** operator.
- You can check if one frozenset is a subset or superset of another using **issubset()** and **issuperset()** methods.
- You can get the number of elements in the frozenset using the **len()** function.
- You can convert a frozenset to a regular set, list, or tuple using the appropriate constructor.
- You can create a copy of a frozenset using the constructor or the **copy()** method.
- Since frozensets are hashable, you can use them as keys in dictionaries.
- You can iterate through the elements of a frozenset using a for loop.
- Unlike mutable sets, frozensets do not support methods like add, remove, or discard because they are immutable.
- It only requires the keys of the dictionary to produce a frozen set when you utilise a dictionary as an iterable.

```
>>> person = {'name': 'John', 'age': 23, 'sex': 'male'}
>>> frozenset(person)
frozenset({'age', 'name', 'sex'})
```

7.5 Dictionary

Python provides another composite data type called a dictionary, which is similar to a list in that it is a collection of objects. Dictionaries and lists share the following characteristics:

- Both are mutable.
- Both are dynamic. They can grow and shrink as needed.
- Both can be nested means a list and dictionary can contain another list or dictionary. A dictionary can also contain a list, and vice versa.

Dictionaries differ from lists primarily in how elements are accessed:

- List elements are accessed by their position in the list, via indexing.
- Dictionary elements are accessed via keys.

Dictionaries are Python's implementation of a data structure that is more generally known as an associative array. A dictionary consists of a collection of key-value pairs. Each key-value pair maps the key to its associated value. You can define a dictionary by enclosing a comma-separated list of key-value pairs in curly braces (`{}`). A colon (`:`) separates each key from its associated value:

```
records = {'name': 'Goutam',
           'age': 37,
           'city': 'Vellore',
           'Country': 'India'
}
```

You can also construct a dictionary with the built-in `dict()` function. The argument to `dict()` should be a sequence of key-value pairs. A list of tuples works well for this:

```
>>> records = dict([('name', 'Goutam'), ('age', 37), ('city', 'Vellore'),
...                 ('Country', 'India')])
>>> records
{'name': 'Goutam', 'age': 37, 'city': 'Vellore', 'Country': 'India'}
```

If the key values are simple strings, they can be specified as keyword arguments. So here is yet another way to define records:

```
>>> records = dict(name = 'Goutam', age = 37, city = 'Vellore', Country =
'India')
>>> records
{'name': 'Goutam', 'age': 37, 'city': 'Vellore', 'Country': 'India'}
```

The entries in the dictionary display in the order they were defined. But that is irrelevant when it comes to retrieving them. Dictionary elements are not accessed by numerical index like list. It will raise a `KeyError`, as shown below:

```
>>> records
{'name': 'Goutam', 'age': 37, 'city': 'Vellore', 'Country': 'India'}
>>> records[0]
Traceback (most recent call last):
  File "<pyshell#28>", line 1, in <module>
    records[0]
KeyError: 0
```

7.5.1 Building a Dictionary Incrementally

Defining a dictionary using curly braces and a list of key-value pairs, as shown above, is fine if you know all the keys and values in advance. But what if you want to build a dictionary on the fly? That you can start by creating an empty dictionary, which is specified by empty curly braces (`{}`). Then you can add new keys and values one at a time.

```
>>>person = {}
>>>type(person)
<class 'dict'>
```

Once an empty dictionary is created then many ways key-value pairs can be added to the dictionary. One of the methods is by using square brackets and the assignment operator (`=`). If the key does not exist, it will be added; if it does exist, the value associated with that key will be updated.

```
>>>person['fname'] = 'Joe'
>>>person['lname'] = 'Fonebone'
>>>person['age'] = 51
>>>person['pets'] = {'dog': 'Fido', 'cat': 'Sox'}
>>>person
{'fname': 'Joe', 'lname': 'Fonebone', 'age': 51, 'pets': {'dog': 'Fido',
'cat': 'Sox'}}
```

You can use the `update()` method to add multiple key-value pairs from another dictionary or an iterable of key-value pairs.

```
>>>my_dict = {}
>>>my_dict.update({'name': 'John', 'age': 30})
>>>my_dict
{'name': 'John', 'age': 30}
```

You can build a dictionary incrementally by iterating through an iterable and adding key-value pairs based on the elements of the iterable. See the illustration given below:

```
1 my_dict = {}
2 data = [('name', 'John'), ('age', 30)]
3 for key, value in data:
4     my_dict[key] = value
5
6 print(my_dict)

{'name': 'John', 'age': 30}
```

You can create a dictionary incrementally using a dictionary comprehension, which allows you to transform one or more iterables into a dictionary.

```
>>>keys = ['name', 'age']
>>>values = ['John', 30]
>>>my_dict = {k: v for k, v in zip(keys, values)}
>>>my_dict
{'name': 'John', 'age': 30}
```

You can define a function that adds key-value pairs to a dictionary and call that function to build the dictionary incrementally. More about `def` is discussed in Chapter 8.

```
1 def add_to_dict(d, key, value):
2     d[key] = value
3
4 my_dict = {}
5 add_to_dict(my_dict, 'name', 'John')
6 add_to_dict(my_dict, 'age', 30)
7 print(my_dict)

{'name': 'John', 'age': 30}
```

7.5.2 Accessing Dictionary Elements

A value is retrieved from a dictionary by specifying its corresponding key inside a pair of square brackets (`[]`):

```
>>>records['name']
'Goutam'
```

Adding an entry to an existing dictionary is simply a matter of assigning a new key and value using assignment operator:

```
>>>records['height'] = 5.56
>>>records
{'name': 'Goutam', 'age': 37, 'city': 'Vellore', 'Country': 'India',
'height': 5.56}
```

If you want to update an entry, you can just assign a new value to an existing key:

```
>>>records['city'] = 'Agartala'
>>>records
{'name': 'Goutam', 'age': 37, 'city': 'Agartala', 'Country': 'India',
'height': 5.56}
```

Retrieving the values in the sublist or subdictionary requires an additional index or key. Consider the **person** dictionary, which has children and pets as keys and values are of type list and dictionary.

```
>>>person = {'fname': 'Joe', 'lname': 'Fonebone', 'age': 51, 'spouse':
```

```
'Edna','children': ['Ralph', 'Betty', 'Joey'],
      'pets': {'dog': 'Fido', 'cat': 'Sox'}}
>>>person['children'][1]
'Betty'
>>>person['pets']['dog']
'Fido'
```

This example exhibits another feature of dictionaries: the values contained in the dictionary don't need to be the same type. In **person**, some of the values are strings, one is an integer, one is a list, and one is another dictionary. Just as the values in a dictionary don't need to be of the same type, the keys don't either. See the next example, illustrates a dictionary with different types of keys:

```
>>>foo = {42: 'aaa', 2.78: 'bbb', True: 'ccc'}
>>>foo
{42: 'aaa', 2.78: 'bbb', True: 'ccc'}
```

To delete an entry, use the **del** statement, specifying the key to delete:

```
>>>del records['height']
>>>records
{'name': 'Goutam', 'age': 37, 'city': 'Agartala', 'Country': 'India'}
```

7.5.3 Types of key's

In Python, dictionary keys can be of any hashable data type. A hashable data type is one that is both immutable and has a hash value that never changes during its lifetime. This is because dictionaries use a hash table to store their key-value pairs, and keys must be hashable to ensure efficient lookup and retrieval. Here are some common data types that can be used as dictionary keys:

- **Numeric types:** numeric data types such as integers, floating-point numbers, and complex numbers are generally considered hashable because they are immutable. As long as the value of a numeric data type remains constant, its hash value will also remain the same. Here are a few examples of numeric data types as dictionary keys:

```
# Integers as keys
int_dict = {1: "one", 2: "two", 3: "three"}
# Floats as keys
float_dict = {3.14: "pi", 2.71828: "e"}
# Complex numbers as keys
complex_dict = {1 + 2j: "complex_number_1", 3 - 4j: "complex_number_2"}
```

- We have already seen some examples of frozenset and string as dictionary key.
- You can even use built-in objects like **types** and **functions** as key:

```
>>>d = {int: 1, float: 2, bool: 3}
>>>d
{<class 'int'>: 1, <class 'float'>: 2, <class 'bool'>: 3}
```

```
>>>d[int]
1
```

- Custom Objects: You can use custom objects as dictionary keys if you implement the `__hash__()` and `__eq__()` methods to define their hash value and equality.
- It's important to note that mutable objects, such as lists and dictionaries, cannot be used as dictionary keys because they are not hashable, and their content can change, leading to unpredictable behavior. Using an unhashable object as a key will result in a `TypeError`.

```
d = {[1,2]:12}
```

```
Traceback (most recent call last):
```

```
File "<pyshell#36>", line 1, in <module>
    d = {[1,2]:12}
```

```
TypeError: unhashable type: 'list'
```

Key points related to dictionary key

- A given key can appear in a dictionary only once. Duplicate keys are not allowed. A dictionary maps each key to a corresponding value, so it doesn't make sense to map a particular key more than once.
- When you assign a value to an already existing dictionary key, it does not add the key a second time, but replaces the existing value

```
>>>MLB_team = {'Colorado' : 'Rockies', 'Boston' : 'Red Sox',
               'Minnesota': 'Twins', 'Milwaukee': 'Brewers', 'Seattle': 'Mariners'
               }
>>>MLB_team['Minnesota'] = 'Timberwolves'
>>>MLB_team
{'Colorado': 'Rockies', 'Boston': 'Red Sox', 'Minnesota': 'Timberwolves',
 'Milwaukee': 'Brewers', 'Seattle': 'Mariners'}
```

Instead of adding `'Minnesota': 'Timberwolves'` to the dictionary, it updates the associated value. Similarly, if you specify a key second time during the initial creation of a dictionary, the second occurrence will override the first:

```
>>>MLB_team = {'Colorado' : 'Rockies', 'Boston' : 'Red Sox',
               'Minnesota': 'Timberwolves', 'Milwaukee': 'Brewers',
               'Seattle' : 'Mariners', 'Minnesota': 'Twins'
               }
>>>MLB_team
{'Colorado': 'Rockies', 'Boston': 'Red Sox', 'Minnesota': 'Twins',
 'Milwaukee': 'Brewers', 'Seattle': 'Mariners'}
```

A dictionary key must be of a type that is immutable. You have already seen examples where several of the immutable types you are familiar with—integer, float, string, and Boolean—have served as dictionary keys. A tuple can also be a dictionary key, because tuples are immutable. However, neither a list nor

another dictionary can serve as a dictionary key, because lists and dictionaries are mutable:

```
>>>d = {[1, 1]: 'a', [1, 2]: 'b'}
Traceback (most recent call last):
  File "<pyshell#29>", line 1, in <module>
    d = {[1, 1]: 'a', [1, 2]: 'b'}
TypeError: unhashable type: 'list'
```

What does the error message say unhashable?

Technically, it is not quite correct to say an object must be immutable to be used as a dictionary key. More precisely, an object must be hashable, which means it can be passed to a hash function. A hash function takes data of arbitrary size and maps it to a relatively simpler fixed-size value called a hash value (or simply hash), which is used for table lookup and comparison. Python's built-in `hash()` function returns the hash value for an object which is hashable, and raises an exception for an object which isn't:

```
>>>hash('foo')
6821459173919347162
>>>hash([1, 2, 3])
Traceback (most recent call last):
  File "<pyshell#31>", line 1, in <module>
    hash([1, 2, 3])
TypeError: unhashable type: 'list'
```

All of the built-in immutable types you have learned about so far are hashable, and the mutable container types (lists and dictionaries) are not. So, for key purposes, you can think of hashable and immutable as more or less synonymous.

7.5.4 Built-in Methods

Dictionary come with a variety of built-in methods to manipulate and work with the data stored in them. Here are some common dictionary methods:

- **dict.keys():** Returns a view of all the keys in the dictionary as a type of `dict_keys`. An object of type `dict_keys` can be passed inside a `list()` to convert it a list of keys. See the illustration shown below:

```
>>>d = {'a': 10, 'b': 20, 'c': 30}
>>>d.keys()
dict_keys(['a', 'b', 'c'])
>>>list(d.keys())
['a', 'b', 'c']
```

This method can be also used in a for loop to iterate over a dictionary and get the associated value with respective key:

```
>>> d = {'a': 10, 'b': 20, 'c': 30}
>>> for key in d.keys():
...     print(d[key],end=' ')
...
10 20 30
```

- **dict.values():** Returns a view of all the values in the dictionary as a type of dict_values and can be passed inside built-in **list()** method to get all the values as a list.

```
>>> d.values()
dict_values([10, 20, 30])
```

- **dict.items():** Returns a view of all key-value pairs (tuples) in the dictionary. Instead of using keys() and values(), if we need both then items() can be used in a for loop to iterate key and value pair.

```
>>> d = {'a': 10, 'b': 20, 'c': 30}
>>> for key, value in d.items():
...     print(f'{key}:{value}',end=' ')
...
a:10 b:20 c:30
```

- **dict.get(key, default):** Returns the value for the specified key, or a default value if the key is not found.

```
>>>my_dict = {'name': 'John', 'age': 30}
>>>my_dict.get('name', 'Unknown') #returns John
'John'
>>>my_dict.get('country', 'USA') #returns default value
'USA'
```

- **dict.popitem():** Removes and returns an arbitrary key-value pair as a tuple.

```
>>>d = {'a': 10, 'b': 20, 'c': 30}
>>>d.popitem()
('c', 30)
>>>type(d.popitem())
<class 'tuple'>
>>>d
{'a': 10, 'b': 20}
```

The arbitrariness of the removed pair means that you cannot predict which key-value pair **popitem()** will return. If you need to remove the last item from a dictionary, you can use the **pop()** method by specifying the key you want to remove. Here's an example:

```
>>>d.pop('c')
```

30

```
>>>d
{'a': 10, 'b': 20}
```

- **dict.clear()**: Removes all key-value pairs from the dictionary, leaving it empty.

```
>>>d = {'a': 10, 'b': 20, 'c': 30}
>>>d.clear()
>>>d
{}
```

- **dict.copy()**: Returns a shallow copy of the dictionary. A shallow copy of a dictionary is a new dictionary that contains references to the same objects (key-value pairs) as the original dictionary, rather than creating copies of the objects themselves. Consider the dictionary given below, which contains nested dictionary as value of **original_dict**:

```
original_dict = {
    'name': 'John',
    'info': {
        'age': 30,
        'address': {
            'city': 'New York',
            'state': 'NY'
        }
    }
}
```

In the above declaration, a dictionary **info** is nested inside **original_dict** and **address** is nested inside **info**. Creating a shallow copy of it, means that changes made to the objects within the dictionary will be reflected in both the original and the shallow copy, as they still reference the same objects.

```
# Create a shallow copy of the original_dict
>>> shallow_copy = original_dict.copy()
>>> shallow_copy['name'] = 'Alice'
>>> shallow_copy['info']['age'] = 25
>>> shallow_copy['info']['address']['city'] = 'Los Angeles'
>>> shallow_copy
{'name': 'Alice', 'info': {'age': 25, 'address': {'city': 'Los Angeles',
'state': 'NY'}}}
>>> original_dict
{'name': 'John', 'info': {'age': 25, 'address': {'city': 'Los Angeles',
'state': 'NY'}}}
```

Since we have modified the nested values associated with age and city through **shallow_copy**, the values get inside the **original_dict** gets updated. Because both of the dictionary objects refers the same dictionary object and that can be verified by checking the identity of these objects. But the associated value with key **'name'** remains same in the **original_dict**.

If you don't want to modify the **original_dict**, while modifying the copied dictionary, then you need to produce a deeper copy of the **original_dict** by using **deepcopy()** method of **copy** module. An example shown below:

```
>>> import copy
>>> deep_copy = copy.deepcopy(original_dict)
>>> # Modify the deep_copy
>>> deep_copy['name'] = 'Alice'
>>> deep_copy['info']['age'] = 25
>>> deep_copy['info']['address']['city'] = 'Los Angeles'
>>> # Check the changes in the deep_copy and original_dict
>>> original_dict
{'name': 'John', 'info': {'age': 30, 'address': {'city': 'New York', 'state': 'NY'}}}
>>> deep_copy
{'name': 'Alice', 'info': {'age': 25, 'address': {'city': 'Los Angeles', 'state': 'NY'}}}
```

Before illustration of the above code, we have redefined the **original_dict** with previous key-value pairs. The recursive part of deep copying involves copying not only the top-level objects but also delving into nested objects and copying them as well. This process continues until all nested objects are duplicated, creating a complete independent copy of the entire structure. In contrast, a shallow copy of the same compound object would only copy references to the nested objects, meaning changes in the nested objects would still be reflected in both the original and the shallow copy.

- **dict.fromkeys(keys, value):** This method is used to create a new dictionary from keys and an optional value passed as parameter. The keys must be an iterable object such as list, tuple or string and if you don't provide a value in the second argument, all the keys will be assigned **None** by default.

```
>>> keys = ['name', 'age', 'country']
>>> new_dict = dict.fromkeys(keys)
>>> new_dict
{'name': None, 'age': None, 'country': None}
```

- **dict.setdefault():** method is used with dictionaries to add a new key-value pair to an existing dictionary object, if the key does not exist. Else it will return the associated value with if key exist. This method is particularly useful when you want to ensure that a key is present in a dictionary and provide a default value if it's not.

```
>>> my_dict = {'name': 'John', 'age': 30}
>>> my_dict.setdefault('country', 'USA')
'USA'
>>> my_dict
{'name': 'John', 'age': 30, 'country': 'USA'}
>>> my_dict.setdefault('country', 'India')
'USA'
```

In the above illustration, the key-value pair (**country : USA**) is added to **my_dict**, and on trying to adding a same key, the method returns the existing associated value with it. If you didn't pass the associated value, a **None** value will be added with the key. In the example listed below, the **'city'** key

did not exist, so `setdefault()` adds the key with a value of `None`.

```
>>>my_dict = {'name': 'John', 'age': 30}
>>>city = my_dict.setdefault('city')
>>>print(my_dict)
{'name': 'John', 'age': 30, 'city': None}
>>>print(city)
None
```

- `dict.update(other_dict)`: method in Python is used to update or merge the key-value pairs of one dictionary (`other_dict`) into another dictionary. It is a way to add, modify, or overwrite key-value pairs in the original dictionary with those from `other_dict`. Two dictionaries can be merged using `update()` method as follows:

```
>>>dict1 = {'name': 'John', 'age': 30}
>>>dict2 = {'city': 'New York', 'age': 31}
>>>dict1.update(dict2)
>>>print(dict1)
{'name': 'John', 'age': 31, 'city': 'New York'}
```

Values of one dictionary can be updated using the key:value pair from another dictionary using `update()` method.

```
>>>dict1 = {'name': 'John', 'age': 30}
>>>dict2 = {'age': 31, 'city': 'New York'}
>>>dict1.update(dict2)
>>>print(dict1)
{'name': 'John', 'age': 31, 'city': 'New York'}
```

You can also use the keyword arguments syntax to pass key-value pairs directly to the `update` method.

```
>>>dict1 = {'name': 'John', 'age': 30}
>>>dict1.update(city='New York', job='Engineer')
>>>print(dict1)
{'name': 'John', 'age': 30, 'city': 'New York', 'job': 'Engineer'}
```

Key Takeaway's

- Lists are created by enclosing a comma-separated sequence of elements within square brackets `[]`.
- Be cautious about aliasing (two variables referring to the same list), which can lead to unintended side effects.
- Use `list.copy()` or slicing `[:]` to create a shallow copy of a list.
- You can clear a list by assigning an empty list to it: `my_list = []`
- List supports common operations like adding, removing, searching and counting elements.
- Lists can be memory-intensive when they contain many elements. In such cases, consider using generators or other data structures.

- Lists are suitable for storing collections of items when you need to maintain order, and you need the ability to modify the collection during the program's execution
- Lists are indexed by integers, while dictionaries are indexed by keys.
- Lists allow duplicates and have an order, while sets do not allow duplicates and have no order.
- Lists are like tuples but are mutable, whereas tuples are immutable.
- Sets are created using curly braces `{}` or the `set()` constructor method. But a pair of empty curly braces `{}` creates a dictionary, instead of a set.
- Sets are mutable, meaning you can add and remove elements.
- Sets do not support indexing, slicing, or other sequence operations because they are unordered.
- Sets are commonly used for eliminating duplicates and performing mathematical set operations.
- A frozenset is an immutable version of a set and created using the `frozenset()` constructor method.
- Frozensets are hashable and can be used as keys in dictionaries, unlike sets.
- Frozensets are useful when you need an immutable collection of unique elements and when you want to use them as keys in dictionaries.
- A dictionary is an unordered collection of key-value pairs are created using curly braces `{}` or the `dict()` constructor method.
- Dictionary keys must be unique, and they are case-sensitive.
- Values in a dictionary do not need to be unique, and the same value can be associated with multiple keys.
- In Python versions prior to 3.7, dictionaries were unordered, meaning the order of key-value pairs was not guaranteed. In Python 3.7 and later, dictionaries maintain insertion order.
- Dictionaries are used to store and retrieve data efficiently, and they are suitable for situations where you need to associate keys with values.

Practice Questions

1. Analyse a stock portfolio by storing stock symbols, quantities, and prices in tuples, and then calculate total portfolio value, average price, maximum stock price, minimum stock price, total stock. Consider the given portfolio:

```
portfolio = [("AAPL", 10, 150.0), ("GOOGL", 5, 2700.0),
             ("TSLA", 8, 700.0),
             ("AMZN", 4, 3500.0),
            ]
```

2. Take a list of numbers as input from user and finds the sum and product of all the numbers.
3. Take a list of numbers as input from user and find the maximum and minimum values and print them without using `min()` and `max()` function.
4. Write a program to manage student records with attributes like name, age, and CGPA. Store each student's information as a tuple in a list and display the student record with highest CGPA.
5. Take a list of numbers as input and calculates mean, median, mode for a list of numbers.

Hint: The **median** of a list of numbers is the "middle" value when the list is ordered from least to greatest. It represents the point where half the numbers in the list are smaller, and the other half are larger. For odd number of items, the median is the middle number and for even number of items

median is the average of the two middle numbers. The **mean** of a list of numbers, also called the average.

The **mode** of a list of numbers is the number that appears most frequently within the list. There can be one mode, multiple modes, or even no mode at all in a given list.

6. Create a program that sums up even numbers from a given list of integers. Use continue to skip odd numbers and break if the sum reaches 100 or more.
7. Given a list of names, write a program that continues to ask the user for a name until it finds a match in the list. If a match is found, break the loop and print the position of the name in the list.
8. Write a program to take a string as input and form a dictionary, that will have every character as key and frequency count of each character will be the value. See the sample input output.

Input: "Mississippi"

Output: {'M': 1, 'i': 4, 's': 4, 'p': 2}

9. Write a program to take a string with repeated characters as input and compress the string by converting repeated characters into tuples containing the character and the number of repetitions as a list (e.g., "aaabbbcc" becomes [('a', 3), ('b', 4), ('c', 2)]).
10. Create a dictionary containing the information about subject's code and number. Subject codes will be the key and score will be the associated value. See the dictionary as given below and perform the operations as listed from (a) – (f).

subjects = {"MATH101": 85, "PHYS202": 92, "CS303": 78}

- (a) Add the details of two other subjects as "CSE202": 67, "INT304": 56.
- (b) Check a subject code already exists or not, if already there, then display a message 'Already Added', else add the new subject.
- (c) Display the subject name that have highest marks
- (d) Calculate the average and total marks
- (e) Design the functionality to remove any subject from the dictionary
11. Write down a program and take two lists of integers from user as input and find the followings:
 - (a) Prepare a third list which have all the unique numbers after removing the duplicate items
 - (b) Prepare a third list having elements that are present in both lists.
 - (c) Prepare a list that will have the elements present in the first list but not in the second
 - (d) Prepare a list that have elements that are in either list but not in both.
 - (e) Print True, if the list has no common elements, else False.
12. Prepare a python script that will take two sets as input from the user and display a dictionary containing the following set operations:
 - (a) Union of set1 and set2.
 - (b) Intersection of set1 and set2.
 - (c) Difference of set1 minus set2.
 - (d) Symmetric difference of set1 and set2.

The dictionary should have keys "union", "intersection", "difference", and "symmetric_difference".

13. Write a Python script that takes two dictionaries as input and returns a new dictionary containing all the key-value pairs from both dictionaries. If there are overlapping keys, the values from dict2 should overwrite those from dict1.

14. Create a python script that takes a dictionary as input and returns a new dictionary where the keys and values are swapped. The input dictionary should be assumed to have unique values.

*****END OF CHAPTER*****

8 FUNCTIONS

You may be familiar with the mathematical concept of a function. A function is a relationship or mapping between one or more inputs and a set of outputs. In mathematics, a function is typically represented like this:

$$z = f(x, y)$$

Here f is a function that operates on the inputs x and y . The output of the function is z . However, programming functions are much more generalized and versatile than this mathematical definition. In fact, appropriate function definition and use is so critical to proper software development that virtually all modern programming languages support both built-in and user-defined functions.

In programming, a function is a self-contained block of code that encapsulates a specific task or related group of tasks. Previously we have been introduced to some of the built-in functions provided by Python such as `len()` , `id()`. Both these functions take one argument and `len()` returns the length of the object and `id()` returns the object's unique integer identifier.

```
>>> msg = "Hello, All!"
>>> numbers = list(range(1,10))
>>> print(msg, 'contains', len(msg), 'characters')
Hello, All! contains 11 characters
>>> print(numbers, 'contains', len(numbers), 'numbers')
[1, 2, 3, 4, 5, 6, 7, 8, 9] contains 9 numbers
```

The `len()` function returns the number of items in a given sequence. At first, we have passed `msg` as input to `len()`, returns the number of characters in “Hello, All!”. Later a list is passed as an argument and it returns the number of items in the list `numbers` and `id()` returns the unique identification number of the specified object.

```
>>> id(msg)
1815290128560
>>> id(numbers)
1815290131776
```

Each of these built-in functions performs a specific task. The code that performs the specified task is defined somewhere, but we don't need to think about the working of the code. Only we need to know about the interface of the function which may or may not have the following: (a) one or many arguments, it takes as input and (b) one or many values it returns.

In the examples you have seen that, we have passed the appropriate arguments to `id()` and `len()`. After the successful execution of a function call both functions return a value. Once we call a function, program execution goes off to the designated body of the code and does its useful thing. When the function body is finished, program execution returns to the code from where it was left off. Let see the pictorial view of a function call:

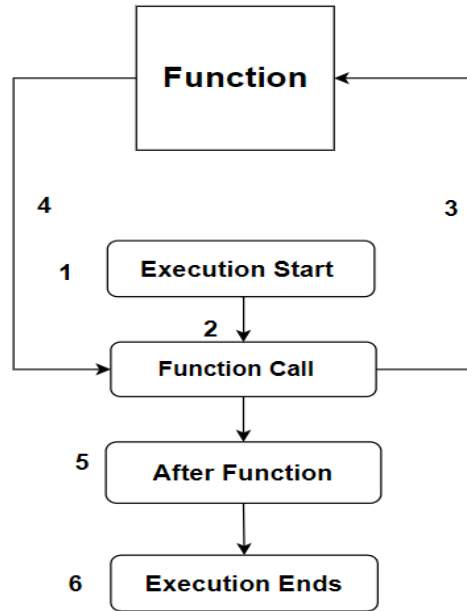


Figure 8.1: Flow of a program execution with a function call

In Fig. 8.1, we have depicted a flow of program execution and shown in a sequence of numbers from 1 – 6. The box written with Function caption represents a definition of the function and program execution starts from sequence number 1. At this point your script file can have any valid python statements, such as declaration or assignment. Thereafter at sequence number 2 the function calls get executed. Due to this, the program control has jumped (mentioned as number 3) inside the function, to execute the line of statements.

After the successful execution of the function, program control returns to the same point (represented as sequence number 2) of the script, from where the control was jumped. While the program control is returning, it can have some return value as well. Thereafter, program execution will continue till the execution of the last line of code. When you define your own Python function, it works just the same. From somewhere in your code, you'll call your Python function and program execution will transfer to the body of the code that makes up the function.

When the function is finished, execution returns to the location where the function was called. Depending on how you designed the function's interface, data may be passed in when the function is got called and return values may be passed back when it finishes.

8.1 Importance of Python Function

Virtually all programming languages support a form of user-defined functions, although they aren't always called functions. In other languages, you may see them referred to as, (a) subroutines; (b)

procedures; (c) methods; and (d) subprograms. So, why bother defining functions? There are several very good reasons. Let's go over a few now.

8.1.1 Abstraction and Reusability

Suppose you write some code that does something useful. As you continue development, you find that the task performed by that code is one you need often, in many different locations within your application. What should you do? Well, you could just replicate the code repeatedly, using your editor's copy-and-paste capability.

Remember it looks like a reasonable solution, but later the maintenance of the code will be very difficult. Later you decided that the code requires some modification. You'll either find something wrong with it that needs to be fixed, or you'll want to enhance it in some way. If copies of the code are scattered all over your application, then you'll need to make the necessary changes in every location.

So, the solution to the above problem is to define a Python function that performs the task. Anywhere in your application that you need to accomplish the task, you simply call the function. Down the line, if you decide to change how it works, then you only need to change the code in one location (the box written with Function caption in Figure 8.1), which is the place where the function is defined. The changes will automatically be picked up anywhere the function is called. The abstraction of functionality into a function definition is an example of the "Don't Repeat Yourself (DRY) Principle" of software development. This is arguably the strongest motivation for using functions.

8.1.2 Modularity

Functions allow complex processes to be broken up into smaller steps. Imagine, for example, that you have a program that reads in a file, processes the file contents, and then writes an output file. Your code could be look like this:

```

1  # Main Program
2  # Opening a file for reading
3  statements1
4  statements2
5  statements3
6
7  # Processing file content
8  statements1
9  statements2
10 statements3
11
12 # Updating file
13 statements1
14 statements2
15 statements3

```

Figure 8.2: Putting Everything in a Main Program

In Fig. 8.2, we have listed three different segments under the main program and have put all the code segments together in a long sequence. Further, if we consider the whitespace and comments to help

organize the code, the length of the script will increase. However, if the code were to get much lengthier, it would become very difficult to manage the code. Instead of this we can use the modular approach to separate the code segments using functions and later we can call the functions accordingly. A simple modular approach of python script is shown in Fig. 8.3.

```

1  # Opening a file for reading
2  ~def fileRead():
3      statements1
4      statements2
5  # Processing file content
6  ~def fileProcess():
7      statements1
8      statements2
9  # Updating file
10 ~def fileWrite():
11     statements1
12     statements2
13
14 # Main Program
15 fileRead()
16 fileProcess()
17 fileWrite()

```

Figure 8.3: A Sample of Modular approach using Python function

Instead of all the code being put together, it's broken out into separate functions, each of which focuses on a specific task. Those tasks are reading, processing, and writing. The main program now simply needs to call each of these in turn. Breaking a large task into smaller, bite-sized sub-tasks helps make the large task easier to think about and manage. As programs become more complicated, it becomes increasingly beneficial to modularize them in this way.

8.1.3 Namespace Separation

A namespace is a region of a program in which identifiers have meaning. As you'll see below, when a Python function is called, a new namespace is created for that function, one that is distinct from all other namespaces that already exist. The practical upshot of this is that variables can be defined and used within a Python function even if they have the same name as variables defined in other functions or in the main program. In these cases, there will be no confusion or interference because they're kept in separate namespaces.

This means that when you write code within a function, you can use variable names and identifiers without worrying about whether they're already used elsewhere outside the function. This helps minimize errors in code considerably. Till this point of study, we have tried to make you understand these two questions: (a) What function can have or not? (b) Why are python functions important?

In the next part of this Chapter, we will introduce various techniques related to Python function, though we can achieve better code reusability and maintenance of the code will be easy.

8.2 Define and call a function

A usual syntax to defining a function in Python is given below and each component are discussed in Table 8.1:

```
>>> def function_name(parameters):
...     statement(s)
```

Table 8.1: Details of each component of a Python function

Component	Meaning
Def	The keyword that informs Python that a function is being defined
function_name	A valid python identifier name
parameters	An optional, comma-separated list of parameters that may be passed to the function
:	Punctuation that denotes the end of the Python function header (the name and parameter list)
statement(s)	A block of valid Python statements

The **statement(s)**, is called the body of the function. The body is a block of statements that will be executed when the function is called. The body of a function is defined by indentation just like other control structures such as: an **if** or **while** statement. Once the function definition is completed, we need to call the function so that code associated within the function gets executed. Consider the syntax given below to call a function:

```
>>> function_name(arguments)
```

arguments are the values passed into the function. They correspond to the **parameters** in the function definition. You can define a function that doesn't take any arguments, but the parentheses are still required after a function name either in definition or a function call, even if they're empty. Let us start with a basic example to understand the basic concept of python function:

```
1 def fun():
2     msg = 'inside fun'
3     print(msg)
4
5 print('Start of Program')
6 fun()
7 print('End of Program')
```

```
Start of Program
inside fun
End of Program
```

Details of the above sample code are discussed below:

- Line 1: uses the **def** keyword to indicate that a function is being defined. Execution of the **def** statement creates the definition for the function **fun()**. All the following lines that are indented (lines 2 to 3) become part of the body of **fun()** and are stored as its definition, but they aren't executed yet.

- Line 4: a whitespace between the function definition and the first line of the main program just improves the code readability. This whitespace segregates the function definition from the main script.
- Line 5: is the first statement that isn't indented because it isn't a part of the definition of `fun()`. It's the start of the main program and when the executes, this statement is executed first.
- Line 6: is a call to `fun()`. Note that empty parentheses are always required in both a function definition and a function call, even when there are no parameters or arguments. Execution proceeds to `fun()` and then statements in the body of `fun()` are executed. A sequence of program execution is depicted in Figure 8.1.
- Line 7: is the next line to execute once the body of `fun()` has finished. Execution returns to this `print()` statement.

8.2.1 Defining an empty function

Sometimes, you may want to define an empty function that does nothing. This is referred to as a stub, which is usually a temporary placeholder for a Python function that will be fully implemented later. Just as a block in a control structure can't be empty, neither can the body of a function. To define a stub function, use the `pass` statement, as follows:

```
1 def fun():
2     pass
3
4 print('Start of Program')
5 fun()
6 print('End of Program')
```

```
Start of Program
End of Program
```

We can see that calling an empty function is syntactically correct but doesn't add any value to the output.

8.3 Types of Function Arguments

So far, we have discussed how to define a function and call a function but the functions we have defined haven't taken any arguments. That can sometimes be useful, and you'll occasionally write such functions. More often, though, you'll want to pass data into a function so that its behavior can vary from one invocation to the next. Let's see how to do that.

8.3.1 Positional Arguments

The most straightforward way to pass arguments to a Python function is with positional arguments (also called required arguments). In the function definition, you can specify a list of comma-separated parameters inside the parentheses, as shown below:

```
1 #defining a function with two parameters
2 def fun(name, age):
3     print(f'{name} is {age} years old')
4
5 #Calling the fun
```

```
6 fun('Tridib', 7)
7 fun('Goutam', 37)
```

Tridib is 7 years old
Goutam is 37 years old

A function `fun(name, age)` is defined with two parameters `name` and `age`. These parameters behave like variables that are defined locally to the function. When the function is called `fun('Tridib', 7)` at line 6, are bound to the parameters in order, as though by variable assignment and `'Rahul'` and `45` is assigned to `name` and `age` in order. While calling a function following points need to take care:

- The number of parameters in the function definition should match the numbers of arguments in the function call. Now let's try to call the above function with one argument name, as follows:

```
>>>fun('Rahul')
Traceback (most recent call last):
  File "<pyshell#8>", line 1, in <module>
    fun('Rahul')
TypeError: fun() missing 1 required positional argument: 'age'
```

Interpreter raised a `TypeError` as one positional argument `age` is missing. Even though, if we pass more number arguments than the required number, interpreter will raise a `TypeError`. See the function call given below with the error:

```
>>>fun('Rahul', 25, 'Male')
Traceback (most recent call last):
  File "<pyshell#0>", line 1, in <module>
    fun('Rahul', 25, 'Male')
TypeError: fun() takes 2 positional arguments but 3 were given
```

- We need to take care of the order of arguments while calling a function. The order should match the order of the parameters in the function definition. There's nothing to stop you from specifying positional arguments out of order, of course:

```
>>>fun(35, 'Rahul')
35 is Rahul years old
```

The function may even still run, but due to change of order in the arguments we did not get the desired output because 35 is assigned to `name` and Rahul is assigned to `age`. It's the responsibility of the programmer who defines the function to document what the appropriate arguments should be, and it's the responsibility of the user of the function to be aware of that information and abide by it.

Positional arguments are conceptually straightforward to use, but they're not very forgiving. You must specify the same number of arguments in the function call as there are parameters in the definition, and in the same order. Later in this chapter, you will find some argument-passing techniques that relax these restrictions.

8.3.2 Keyword Arguments

When you're calling a function, you can specify arguments in the form `keyword = value`. In this case, each `keyword` must match a parameter name present in the function definition. For example, the

previously defined function `fun()` may be called with keyword arguments as follows:

```
>>>fun(name = 'Rahul', age = 45)
```

Rahul is 45 years old

While we are calling the function with keyword order of parameter is not important anymore. Consider the next function call given below:

```
>>>fun(age = 45, name = 'Rahul')
```

Rahul is 45 years old

Now see the above function call, we didn't follow the order of the parameters. Using keyword arguments lifts the restriction on argument order. Each keyword argument explicitly designates a specific parameter by name, so you can specify them in any order and Python will still know which argument goes with which parameter. Referencing a keyword that doesn't match any of the declared parameters generates a **TypeError**. See the below function call:

```
>>>fun(nam = 'Rahul', age = 45)
Traceback (most recent call last):
  File "<pysHELL#10>", line 1, in <module>
    fun(nam = 'Rahul', age = 45)
TypeError: fun() got an unexpected keyword argument 'nam'
```

While calling the function we have put a wrong keyword name that does not match the function parameter. Instead of the given parameter **name**, we have used **nam** maybe knowingly or unknowingly and thus keywords didn't match during the function call and raised a **TypeError**. Like positional arguments, keyword arguments must match the number of parameters, or else raise a **TypeError**.

8.3.3 Default Parameters

If a parameter specified in a Python function definition has the form **name = value**, then **value** becomes a default value for that parameter. Parameters defined this way are referred to as default or optional parameters. An example of a function definition with default parameters is shown below:

```
1 def fun(a=3, b=4):
2     print(f'a = {a}, b = {b}')
```

In the function definition values 3 and 4 are assigned to the parameter 'a' and 'b'. During the function call, if we forget to pass value to any parameters, then the default value is considered. See the various form of function call `fun()`:

```
>>>fun()
>>>fun(10)
>>>fun(10, 20)
```

All these function calls are valid, and we will get the desired output. The first function call `fun()`, doesn't take any argument and the default values 3 and 4 is assumed for the parameters 'a' and 'b'. The second call `fun(10)`, has one argument, and values are assigned according to the position in

function definition from left to right. It means the default value for the parameter **'a'** is overwritten by 10 and default value for the parameter **'b'** will be retained. In the third call **fun(10,20)**, we have passed all the arguments and all the default values for the parameters are overwritten, means 10 and 20 is assigned to **'a'** and **'b'**. The output of all the function calls is given below:

```
a = 3, b = 4
a = 10, b = 4
a = 10, b = 20
```

The second call **fun(10)** is assigned value 10 to the parameter **a**, what if, we want to assign the passing argument to parameter **'b'** not **'a'**. To achieve this, we can mention the keyword at the time of function calling, provided all the parameters assigned with default value. See the function call given below:

```
>>>fun(b=9)
a = 3, b = 9
```

In the above function call default value for **'a'** is assumed and 9 is assigned to **'b'**. Now the question is can we mix default parameters with non-default parameters in a function definition. Let's us consider the function definitions given below:

```
def fun1(a, b = 4):
    print(f'a = {a}, b = {b}')

def fun1(a = 3, b):
    print(f'a = {a}, b = {b}')
```

In both the functions **fun1()**, **fun2()** definitions we have assigned only one default parameter. In **fun1()**, default argument follows non-default argument and in **fun2()**, non-default argument follows default argument. Now let's execute the above function and see if both the definitions are accepted by Python or not.

On execution you will get a syntax error, specifying that “non-default argument follows default argument” is pointed to the function definition **fun2()**. Means out of both the function definitions only **fun1()** is valid in Python. We must keep in mind that a function where default parameters are used must be assigned from right to left or for all the parameters. That's why definition for **fun1()** doesn't point to any SyntaxError.

This discussion will be incomplete if we don't use any mutable type as default parameter. Things can get weird if you specify a default parameter value that is a mutable object, specifically an empty list. Consider this Python function definition:

```
1 def fun(my_list = []):
2     my_list.append('e')
3     print(id(my_list), my_list)
4
5 fun()
6 fun(['a', 'b', 'c'])
7 fun()
```

First call **fun()**, takes a single list parameter, appends the string **e** to the end of the list, and prints the

id and the items of the list. Let's check the output:

```
1981250783552 ['e']
1981250785024 ['a', 'b', 'c', 'e']
1981250783552 ['e', 'e']
```

If we didn't pass any argument the default parameter is assumed, and an empty list is assigned to the parameter `my_list`. But between the consecutive function calls if we didn't pass any argument then the same list is referred. So, after the 3rd call, we are getting `['e', 'e']` as output, and the `id` of `my_list` is the same as first function call. While in the second call we have pass a list `['a', 'b', 'c']` as argument and in this case default value is overwritten and new `id` is referred.

In Python, default parameter values are defined only once when the function is defined (i.e., when the `def` statements are executed). The default value isn't re-defined each time the function is called. Thus, each time you call `fun()` without a parameter, you're performing `.append()` on the same list.

To stop these duplicate items due to the default list we can assign a default value `None` to the parameter. Later inside the function, we can check if the value of the parameter is `None` or not, and based on that we can create the list and then append the value. See the code solution given below:

```
1 def fun(my_list = None):
2     if my_list is None:
3         my_list = []
4     my_list.append('e')
5     print(id(my_list), my_list)
6
7 fun()
8 fun(['a', 'b', 'c'])
9 fun()
```

```
1471994294208 ['e']
1471994294208 ['a', 'b', 'c', 'e']
1471994294208 ['e']
```

In this way, we can ensure that an empty list will be created always if we didn't pass any argument while calling the function `fun()`.

8.4 Passing arguments by reference

In this section we have first highlighted two different approaches to passing an argument to a function. To discuss this, we have taken a reference from another programming language i.e., C and later in this Section, we have highlighted how Python passes the arguments to a function. In any programming language, there are two common ways to pass arguments to a function: (a) pass by value; and (b) pass by reference

In pass by value a copy of the argument is passed to a function, and changes made inside the function to the parameters aren't reflected outside of the function. Let us understand how arguments are passed by value to a function with a program written in C programming language depicted in Fig. 8.4.

```

1 #include<stdio.h>
2 void fun(int a){
3     a = 40;
4 }
5 int main(){
6     int a = 20;
7     fun(a);
8     printf("%d",a);
9 }

```

Figure 8.4: Sample C Program for passing arguments by Value

The function `void fun(int a)` (at line 2) is defined and get called inside another function `main()`. Like python, in C programming also, we need to call the function to execute the code written inside the function. In C programming, program execution started from the `main()` function (means every program written in C must have one `main()` function).

In the above sample code at first a variable '`a`' of type `int` is declared and initialized to 20 (at line 6). Thereafter '`a`' is passed as argument to the function `fun(a)` (at line 7). While calling the function the value of '`a`' is passed and assigned to the parameter '`a`' (listed in the function header at line 2) (a copy of argument '`a`'). Later inside the function, the `a`'s value is changed to 40 (at line 3) and program control is returned to the `main()` function.

In call by value, all the changes have been made to the copy of '`a`' which is created inside the `fun()`. So, the changes aren't reflected outside of the function `fun()` and the value of '`a`' remains unchanged. So, the `printf("%d", a)` statement prints the value of '`a`' as 20.

In call by reference, reference represents the address of a memory block. So instead of passing the value of the argument, we pass the address of the argument and changes made inside the function will be done over the actual argument and reflected outside of the function. In Fig. 8.5 the flow of call by value (in Fig. 8.5(a)) and call by reference (in Fig. 8.5(b)) is depicted.

While passing the argument in the call by reference we have passed the address of a `fun(&a)` and the changes made by the statement in the header these two arguments passing approaches and later we have re-written the above program and passed the argument by reference.

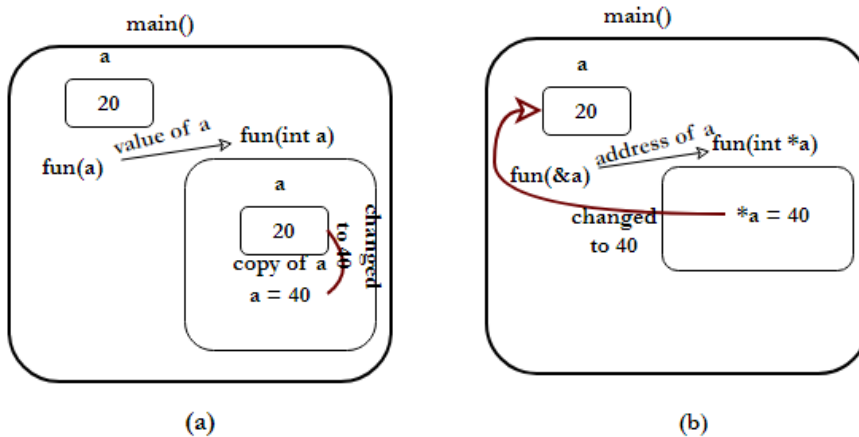


Figure 8.5: Passing argument (a) Call-by-value and (b) Call-by-reference

```

1 #include<stdio.h>
2 void fun(int *a){
3     *a = 40;
4 }
5 int main(){
6     int a = 20;
7     fun(&a);
8     printf("%d",a);
9 }

```

Figure 8.6: Sample C Program for passing arguments by Reference

See the above code at line 7 where the function `fun(int *a)` gets called and instead of the value of `'a'`, address of `(&a)` is passed. To hold the address of an argument, a pointer parameter type `int *a` is used in the function header at line 2. To access the memory block of `'a'`, instead of `a = 40`, `*a = 40` is used and the value `'a'` is changed to 40.

8.5 How does python pass the arguments?

In python arguments neither pass by value or reference exactly what it means in C programming. That's because in Python a reference doesn't mean quite the same thing as C programming. In Python, every piece of data is an object. So, a reference points to an object, not a specific memory location. That means the assignment isn't interpreted the same way in Python as it is in C. Consider the following statements in C:

```

int a = 20;
a = 40;

```

The first statement declares a variable `'a'` and refers to a memory block and later value 20 is kept on it. The next statement overwrites 20 and puts 40 on it. The scenario in python is quite different. Consider the statements given below:

```

>>>a = 20
>>>a = 40

```

The first statement causes `'a'` to point to an object whose value is 20. The next statement reassigns `'a'` to a new object whose value is 40. Stated another way, the second assignment rebinds `'a'` to a different object with value 40. In Python, when you pass an argument to a function, a similar rebinding occurs. Consider this example:

```

1 def fun(x):
2     x = 10
3
4 a = 5
5 fun(a)
6 print(a)

```

In this program, the statement `a = 5` (in line 4) creates a reference named `'a'` bound to an object whose value is 5. `fun()` is then called on line 5, with `'a'` as an argument. When the function `f()` executes first, a new reference called `'x'` is created, which initially points to the same 5 object as `'a'` does. However, when the statement `x = 10` at line 2 is executed, function `fun()` rebinds `'x'` to a new

object whose value is 10. The two references, `'a'` and `'x'`, are uncoupled from one another. Nothing else that `fun()` does will affect `'a'`, and when `fun()` terminates, `'a'` will still point to the object 5, as it did prior to the function call. We can verify this using `id()` inside and outside of function `fun()`. See the code given below:

```
1 def fun(x):
2     print(f'id(x) = {id(x)}')
3     x = 10
4     print(f'id(x) = {id(x)}')
5
6 a = 5
7 print('Before function call...')
8 print(f'id(a) = {id(a)}')
9 fun(a)
10 print('After function call...')
11 print(f'id(a) = {id(a)}')
```

Before function call...

id(a) = 2776095981936

id(x) = 2776095981936

id(x) = 2776095982096

After function call...

id(a) = 2776095981936

The above code is a slightly modified version of the previous function and along with the value of `'a'`, we are also checked the `id`. At first, `'a'` is bind with the object 5 in line 6, and the `id(a)` before and after the function call `fun(a)` didn't change, even though we have assigned a new object to `x` inside the function `fun()` at line 3.

There is another point we need to check that, the `id(x)` at line 2 and 4 gets a different value. When we check `id(x)` at line 2, it is found that `x` inside `f()` and `a` outside of `f()`, represent the same `id`. But once we have changed the object pointed by `x` at line 3, `x's id` get modified.

We need to remember that, even though python passed arguments to a function as reference (not like C programming), once we have assigned a new object that changes will not reflect outside of the function. Argument passing in Python is somewhat of a hybrid between pass-by-value and pass-by-reference. What gets passed to the function is a reference to an object, but the reference is passed by value.

Note: Python's argument-passing mechanism has been called pass-by-assignment. This is because parameter names are bound to objects on function entries in Python, and assignment is also the process of binding a name to an object. You may also see the terms pass-by-object, pass-by-object-reference, or pass-by-sharing. The key takeaway here is that a Python function can't change the value of an argument by reassigning the corresponding parameter to something else. The following example demonstrates this:

```
1 def fun(item):
2     item = 34
3
4 for i in (2, 'Hello', [2,3,4], {'a':2, 'b':3}, {3,4}, (3,5,6)):
5     fun(i)
6     print(i)
```

2
Hello
[2, 3, 4]

```
{'a': 2, 'b': 3}
{3, 4}
(3, 5, 6)
```

Here we have passed objects of type `int`, `str`, `list`, `dict`, `set` and `tuple` to `fun()` as an argument. The function `fun()` tries to assign an `int` object `34` and once the program control comes back to the calling environment the value of the argument `i` remains unchanged. Does that mean a Python function can never modify its arguments at all? No, that isn't the case! Watch what happens here:

```
1 def fun(item):
2     if type(item) is list:
3         item.append(30) #appending an item to the list
4     elif type(item) is dict:
5         item.update({'c':9}) #updating the dictionary
6     elif type(item) is set:
7         item.add(40) #adding an item to the set
8
9 for i in ([2,3,4], {'a':2, 'b':3}, {3,4}):
10     fun(i)
11     print(i)
```

```
[2, 3, 4, 30]
{'a': 2, 'b': 3, 'c': 9}
{40, 3, 4}
```

Here we have passed mutable type objects as arguments to the function `fun()` and instead of assigning a new object to the parameter `item`, we have updated the `item` for the type `list`, `dict` and `set`. As shown in the output, we have found that once the program control returns to the calling environment changes are made inside the function that reflect outside of the function for mutable types. If the argument is a number or a string, the argument is not affected, regardless of the changes made to the parameter inside the function.

8.6 The *return* Statement

Sometimes it is necessary that the calling environment expects the calling function to return a value. What's a Python function to do then? After all, in many cases, if a function doesn't cause some change in the calling environment, then there isn't much point in calling it at all. How should a function affect its caller? Well, one possibility is to use function return values. A return statement in a Python function serves two purposes:

- It immediately terminates the function and passes execution control back to the caller.
- It provides a mechanism by which the function can pass data back to the caller.

8.6.1 Exiting from a Function

Within a function, a `return` statement causes an immediate exit from the Python function and transfers the execution back to the caller. Consider the example given below:

```
1 def fun():
2     print('start of fun()')
3     print('end of fun()')
```

```

4     return
5
6 fun()
7 print('after the fun')

```

start of fun()

end of fun()

after the fun

In this example there is no need for a **return** statement, which returns the program control back to the calling environment. Even if we didn't use the **return** statement, we are also getting the same output. It is not necessary that **return** statement should be last statement of the function, even we can place it anywhere inside the function, and even multiple times. Consider the sample code given below:

```

1 def fun(num):
2     if num > 0:
3         return
4     elif num < 0:
5         return
6     print('End of function')
7
8 fun(20)
9 fun(-1)
10 fun(0)

```

End of function

For the first two function calls at line 8 and 9, **print** statement at line 6 remain unreachable due to the pre-execution of the **return** statement at line 3, and 5. The third call **fun(0)** doesn't satisfy any of the condition of the function body written at line 2 and 4 and due to this none of the **return** statement is executed, so the **print** statement gets executed. Due to the misplaced of **return** statement sometimes it is also possible that code written inside the function body never executed. Consider the sample code given below:

```

1 def fun(num):
2     if num % 2 == 0:
3         return
4     elif num == 0:
5         return
6     else:
7         return
8     print('End of function')
9
10 fun(20)
11 fun(0)
12 fun(11)

```

The first two function call at line 10, 11 passed an argument to the function and subsequently the conditions written at line 2 and 4 become **True**. Due to this **return** statements written at line 3 and 5 get executed. Like this, third call **fun(11)** doesn't satisfy any of the conditions and the **else** clause having a **return** statement gets executed. In this example, multiple **return** statements are used and get executed before reaching the last statement at line 8 for all the function calls and the **print** statement is never executed.

Note: Once the function reached any of the **return** statement, lines written after it were never

considered for the execution. This sort of paradigm can be useful for error checking in a function. You can check several error conditions at the start of the function, with return statements that bail out if there's a problem.

```
def fun(optional_arguments):
    if error_cond1:
        return
    elif error_cond2:
        return
    elif error_cond3:
        return
    #Actual body of the function goes here
```

If none of the error conditions are encountered, then the function can proceed with its normal processing.

8.6.2 Returning values to the calling environment

The actual use of return statement is not only returning the program control to the calling environment, instead return statement is used to return some values to it. Let's see a better example to understand the use of **return**:

```
1 def fun():
2     return "I'm ok!"
3
4 msg = fun()
5 print(msg)
```

I'm ok!

The function call **fun()** at line 4 is considered as an expression and the return value from the function body **I'm ok!** is assigned to the variable **msg**. Using this mechanism, we can build communication between a calling environment and a function. In python, a function can return any type of object to the calling environment. A function call can be used syntactically anywhere in the calling environment that makes sense for the type of object the function returns.

```
1 def add(a, b): # returns a number
2     return a + b
3
4 def greater(a, b): # returns a bool
5     return True if a > b else False
6
7 def swap(a, b): # returns a list
8     a, b = b, a
9     return [a, b]
10
11 def my_dict(a, b): # returns a dictionary
12     return {'sum': a+b, 'sub': a-b, 'prod': a*b}
13
14 print(add(3, 4))
15 print(greater(10, 20))
16 swap_values = swap(3,4)
17 print(swap_values[0], swap_values[1])
18 print(my_dict(3,4))
```

The above scripts did the followings:

- Function **add()** returns the addition of two numbers of any type.
- The **greater()** function returns a Boolean type as **True** if the first parameter is larger than the second else return **False**.
- Function **swap()** interchanges the values of two parameters and returns them as a list.
- Function **my_dict()** returns a dictionary object containing the result of basic mathematical operation of two parameters.

The return type of a function also depends on how values are returned from a function or how the calling environment accepts them. Consider the example given below:

```
1 def swap(a, b): # returns multiple values
2     a, b = b, a
3     return a, b
4
5 a, b = 10, 20
6 values = swap(a, b) # received as tuple
7 print(type(values))
8 a, b = swap(a, b) # received as int object
9 print(type(a), type(b))
10 print(type(list(swap(a, b)))) # received as list
```

```
<class 'tuple'>
<class 'int'> <class 'int'>
<class 'list'>
```

The **swap()** function called thrice at line 6, 8, and 10, where function returns the values of **a** and **b** as integer after swapping. The first call at line 6, receives the return values in one variable, as tuple and this mechanism is called packing. The second call at line 8 uses two variables to hold the return values, where the number of variables is same as number of return values returns from the function. The third call received the return values as a list, as **swap()** get called inside **list()** function. In Python, it is also possible for a return statement did return nothing, instead returns the special Python value **None**, like the **pass** statement, as shown below:

```
def fun1():
    pass

def fun2():
    return

def fun3():
    print()

print(fun1(), fun2(), fun3())
```

8.6.3 Returning a function

In some scenarios it becomes compulsory to design a function whose behavior may depend on the user input that passing as argument. To handle such a scenario, we need to create functions with dynamic behavior. This can be done by nesting one function inside another, meaning outer function return an inner function. Like any other data type Python functions can be treated as first-class objects, that allows functions to be returned from other functions. The working of nesting function is summarized below:

- Define an outer function that creates an inner function. The inner function can access variables from the outer function's scope, even after the outer function has finished executing. This is because the inner function remembers the state of the outer function at the time it was created.
- The outer function returns the inner function.

Let's take one scenario where you need to create functions whose behavior depends on external factors or user input. Returning a function allows you to capture this dynamic behavior and create functions tailored to specific situations, such as unit converter.

```

1 def get_converter(unit_from):
2     """
3     This function takes a unit type and returns a function that converts
4     temperatures from that unit to Celsius.
5     """
6     def convert_to_celsius(value):
7         """
8         This function converts a temperature value to Celsius based on the unit_from.
9         """
10        if unit_from == "fahrenheit":
11            return (value - 32) * 5 / 9
12        elif unit_from == "celsius":
13            return value
14        else:
15            raise ValueError("Unsupported unit type:", unit_from)
16    return convert_to_celsius
17
18 # Get a converter function for Fahrenheit to Celsius
19 celsius_converter = get_converter("fahrenheit")
20
21 # Use the converter function
22 converted_value = celsius_converter(77)
23 print(f"{converted_value:.2f} degrees Celsius")

```

Here, **get_converter** creates a function (**convert_to_celsius**) specific to the provided unit (**fahrenheit**). This allows for code reusability and flexibility in handling different unit conversions. Explanation of the code given below:

- Line 1: Defines a function named **get_converter** that takes a single argument **unit_from**.
- Line 6: Defines another function named **convert_to_celsius** inside the **get_converter** function. This is the nested function.
- Lines 10-11: If **unit_from** is "fahrenheit", it converts from Fahrenheit to Celsius using the formula.
- Lines 12-13: If **unit_from** is "celsius", no conversion is needed, so the value is returned directly.
- Lines 14-15: If **unit_from** is not supported, it raises a **ValueError** exception.
- Line 16: The return statement in the inner function (**convert_to_celsius**) returns the calculated value or the original value depending on the unit. This return statement within the inner function is crucial. It's what makes **get_converter** a function that returns a function.
- Line 18: Calls the **get_converter** function with the argument "fahrenheit". This execution creates a new instance of the inner function (**convert_to_celsius**) specifically for converting from Fahrenheit.
- Line 19: Assigns the returned function (the conversion function for Fahrenheit to Celsius) to the variable **celsius_converter**.

- Line 21: Calls the `celsius_converter` function (which is the inner `convert_to_celsius` function) with the value 77 (presumably Fahrenheit).

In summary, the `get_converter` function acts as a factory that creates conversion functions based on the provided unit type. This allows for code reusability and avoids writing separate conversion functions for each unit.

8.7 Variable-Length Argument List

In some cases, when you're defining a function, you may not know beforehand how many arguments you'll want it to take. Suppose, for example, that you want to write a Python function that computes the average of several values. You could start with something like this:

```
1 def avg(a, b, c):
2     return (a + b + c)/3
```

This function will work fine if we call `avg()` with three arguments only. With this flavor of `avg()` passing more or lesser number of arguments will raise `TypeError` as we have seen in case of positional arguments. Further to provide some flexibility to the caller we can assign default values to the arguments and make some of them optional while calling. The sample code listed below highlights all the valid function calls from line 4 – 8.

```
1 def avg(a, b = 0, c = 0, d = 0, e = 0):
2     return (a + b + c + d + e)/5
3
4 print(avg(1))
5 print(avg(1,2))
6 print(avg(1,2,3))
7 print(avg(1,2,3,4))
8 print(avg(1,2,3,4,5))
```

```
0.2
0.6
1.2
2.0
3.0
```

But this approach still suffers from a couple of problems. For starters, it still only allows up to five arguments, not an arbitrary number. Worse yet, there's no way to distinguish between the arguments that were specified and those that were allowed to default. The function has no way to know how many arguments were passed, so it doesn't know what to divide by and always it gets divided by 5.

Without knowing the number of parameters passed from the calling environment it is difficult to return the correct average value by the function. To know the exact number of arguments, a calling environment can pass a list of items as argument and function can use `len()` to know the number of items, to produce the exact average value.

```
1 def avg(nums):
2     total = 0
3     for item in nums:
4         total += item
5     return total / len(nums)
6
7 print(avg([1]))
```

```
8 print(avg([1,2]))
9 print(avg([1,2,3]))
```

```
1.0
1.5
2.0
```

At least this works. It allows an arbitrary number of values and produces a correct result. As a bonus, it works when the argument is a tuple as well. The drawback of this approach is that the user must be aware that to group the values into a list or tuple is probably not something the user of the function would expect, and it isn't very elegant. To make the code more user-friendly python provides a better approach to pass variable number of arguments to a function using following methods:

- Argument packing as tuple using an asterisk (*) with the parameter name
- Argument packing as **key = value** pair as dictionary using double asterisk (**) with parameter name

8.7.1 Argument packing using tuple

When a parameter name in a function definition is preceded by an asterisk (*), it indicates any number of arguments are packed in a tuple. Later inside the function those parameters can be used by the given name of the parameter. Consider the example given below:

```
1 def avg(*args):
2     total = 0
3     for item in args:
4         total += item
5     return total / len(args)
6
7 print(avg([1,2,3]))
8 print(avg([1,2,3,4,5,6]))
```

In the above example function **avg()** is rewritten using the tuple packing mechanism by preceding the parameter **args** by an asterisk (*) in the function header. While receiving, parameters are packed as a single object of type tuple and inside the function the **args** considered as tuple and while calling the function, the user no need to do anything extra to group the arguments either as list or tuple as we did for the last example.

8.7.2 Arguments packing using dictionary

This process is also known as keyword argument packing. In this method we can pass variable number of arguments in a **key = value** pair, as follows:

```
1 def fun(**kwargs):
2     for key, value in kwargs.items():
3         print(f'{key}: {value}', end = ' ')
4     print()
5
6 fun(a = 2)
7 fun(a = 2, b = 3)
8 fun(a = 2, b = 3, c = 5)
```

```
a: 2
```

```
a: 2 b: 3
a: 2 b: 3 c: 5
```

The double asterisk (******) is used with function parameter name to specify the dictionary packing. In the above code **fun()** get called three times with variable arguments as **key = value** pair. Preceding a parameter name in a Python function definition by double asterisk (******) indicates that the corresponding arguments, which are expected to be **key = value** pairs, should be packed into a dictionary. While packing the arguments to a dictionary the name **kwargs** is used, you can use any other name. Here **kwargs** represents keyword argument and from the output of the code it is clear about the type of parameter **kwargs** as dictionary.

8.8 Arguments unpacking

Like tuple and dictionary packing with the parameters, reverse one means unpacking is also allowed in python. Following ways, we can unpack an argument during the function call:

- unpacking an iterable object such as tuple, list and set using an asterisk (*) with the argument name
- unpacking a dictionary as keyword argument using double asterisk (**) with argument name

8.8.1 Argument tuple unpacking

We have seen how to pass variable number of arguments by packing them to a tuple. In python we can also unpack a tuple or any iterable object before passing it as an argument to a function. Consider the example given below:

```
1 def avg(a,b,c,d):
2     return (a+b+c+d)/4
3
4 args = (3,4,5,6) #packing within a tuple
5 print(avg(*args)) #unpacking before passing the arguments
6 args = [2,3,4,5] #packing within a list
7 print(avg(*args)) #unpacking before passing the arguments
```

An asterisk (*) is placed before the name of the argument to unpack an iterable object before passing it as argument. The function call at line 5 and 7 uses an asterisk (*) before the name of argument **args** and items of the object are unpacked and then pass it to the function **avg()** as positional arguments. While unpacking any iterable it must match the number of positional parameters in the function header, else it will raise a **TypeError**.

We can also combine the packing and unpacking mechanism while passing the arguments to a function. Let's write a more user-interactive code to take input from user to find the largest of all the items. Consider the example given below:

```
1 def find_largest(*args):
2     max = args[0]
3     for i in range(1, len(args)):
4         if args[i] > max:
5             max = args[i]
6     return max
7
8 args = eval(input('Enter comma-separated values:'))
```

```
9 print(f'largest of all: {find_largest(*args)}')
```

8.8.2 Argument Dictionary Unpacking

While unpacking a dictionary the double asterisk (******) is used with argument name at the time of function calling. While unpacking a dictionary key of a dictionary should match the name of the parameters of the function. For any mismatch between a key and parameter name python will be raised a `TypeError` as got an unexpected keyword argument.

```
1 def fun(a, b, c):
2     print(f'a = {a}, b = {b}, c = {c}')
3
4 my_dict = {'a':1, 'b':2, 'c':3}
5 fun(**my_dict)
```

8.9 Keyword-only argument

As of now we have used the character ***** for two purposes (i) using ***args** to implement the variable list of arguments as a tuple and (ii) using ****kwlist** to pass only keyword list arguments as a dictionary. Suppose there is some necessity to write a function that takes any variable number of strings and concatenates them by separating an optional character. Something like this:

```
>>> def joinstr(*args):
...     print('.'.join(args))
...
>>> joinstr('a','b','c')
a.b.c
```

As of now whatever the strings we are sending it is always concatenates with a period (.) only. What if you want to modify the function to accept an argument which accepts the character also which will be used for this purpose, as the solution given below:

```
>>> def joinstr(char, *args):
...     print(char.join(args))
...
>>> joinstr('#','a','b','c')
a#b#c
```

This function could solve our purpose, but there are a couple of undesirable things about this solution:

- How could someone know that the first argument serving a specific purpose while joining other variable number of strings?
- The parameter **char** is not optional, it must be passed always.

One of the ways we can satisfy the above requirements, by making the argument **char** as default argument, which makes it optional. Let's us try this:

```
>>> def joinstr(char='.', *args):
...     print(char.join(args))
...
>>> joinstr('a','b','c')
bac
```

Unfortunately, the solution does not work, because the first string argument 'a' is considered as the special character argument, which is used for joining. Now let's us try by specifying a keyword argument before the variable number of strings as follows:

```
>>> joinstr(char='#', 'a', 'b', 'c')
File "<stdin>", line 1
    joinstr(char='#', 'a', 'b', 'c')
                                ^
```

SyntaxError: positional argument follows keyword argument

This also does not work, and the solution leads to error as positional argument follows keyword argument. So, neither of the solutions solve our purpose. Let's see how Python can help us. Syntactically the solution is very simple. The first change is to allow the optional argument to appear after a ***args** argument in function definition.

```
>>> def joinstr(*args, char='.'):
...     print(char.join(args))
...
>>> joinstr('a', 'b', 'c', char='#')
a#b#c
```

Keyword-only arguments allow a Python function to take a variable number of arguments, followed by one or more additional options as keyword arguments. If you wanted to modify **joinstr()** so that a separator (**sep**) along with an **ends** character can optionally be specified as well, by adding an additional keyword-only argument:

```
>>> def joinstr(*args, sep = ' ', ends = '.'):
...     print(sep.join(args) + ends)
...
>>> joinstr('This', 'is', 'Python', '3.10', sep = '|', ends = '!')
This|is|Python|3.10!
```

The following points need to remember to work with keyword-only arguments: (a) If the keyword argument didn't pass at the time of calling then the default value will be considered. (b) If the default value didn't provide then passing the values for the keyword argument is compulsory. Not doing so will raise an error.

The second approach to implement the keyword-only argument is achieved by a syntactical change, which allow the argument name to be omitted for a variable list argument (varargs), as follows:

```
>>> def fun(a, b, *, karg = None):
...     pass
```

This approach is useful when we want to define a function is not allowed to take variable list arguments and take a keyword argument. Consider the example given below, which performs a designated operation over two arguments:

```
1 def calculator(a, b, ope='+'):
2     if ope == '+':
3         return a + b
4     elif ope == '-':
5         return a - b
6     elif ope == '*':
7         return a * b
```

```

8     elif ope == '/':
9         return a - b
10    else:
11        return None

```

For the above function the type of operation performed it depends on the **ope** parameter. If we want the **ope** argument should pass as keyword only, we can add an unnamed ***** parameter before the **ope**. In this case the **ope** argument become compulsory to pass as keyword-only or optional, because of the default value assigned to it. Now check the next three function calls:

```

>>> def calculator(a, b, *, ope='+'):
...     if ope == '+':
...         return a + b
...     elif ope == '-':
...         return a - b
...     elif ope == '*':
...         return a * b
...     elif ope == '/':
...         return a - b
...     else:
...         return None
...
>>> calculator(3,4, ope = '-')
-1
>>> calculator(3,4)
7
>>> calculator(3,4,'-')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: calculator() takes 2 positional arguments but 3 were given

```

First two calls work fine, but the third call pass the **ope** argument as positional which leads to an exception. Because it considered it as positional argument. What if you pass some extraneous positional argument with two required positional arguments and keyword-only argument, as follows:

```

>>> calculator(3,4,"no extra values",'-')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: calculator() takes 2 positional arguments but 4 were given

```

The **calculate()** function is not ready yet to take any extra positional argument that can be ignored by the function. For this we can make a dummy argument by using a name with the ***** and later ignored by the function this as follows:

```

>>> def calculator(a, b, *ignore, ope='+'):
...     print(ignore)
...     if ope == '+':
...         return a + b
...     elif ope == '-':
...         return a - b
...     elif ope == '*':
...         return a * b
...     elif ope == '/':
...         return a - b
...     else:

```

```

...     return None
...
...
>>> calculator(3,4,"no extra values",ope = '/')
('no extra values',)
-1

```

It takes only two positional arguments and any extra positional argument pass by the caller will be ignored and the respective operation is performed depends on the **ope** passed as keyword-only argument.

8.10 Positional-only Argument

Python 3.8 introduces positional-only arguments as compare with keyword-only argument. Means the corresponding arguments must be supplied positionally and can't be specified by keyword. Using a forward-slash (/) we can make some parameters as positional-only.

Any parameters to the left of the slash (/) must be specified positionally. For example, in the following function definition, **a** and **b** are positional-only parameters, but **c** may be specified by keyword:

```

>>> def fun(a, b, /, c):
...     print(a, b, c)
...
>>> fun(2, 3, c = 4) #c as keyword
2 3 4
>>> fun(2, 3, 4) #c as positional
2 3 4

```

Above two function calls don't test the actual meaning of **fun()**, which accepts **a** and **b** as positional only. Let's try to send any of these two parameters as keyword and trying to do so leads to a **TypeError** as **fun()** expecting **a** and **b** as positional but pass as keyword.

```

>>> fun(a = 2, b = 3, c = 4)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: fun() got some positional-only arguments passed as keyword arguments: 'a, b'

```

The positional-only arguments can be mixed with keyword-only arguments also, if it is necessary as follows:

```

>>> def fun(a, b, /, *, c):
...     print(a, b, c)
...
>>> fun(2, 3, c = 4) #c must be pass keyword only
2 3 4

```

In the above function definition, an unnamed argument ***** is used after the forward slash (/) and it makes anything passed after the positional argument **b** should be passed as keyword only and not doing so will raised a **TypeError**.

8.11 The docstring

A docstring is a string literal that occurs as the first statement in a module, function, class, or method definition. Such a docstring becomes the **__doc__** special attribute of that object. String literals occurring elsewhere in Python code may also act as documentation and can be retrieve using a software

tool. The following two string literal types can't be retrieved using the runtime object `__doc__`.

- String literals occurring immediately after a simple assignment at the top level of a module, class, or `__init__` method is called “attribute docstrings”.
- String literals occurring immediately after another docstring are called “additional docstrings”.

This section highlights docstrings related to a function and docstring related to class and module is discussed in vol. 2 of this book. In python following ways we can prepare the docstrings:

- For consistency, always use `"""triple double quotes"""` around docstrings.
- To represent a backslash as part of the docstring use `r"""raw triple double quotes"""`.
- For Unicode docstrings, use `u"""Unicode triple-quoted strings"""`.

Note: A docstring is used to supply documentation for a function. It can contain the function's purpose, what arguments it takes, information about return values, or any other information you think would be useful.

8.11.1 One-line docstring

A one-liner docstring should really fit on one line, as follows:

```
>>> def function(arg):
...     """This is __doc__, function's docstring"""
...     pass
... 
```

Following list of points need to be taken care while writing a one-liner docstring:

- Triple quotes are used even though the string fits on one line. This makes it easy to later expand it as multi-liner docstring
- The closing quotes are on the same line as the opening quotes. This looks better for one-liners.
- There's no blank line either before or after the docstring.
- The docstring is a phrase ending in a period. It prescribes the function or method's effect as a command (“Do this”, “Return that”), not as a description.

```
>>> def fun2(a,b):
...     """perform addition of a and b and return the result"""
...     pass
... 
```

```
>>> print(fun2.__doc__)
perform addition of a and b and return the result
```

The runtime object `__doc__` is used to retrieve the docstring of a function, as shown in the output. The one-line docstring should NOT be a “signature” of a function reiterating the function/method parameters (which can be obtained by introspection). Don't do:

```
>>> def fun1(a, b):
...     """fun1(a,b) -> list"""
...     pass
... 
```

The `help()` function is one of the ways to do code introspection, as follows:

```
>>> help(fun1)
Help on function fun1 in module __main__:

fun1(a, b)
    fun1(a,b) -> list
```

8.11.2 Multi-line docstring

Multi-line docstrings consist of a summary line just like a one-line docstring, followed by a blank line, followed by a more elaborate description. An example such type is given below:

```
def getcomplex(real=0.0, imag=0.0):
    """Form a complex number.

    Keyword arguments:
        real: the real part (default 0.0)
        imag: the imaginary part (default 0.0)

    Returns:
        A complex number.
    """

    if imag == 0.0 and real == 0.0:
        return 0j
    elif imag == 0.0:
        return real + 0j
    else:
        return complex(real, imag)

# Print the docstring
print(getcomplex.__doc__)

# Print the result of calling getcomplex() with no arguments
print(getcomplex())

# Print the result of calling getcomplex() with real=3
print(getcomplex(3))

# Print the result of calling getcomplex() with real=4 and imag=5
print(getcomplex(4, 5))
```

Like single line docstring multi-liner is also retrieve by either `help()` function or special attribute `__doc__` as shown above. Further details about the docstring indentation, docstring inside a class or module read the documents provided as footnote^{23,24}.

²³ D. G. a. G. v. Rossum, "PEP 257 – Docstring Conventions," 29 5 2001. [Online]. Available: <https://peps.python.org/pep-0257/>.

²⁴ D. Goodger, "PEP 258 – Docutils Design Specification," 31 5 2001. [Online]. Available: <https://peps.python.org/pep-0258/#attribute-docstrings> . [Accessed 22 10 2022]

8.12 Topics Not Covered

To keep the discussion limited about python function and its features following list of topics are sifted to vol. 2 of this book.

- (a) lambda or anonymous function
- (b) function recursion
- (c) function annotations
- (d) Scope rules

*****END OF CHAPTER*****

ABOUT THE AUTHOR

Dr. Goutam Majumder is an esteemed academic and researcher in the field of Computer Science & Engineering. Born on August 29, 1986, in India, Dr. Majumder has made significant contributions to his field over a career spanning more than 13 years. He completed his Doctorate in Computer Science & Engineering from the National Institute of Technology Mizoram in 2021. Prior to his Ph.D., he earned his M.Tech from Tripura University (A Central University), ranking first in the Science & Technology Department, and his B.E. from the National Institute of Technology Agartala. He also completed PG Diploma in Advance Computing from C-DAC Acts, Pune. Dr. Majumder has held prestigious positions as an Assistant Professor at Vellore Institute of Technology (VIT) Vellore since January 2023 and previously at Lovely Professional University and NIT Mizoram. His research has been recognized with a Scopus ID and WoS ID, and he maintains an active presence on academic platforms like Google Scholar and ResearchGate. In addition to his academic pursuits, Dr. Majumder is a member of the Computer Society of India and has completed certifications in Java, Python, and Machine Learning from renowned institutions such as Oracle, Hackerrank, Coursera, and NPTEL. Dr. Majumder's dedication to education and research is evident in his extensive experience and certifications, marking him as a distinguished figure in the Computer Science community. He can be contacted at goutam.nita@gmail.com

Dr. Ganesh Khakare is currently working in the Department of Computer Science & Engineering at Vellore Institute of Technology, Vellore, India (Deemed University NIRF Rank 08). He has done BE, ME, and PhD in Computer Science and Engineering and currently pursuing Post Doc from the University of Lincoln, Malaysia. He is UGC net exam qualified with 98.15 percentiles. He has been a member of different Societies and professional bodies like ACM since 2014, a lifetime member of the ISTE, IEI, ISDS, and IEEE Senior Member, IAENG, IFERP, and IERD member since 2018. He has more than 15 years of teaching and research experience. His main research work focuses on Data Science, the Internet of Everything, Machine Learning, Networking, Artificial Intelligence, etc. He has published more than 100 research articles in reputed international journals and conferences including Thomson Reuters, ACM, IGI Global, Inderscience, Springer, IEEE, Taylor & Francis, etc. He has more than 1000 Google Scholar citations. He has done 10 patents and 20 copyrights including international Canadian copyright. Guided more than 50 UG as well as PG research projects, currently guiding 6 PhD research scholars. He has organized numerous international conferences, seminars, FDPs, and workshops. He has delivered several invited talks. He is an editorial board member, international advisory board member, associate editor, and guest editor for many International Journals. He serves as a reviewer for various reputed International Journals and Conferences. He participated in many international conferences worldwide as a core organizing committee member, technical program committee member, and Session Chair. Served as a resource person for more than 100 national and international events. Certified with more than 40 training programs from various NITs, IITs, and reputed industries like TCS, Persistent, etc. on the latest technologies. He can be contacted at khekare.123@gmail.com